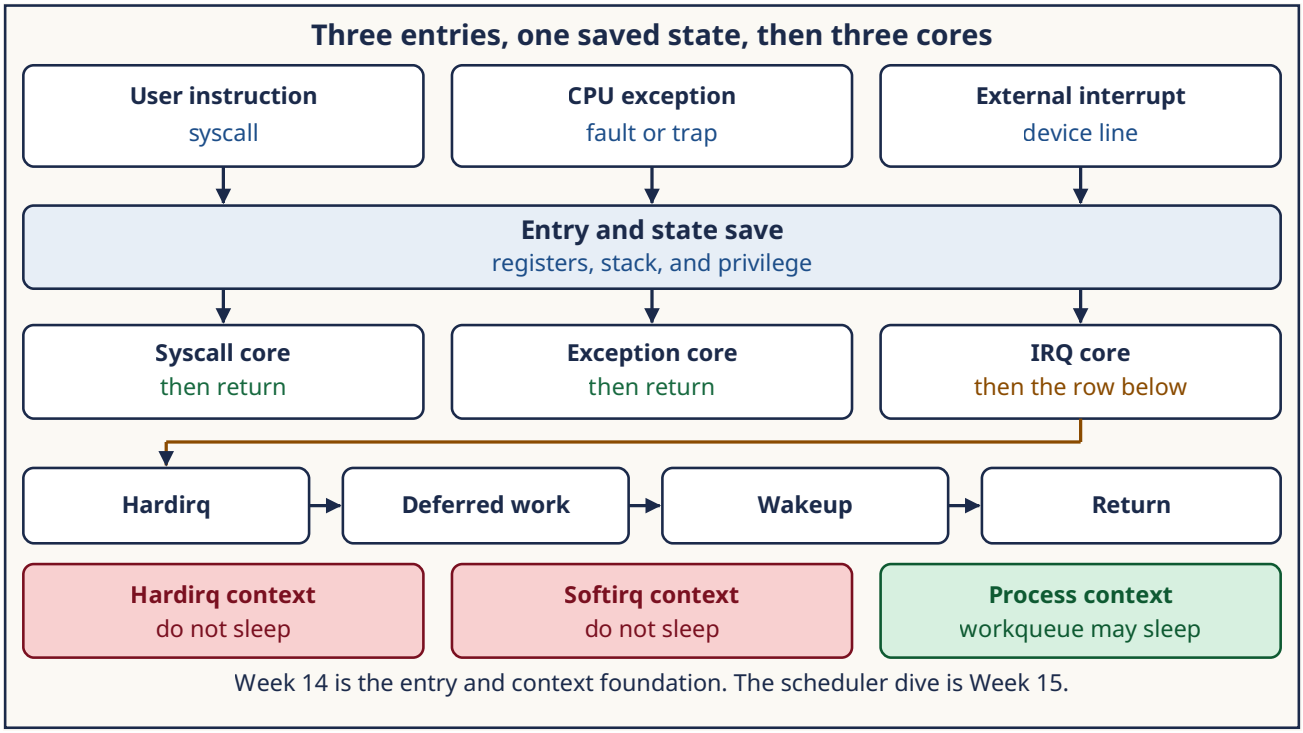
Week 14 — Kernel Entry, Contexts, and Interrupts

Format: seven daily sections collected into one weekly module

Suggested workload: approximately 21 focused hours, adjustable to mastery

> A syscall, a page fault, and a device interrupt all steal the CPU from the current instruction stream. The kernel has to save state, run the right core, and return without sleeping in a context that must stay atomic. Choosing the next task is a later week.

The lab is projects/linux-kernel-contexts. Wrappers build through its Makefile. Kernel names below were read from Linux 6.12, commit `adc218676eef25575469234709c2d87185ca223a`, whose top-level `Makefile` sets `VERSION = 6`, `PATCHLEVEL = 12`, `SUBLEVEL = 0`. The landmark script checks that snapshot and does not modify it. The host that runs the C labs may be another kernel; the wrappers print `uname`.



Required order: syntax, Linux 0.12, evolution, ToyOS, then 6.12

Each day opens in this order, and the order is required:

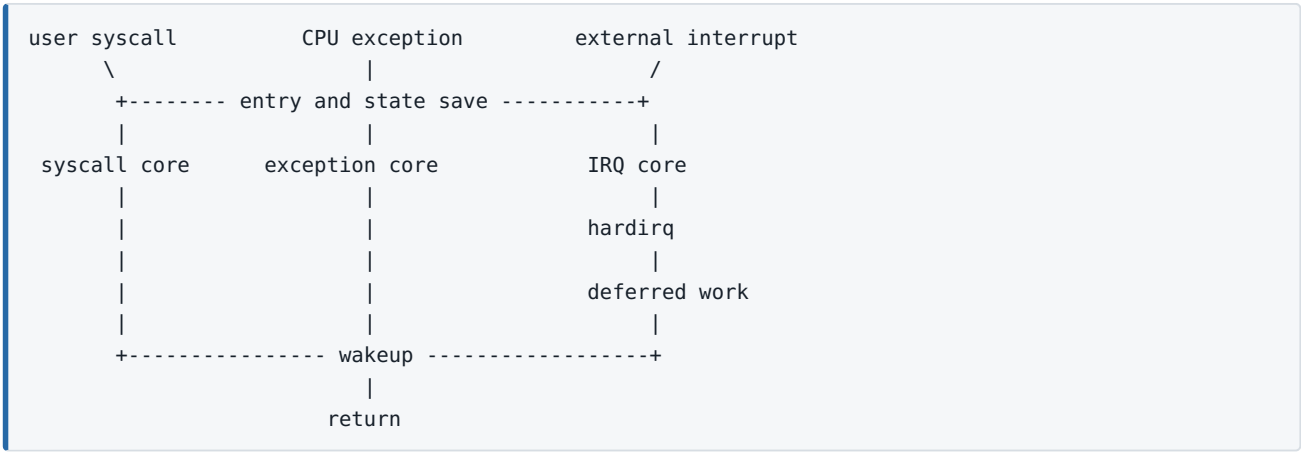
1. the few C or assembly forms that day's source actually uses;
2. the Linux 0.12 path, read while the whole entry still fits in one assembly file;
3. why that early design changed, in the constraint a single `iret`, a global `jiffies`, and a `cli`/`sti` pair could not keep;
4. the ToyOS reconstruction from Weeks 6–9;

5. the Linux 6.12 tree `adc218676eef25575469234709c2d87185ca223a`, then a check against current upstream before a symbol is treated as current.

Linux 0.12 is the first conceptual implementation. Its `int 0x80` entry, `sys_call_table`, page-fault split, and timer IRQ make the control flow visible. They are not the authority for `SYSCALL`/`SYSRET`, `irq_desc`, softirqs, workqueues, or `preempt_count`. Those belong to the 6.12 tree. The shared Linux 0.12 map is the reading method. This week adds the entry files that map does not already walk: `kernel/system_call.s`, `kernel/asm.s`, `kernel/sched.c`, and `mm/memory.c`.

Why these seven days belong together

Week 14 is the Linux kernel entry and context foundation. A full scheduler dive, including which task runs next, waits until Week 15. This week stops at how the CPU enters the kernel, which core runs, what may be deferred, and which contexts may sleep.



Hardirq and softirq context must not sleep. A workqueue worker and ordinary process context may sleep, unless they hold a lock that forbids it. The bands on the roadmap are that rule. They are not a scheduling class.

Daily roadmap

Day 1 — Kernel tree, Kconfig, kbuild, and the context map

Day 1 separates the snapshot (`VERSION`, `PATCHLEVEL`, `SUBLEVEL`), the options in `Kconfig`, and the build rules that compile a unit. The context map is which file owns syscall entry, faults, IRQ descriptors, softirqs, workqueues, and `preempt_count`. Wrapper: challenges/day-001-source-map.sh.

Day 2 — System-call entry and return

Day 2 follows `entry_SYSCALL_64` through `do_syscall_64` and `syscall_exit_to_user_mode`. The live x86-64 dispatch is `x64_sys_call`, a switch. `sys_call_table` remains for tracing and is not the call path. The C lab only shows that `syscall(SYS_getpid)` matches `getpid` and that a pipe byte completes. Wrapper: challenges/day-002-syscall-entry.sh.

Day 3 — Exceptions, page-fault entry, and return

Day 3 reads `exc_page_fault` and `do_user_addr_fault`. The lab's child stores through a `PROT_NONE` mapping and the parent observes `SIGSEGV`. That is the user symptom, not a trace of the fault handler. A second child catches `SIGUSR1`. Wrapper: challenges/day-003-exception-entry.sh.

Day 4 — Hardware interrupts, IRQ descriptors, and handlers

Day 4 reads `struct irq_desc`, the action list, and `__handle_irq_event_percpu`. `/proc/interrupts` is read only when the file can be opened. Counts are not assertions. Reading that file is not an IRQ handler. Wrapper: challenges/day-004-interrupts.sh.

Day 5 — Softirqs, threaded IRQs, and workqueues

Day 5 separates three deferrals: `__do_softirq`, `request_threaded_irq`, and `queue_work_on` into `worker_thread`. The lab's `eventfd`, `timerfd`, and `poll` are userspace analogs and are labeled as such. Wrapper: challenges/day-005-deferred-work.sh.

Day 6 — Preemption, atomic context, and sleepability

Day 6 reads the `preempt_count` masks and `might_sleep`. `in_atomic()` is not the driver test for "may I sleep". The lab prints the lesson table and shows that a zero-timeout `poll` returns while a short `timerfd` wait does not. That wait is not kernel atomic context. Wrapper: challenges/day-006-context-rules.sh.

Day 7 — Entry and interrupt tracing, and a debugging report

Day 7 names `trace_irq_handler_entry` and the irqsoff tracer in the tree, then records only what this host actually produced: `uname`, `/proc/interrupts` when readable, and `voluntary_ctxt_switches` when present. Optional `strace`, `perf`, and tracefs print `SKIP` when they cannot run. Tracefs is not enabled. Wrapper: challenges/day-007-entry-debug-report.sh.

Safety and evidence

The C programs do not create temporary files. The fault parent waits for both children and arms a three-second alarm. Poll timeouts are 0 or 500 ms. The landmark script is read-only; the suite hashes the kernel `Makefile` before and after and rejects a tree that is not this v6.12 commit.

`strace` and `perf` may be missing or blocked. A readable tracefs file is still a skip, because this lab does not turn a tracer on. A skip is not a failed core test and it is not evidence that the entry path was traced.

Completion checkpoint

Finishing the wrappers does not complete mastery. Answer from the source and the logs, and leave a question blank when the run did not speak to it:

1. Which files are the snapshot, the Kconfig choice, and the kbuild rule, and which file owns syscall entry versus `irq_desc`?
2. Which instruction returns with `SYSRET`, and which function is the live x86-64 syscall dispatch?

3. What did the child `SIGSEGV` prove, and what did it not prove about `exc_page_fault`?
4. What does `/proc/interrupts` show when it is readable, and why is that not a handler trace?
5. Which of softirq, threaded IRQ, and workqueue may sleep?
6. Why is `in_atomic()` the wrong test in driver code?
7. Which optional tools ran, which printed `SKIP`, and which host `uname` actually ran the lab?
8. Which names did you re-read in commit `adc218676eef25575469234709c2d87185ca223a`?

Do not auto-fill mastery, confidence, hours, or completion. The quizzes sit beside this overview and do not grade the report. Week 15 takes the scheduler from the wakeup this week only names.

Day 1 — The Tree, the Config Guard, and the Context

Target time: approximately 3 hours

- Syntax that hides a config choice or a function pointer: 30 minutes
- Linux 0.12's flat tree, then why Kconfig and `arch/` appeared: 45 minutes
- The pinned 6.12 map from symbol to object, and the context map: 50 minutes
- One navigation trace on the local tree: 25 minutes
- Mastery, then the wrapper: 30 minutes

> You have `arch/x86/entry/entry_64.S` open. Which commit is this file from, which config symbol decided the entry objects are linked, and which execution context is the CPU in when that code runs?

Weeks 6–9 built one small kernel whose files you can hold in your head. Weeks 10–12 followed processes, threads, and I/O through a modern `task_struct` and the VFS. The next path is the kernel itself: how the source is split, how a config symbol becomes a translation unit, and which context a line of that code is allowed to run in. Syscall entry is Day 2. Page-fault entry is Day 3. Interrupts, softirqs, and scheduling stay later. Today is the map you need before those paths are readable.

The excerpts below are from the local tree `adc218676eef25575469234709c2d87185ca223a`, tag `v6.12`, commit `adc218676eef`. Field layout and line numbers in a later mainline release can move. The names quoted here are what this revision contains.

1. Syntax needed for this source

Four notations show up before any entry path makes sense: a preprocessor guard, a macro that hides a guard, a function pointer, and an assembly label that the linker treats as a function.

A guard keeps a declaration out of the translation unit when a symbol is absent:

```
#ifdef CONFIG_SMP
void kick_remote_cpu(void);
#endif
```

`#ifdef` tests whether the name is defined. In this tree, `CONFIG_FOO=y` becomes `#define CONFIG_FOO 1` in the generated autoconf header. `CONFIG_FOO=m` becomes `#define CONFIG_FOO_MODULE 1` and leaves `CONFIG_FOO` itself undefined. A guard written `#ifdef CONFIG_FOO` therefore compiles the block for a built-in choice and drops it for a module choice.

`include/linux/kconfig.h` (lines 46–73) answers that with macros that expand to the integer 1 or 0, so they can sit inside an `if`:

```
#define IS_BUILTIN(option) __is_defined(option)
#define IS_MODULE(option) __is_defined(option##_MODULE)
#define IS_ENABLED(option) __or(IS_BUILTIN(option), IS_MODULE(option))
```

`IS_ENABLED(CONFIG_SMP)` is 1 when the symbol is `y` or `m`. `IS_BUILTIN` is 1 only for `y`. `IS_REACHABLE` (lines 65–66) is 1 when the caller can actually call the function: built-in code may not call a function that lives only in a module. The helper `__is_defined` (lines 41–43) works by pasting the option onto `__ARG_PLACEHOLDER_`. A defined option that expands to `1` matches `#define __ARG_PLACEHOLDER_1 0,` and the following argument-picking macro selects 1. An undefined option leaves junk in the first argument and the macro selects 0. The point of the trick is that the test is usable in preprocessor conditions and in ordinary C expressions.

A function pointer names code the way an integer names a file. The type in `arch/x86/include/asm/syscall.h` (lines 20–21) is the modern form of the 0.12 system-call table:

```
typedef long (*sys_call_ptr_t)(const struct pt_regs *);
extern const sys_call_ptr_t sys_call_table[];
```

`sys_call_table[i]` is an address of a function that receives the saved registers and returns a `long`. The caller does not spell the callee's name. The table slot does.

An initcall is the same idea with a section attribute. `core_initcall` in `include/linux/init.h` (line 300) stores a function pointer where the linker will gather it:

```
#define core_initcall(fn) __define_initcall(fn, 1)
```

The non-LTO form (lines 268–270) is a static pointer placed in a named section:

```
static initcall_t __name __used \
    __attribute__((__section__(__sec))) = fn;
```

Boot walks that section and calls each pointer. The C source never writes a central list of every subsystem's init function.

Assembly entry points use a macro so the symbol type is right for traces and for the linker. Day 2 reads `SYM_CODE_START(entry_SYSCALL_64)`. Today it is enough to see that the label is the address the CPU will fetch, and that a later `call do_syscall_64` is an ordinary call once the kernel stack and the saved registers exist.

2. Linux 0.12 first: one machine, one Makefile, one table

Linux 0.12 is still the whole kernel in a handful of directories: `boot/`, `init/`, `kernel/`, `mm/`, `fs/`, `lib/`, `include/`, and `tools/`, plus driver objects under `kernel/blk_drv` and `kernel/chr_drv`. There is no `arch/` tree. The machine is the 80386. The top Makefile names the compiler and then lists the archives that become the image: `kernel/kernel.o`, `mm/mm.o`, `fs/fs.o`, the driver archives, and `lib/lib.a`. `kernel/Makefile` lists the objects of that archive by hand, including `sched.o`, `system_call.o`, `traps.o`, and `fork.o`. A `.c` or `.s` file in that list is in the kernel. A file left off the list is not.

Configuration is a header of `#define` lines, not a menu language. The build does not generate a tristate `CONFIG_*` symbol that can mean built-in, module, or absent. Loadable modules, as 6.12 uses them, are not this kernel's problem.

The one indirect call that matters for the rest of the week is the system-call table. `kernel/system_call.s` receives `int 0x80` with the call number in `%eax` and the arguments already in registers. The entry rejects a number past the table, reloads the kernel data segments, and calls through the table:

```
system_call:
    cmpl $nr_system_calls-1,%eax
    ja bad_sys_call
    push %ds
    push %es
    push %fs
    pushl %edx
    pushl %ecx      # push %ebx,%ecx,%edx as parameters
    pushl %ebx      # to the system call
    movl $0x10,%edx     # set up ds,es to kernel space
    mov %dx,%ds
    mov %dx,%es
    movl $0x17,%edx     # fs points to local data space
    mov %dx,%fs
    call sys_call_table(,%eax,4)
    pushl %eax
```

`nr_system_calls` is 72, so `%eax` above 71 goes to `bad_sys_call`, which returns `-ENOSYS` with `iret`. The addressing mode `sys_call_table(,%eax,4)` is base plus index times four: each slot is a 32-bit function pointer. `include/linux/sys.h` defines that array, `fn_ptr sys_call_table[]`, starting `sys_setup`, `sys_exit`, `sys_fork`, `sys_read`, `sys_write`. After the call, the same file checks `current->state` and `current->counter` and may `jmp schedule` before `iret`.

Page faults already have their own assembly entry, `mm/page.s`. Day 3 reads it. The point today is the shape: hardware transfers to a label, the label builds a kernel context, C does the work, `iret` returns.

What carries forward is the directory split, the idea that a build list selects objects, and a function-pointer table indexed by a number the user put in a register. The historical parts are the single architecture, the hand-written object list, the segment reloads (`0x10` kernel data, `0x17` local data), the 32-bit `int 0x80` frame, and a compile-time table of 72 calls.

The week 10–12 0.12 map indexes processes and files. This day's early files are `kernel/system_call.s`, `include/linux/sys.h`, `kernel/Makefile`, and `mm/page.s`. Develop against the 6.12 tree below.

3. Evolution: why the flat tree and the `#ifdef` stopped scaling

Three pressures broke the 0.12 picture.

The first is a second architecture. A kernel that also runs on a different instruction set cannot keep `system_call.s` and `page.s` as the only entry files in a shared `kernel/` and `mm/`. The instruction that enters the kernel, the register that holds the call number, and the page-fault status word are architecture facts. Scheduling policy, the page-fault resolver, and the filesystem are not. The split that survives is `arch/<name>/` for the machine, and `kernel/`, `mm/`, `fs/`, `net/`, `drivers/`, and `lib/` for work that should read the same way on every machine. Shared declarations that every architecture must supply live as names in `include/asm-`

`generic/Kbuild` (`mandatory-y`, starting at line 7: `atomic.h`, `barrier.h`, `current.h`, and the rest). An architecture provides the header or the build generates it from the generic one. That is the boundary Day 2 and Day 3 keep crossing: `kernel/entry/common.c` is the shared enter/exit work, and `arch/x86/entry/entry_64.S` is the x86-64 instruction sequence.

The second pressure is the option that is neither always-on nor always-off. A driver can be built in, built as a module, or left out. A shell script that rewrote a header of `#define` lines could not express the dependency tree, the module case, or a menu that hides options whose parent is off. The language that this tree actually uses is documented in `Documentation/kbuild/kconfig-language.rst`. A symbol is a `bool` or a `tristate`. `depends on` hides a child when the parent is off. The top `Kconfig` (lines 6–32) only pulls other files in:

```
mainmenu "Linux/$(ARCH) $(KERNELVERSION) Kernel Configuration"
source "init/Kconfig"
source "kernel/Kconfig.freezer"
source "mm/Kconfig"
source "fs/Kconfig"
source "drivers/Kconfig"
```

The selected values land in `.config` and in `include/generated/autoconf.h`. `IS_ENABLED` exists because a module choice must still be visible to C that is compiled into the core, while `#ifdef CONFIG_FOO` stays false for that choice.

The third pressure is the object list. Typing every `.o` into one Makefile ties the image to a single machine and a single configuration. Kbuild's rule, in `scripts/Makefile.lib` (lines 28–43), is smaller than a build tutorial and is the whole idea: a directory name written in `obj-y` becomes that directory's `built-in.a`, and `obj-m` contributes a module. `kernel/Makefile` shows the local form (lines 6–17):

```
obj-y     = fork.o exec_domain.o panic.o \
            ...
            signal.o sys.o umh.o workqueue.o pid.o task_work.o \
            ...
obj-$(CONFIG_USERMODE_DRIVER) += usermode_driver.o
obj-$(CONFIG_MULTIUSER) += groups.o
```

`fork.o` is in the built-in kernel. `usermode_driver.o` is compiled when `CONFIG_USERMODE_DRIVER` is enabled. The top Makefile still records the architecture selection that chooses which `arch/` tree participates (lines 381–392): `ARCH ?= $(SUBARCH)`, then `SRCARCH` folds `i386` and `x86_64` onto `arch/x86`. The link of `vmlinux` uses `arch/$(SRCARCH)/kernel/vmlinux.lds` (line 1126). This day stops at that relationship. Recipe order, `syncconfig`, modules_install, and packaging are logistics around the same idea, and they are not the concept.

What survived from 0.12 is a directory per job, a build list that selects objects, and a function pointer where the set of callees is data. What changed is who is allowed to add an object: a Kconfig symbol and an `obj-$(CONFIG_...)` line, with architecture code behind `arch/$(SRCARCH)`.

4. ToyOS reconstruction

ToyOS is the 0.12 shape with the lessons of Weeks 6–9 kept explicit. The files are few enough to name: `src/arch.c` builds the GDT and IDT, `src/isr.S` is the entry assembly, `src/process.c` is the syscall dispatch, `src/memory.c` is the user-copy check. There is no Kconfig language. The config guard is a C macro, `CONFIG_MILESTONE`, tested with `#if` so that a later week can compile more of the same tree.

The function-pointer table is `isr_stub_table` in `src/isr.S` (lines 105–110), one address per exception vector. `arch_init` in `src/arch.c` (lines 207–213) stores those addresses in the IDT. Vector 128 is installed only when the milestone allows user syscalls, and its gate attribute is `0xee` so ring 3 may enter. Vector 32 is the timer. The C function `interrupt_handler` then branches on `frame->vector`: 32 ticks the scheduler, 128 calls `syscall_dispatch`, 14 reports a page fault. That `if` on a vector number is the same design as a table of function pointers, written out because the teaching kernel has three cases.

Contexts in ToyOS are still few, and that is why they are visible. User code runs at ring 3 with its own page-table root. Any entry through the IDT runs `isr_common` on the kernel stack whose top was stored in `tss.rsp[0]` by `arch_set_kernel_stack`. The handler returns with `iretq`. There is no softirq, no workqueue, and no NMI stack policy. When a later Linux function says it may run only `in_task()`, ToyOS has exactly one such kernel context: the path that entered for this process and will `iretq` back to it.

5. Linux 6.12: pin the tree, then follow a symbol to an object

Start with the version, or every later line number is unverifiable. The top of `Makefile` is the contract:

```
VERSION = 6
PATCHLEVEL = 12
SUBLEVEL = 0
EXTRAVERSION =
NAME = Baby Opossum Posse
```

`git rev-parse HEAD` in `/home/jd/Documents/linux-6.12` prints `adc218676eef25575469234709c2d87185ca223a`. `git describe --exact-match` prints `v6.12`. A path cited from a different major line, or from whatever kernel happens to be installed, is a different program. The installed running kernel and this source tree are allowed to differ. Quotes in this week name the tree.

The directories you actually open:

```
arch/x86/entry/     x86-64 syscall and exception assembly
arch/x86/mm/        x86 page-fault classification
arch/arm64/kernel/  AArch64 vectors and syscall dispatch
arch/riscv/kernel/  RISC-V trap entry and scause decode
kernel/entry/       shared enter-from-user and exit-to-user work
kernel/             signals, forks, softirqs, the task
mm/                 handle_mm_fault and the address space
fs/                 read, write, open, once the syscall has entered
include/linux/      shared C headers, including kconfig.h and preempt.h
include/asm-generic/ headers every architecture owes
Documentation/kbuild/ the Kconfig and Kbuild languages
```

`arch/x86/Kconfig` selects the shared entry implementation. Inside `config X86` (line 164):

```
select GENERIC_ENTRY
```

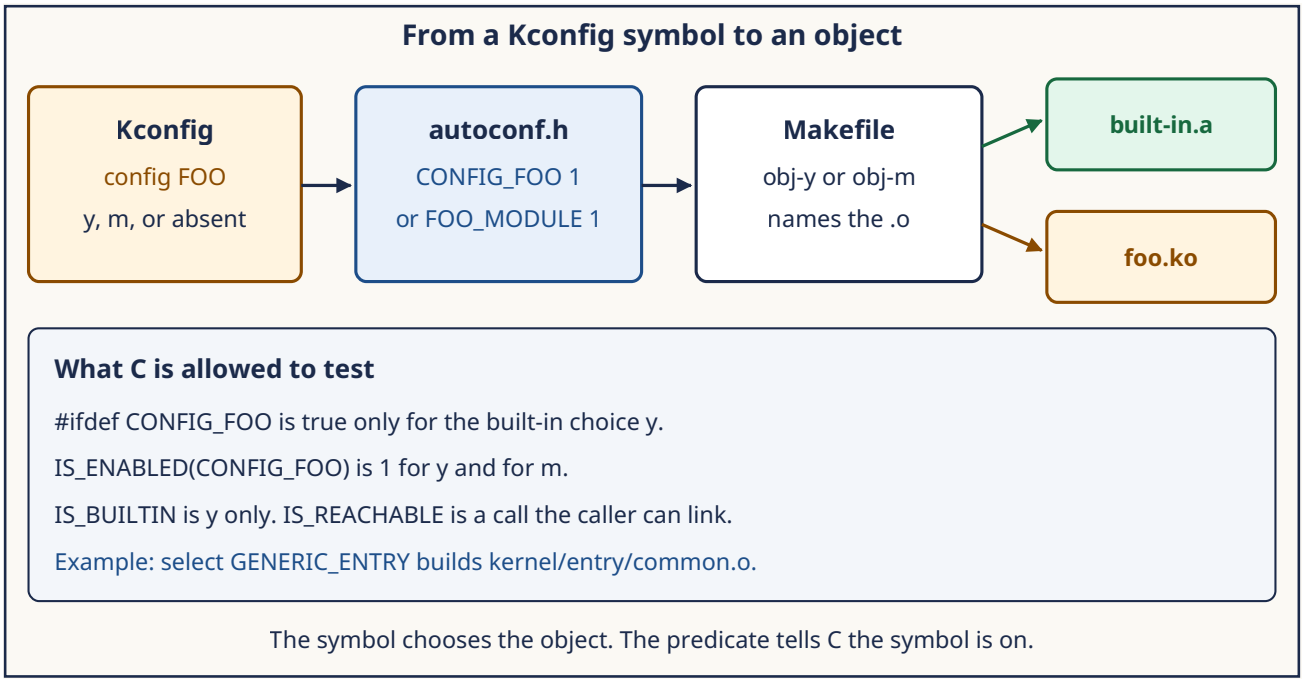
`kernel/entry/Makefile` turns that symbol into objects (line 12):

```
obj-$(CONFIG_GENERIC_ENTRY) += common.o syscall_user_dispatch.o
```

`common.o` is `kernel/entry/common.c`, which defines `syscall_trace_enter` and `exit_to_user_mode_loop`. The architecture Makefile `arch/x86/entry/Makefile` (lines 14–15) always builds the x86 entry objects:

```
obj-y := entry.o entry_$(BITS).o syscall_$(BITS).o
obj-y += common.o
```

`BITS` is 32 or 64, so a 64-bit build compiles `entry_64.o` from `entry_64.S` and `syscall_64.o` from `syscall_64.c`. The Kconfig choice `64BIT` in `arch/x86/Kconfig` (lines 3–8) is what makes `ARCH=x86_64` a 64-bit kernel. The chain is the diagram: a symbol, a generated macro, an `obj-*` line, an object in the link.



A `y` symbol compiles into `built-in.a`. An `m` symbol compiles as a module. An absent symbol produces no object. `IS_ENABLED` sees both `y` and `m`.

Two headers are the daily navigation tools. `include/linux/kconfig.h` is the guard language from section 1. `include/linux/sys.h` in this same tree is the historical name with the body removed: the comment on lines 5–7 says the file is no longer used. The live table is generated under `arch/x86/entry/`. When a 0.12 habit says "open `include/linux/sys.h` for the call table," this revision sends you elsewhere. That is the version discipline in one file.

Source discipline for the rest of the week:

1. The stable idea is the one you can say without a version: a number selects a function, a fault status word selects a handler, a task has a kernel stack.
2. The historical detail stays in the 0.12 quote: segment numbers, `int 0x80`, a 72-entry array written out in a header.
3. The 6.12 check is a path and a line in commit `adc218676eef`. If the line is not there, the explanation is wrong for this tree.

6. The execution-context map

A path is not readable until you know which context it may run in. `include/linux/preempt.h` (lines 118–133) is the map the kernel itself uses. The counters live in the preemption count:

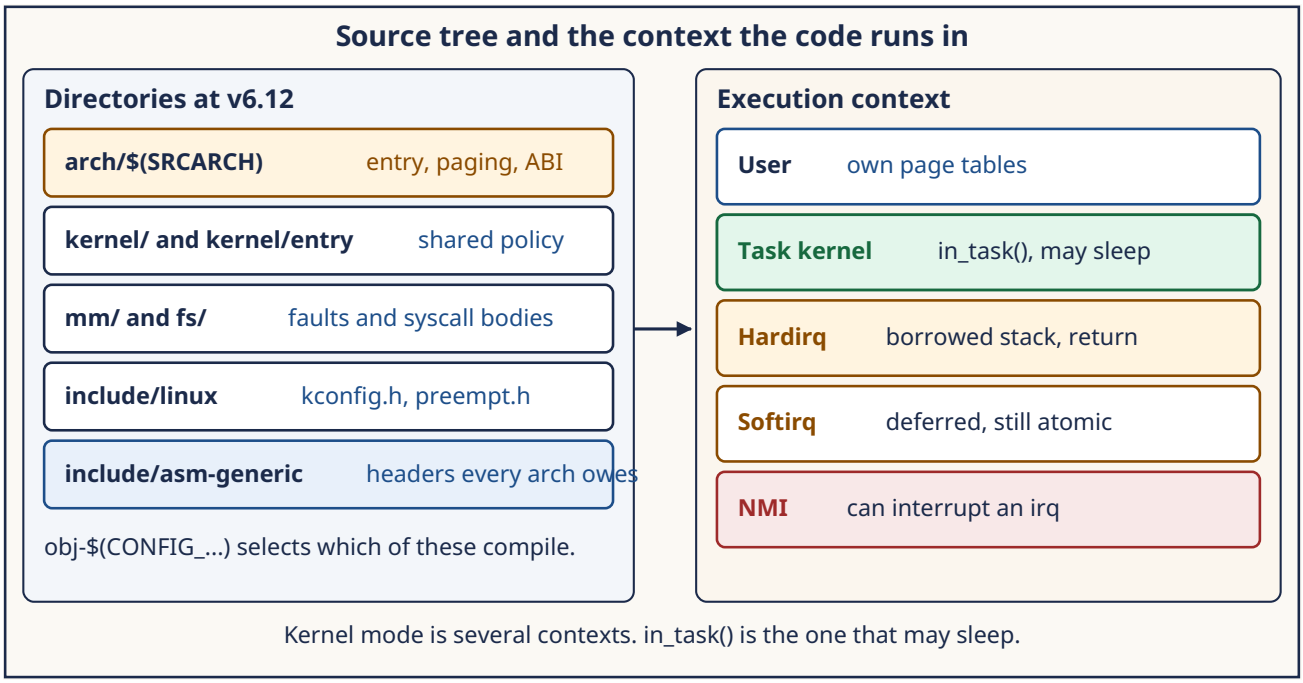
```
#define in_nmi()               (nmi_count())
#define in_hardirq()           (hardirq_count())
#define in_serving_softirq()   (softirq_count() & SOFTIRQ_OFFSET)
#define in_task()              (!(preempt_count() & (NMI_MASK | HARDIRQ_MASK | SOFTIRQ_OFFSET)))
```

The `in_task` line above is the non-PREEMPT_RT form. PREEMPT_RT keeps softirq state in a different word; the meaning of the predicates stays the one the comment states.

```
Context            How you entered                What the code may do
------             ----------------                --------------------
User               ordinary instructions          its own address space
Task, kernel       syscall, fault, exception      sleep, lock, copy user
                   kthread, workqueue             memory, take mmap_lock
Hardirq            device interrupt               finish and return;
                                                  the interrupted context
                                                  is waiting on this stack
Softirq            raised by an interrupt, run    deferred work that is
                   before leaving irq, or by      still atomic with
                   a ksoftirqd task               respect to sleeping
NMI                non-maskable interrupt         almost nothing that
                                                  an interrupted irq
                                                  might already hold
```

A syscall runs `in_task()`. It is kernel mode, on that task's kernel stack, and the scheduler may switch away from it if it sleeps. A hardirq is a different context. The CPU borrowed a stack, `in_hardirq()` is true, and the handler's job is to return to the interrupted context. Sleeping there leaves the interrupted context stacked underneath a context that will not be scheduled. Softirq is the deferred half of that idea: the interrupt acknowledges the device and raises work that runs soon after, still with sleeping disabled unless a ksoftirqd thread has taken over. NMI can interrupt an interrupt. Code that runs there cannot take a lock an IRQ handler might hold.

Day 2's `do_syscall_64` is task context. Day 3's page fault is usually task context too, including a fault taken while a syscall copies from user memory. A fault with `faulthandler_disabled()` or with no `mm` is the kernel telling you this context must not sleep inside the fault handler. Those predicates are Day 3. The map is today's.



Generic directories hold policy. `arch/` holds the entry sequence. The context predicates in `preempt.h` decide what that code may do.

The kernel stack belongs to the context. On x86-64 a syscall switches to the task kernel stack in software, because the `SYSCALL` instruction does not. An interrupt may use an interrupt stack. ToyOS has the small version: one `tss.rsp[0]` per entry from user mode. The 6.12 picture adds per-CPU stacks and per-context counters so a function can ask which context it is in without guessing from the call chain.

7. WHY the boundaries are where they are

The directory boundary exists so a change to the page-fault resolver in `mm/` can be reviewed once, while the line that reads `CR2` stays in `arch/x86/mm/fault.c` because only that architecture has `CR2`. Putting the resolver in `arch/x86` would fork it for AArch64 and RISC-V. Putting the `CR2` read in `mm/` would make `mm/` false on every other machine.

The Kconfig boundary exists so the image you boot is a subset of the tree you cloned. `obj-y` is the subset you cannot turn off in this Makefile. `obj-$(CONFIG_FOO)` is the subset a symbol controls. The preprocessor then needs `IS_ENABLED` so C can mention a function that might live in a module without pretending the symbol is a plain `#define CONFIG_FOO`.

The context boundary exists because the CPU has one privilege state and many reasons to be in it. Kernel mode alone does not tell you whether sleeping is possible. `in_task()` does. Entry code is the code that makes those predicates true or false at the right moment. Day 2 shows that work on the syscall path. Day 3 shows it on the fault path.

8. Execution trace: opening this tree on purpose

This trace is a reading order, not a boot.

```
1. Confirm the commit.
   git -C /home/jd/Documents/linux-6.12 rev-parse HEAD
   adc218676eef25575469234709c2d87185ca223a
   git describe --exact-match HEAD  →  v6.12
   Makefile lines 2–6: VERSION 6, PATCHLEVEL 12, SUBLEVEL 0

2. Confirm which architecture tree a host build would enter.
   Makefile lines 381–392: ARCH defaults to the host sub-architecture.
   x86_64 and i386 both set SRCARCH=x86, so the files live in arch/x86.

3. Confirm the entry objects.
   arch/x86/Kconfig line 164 selects GENERIC_ENTRY.
   kernel/entry/Makefile line 12 builds common.o from that symbol.
   arch/x86/entry/Makefile lines 14–15 build entry_64.o and syscall_64.o
   when BITS is 64.

4. Confirm the context vocabulary before reading entry_64.S.
   include/linux/preempt.h lines 126–132: in_nmi, in_hardirq,
   in_serving_softirq, in_task.

5. Stop at the first instruction of the syscall path, and leave it
   for Day 2.
   arch/x86/entry/entry_64.S line 87: SYM_CODE_START(entry_SYSCALL_64)
   The comment on lines 62–66 records what the CPU has already done:
   rip is in rcx, rflags is in r11, and rsp is still the user stack.
```

At step 5 you know the file is in this commit, you know why it was linked, and you know the context you are about to enter is a transition from user mode into task context on the kernel stack. You do not yet need the `SYSRET` conditions or the seccomp order. Those are the next day's trace.

9. Safe evidence

Run `../challenges/day-001-source-map.sh` after the explanation. The wrapper prints the commit, the version lines, and the landmarks above. It reads source. It does not configure, compile, or boot a kernel, and it does not change the tree.

Record four lines in your notes:

- the full commit and `v6.12`;
- one `obj-$(CONFIG_...)` line and the Kconfig symbol that feeds it;
- the `in_task` predicate and the context it describes;
- the first label of `entry_SYSCALL_64`, left unread past the stack switch.

A grep hit is evidence that the file in this revision matches the notes. It is not evidence that a particular distribution kernel was built with that symbol set to `y`. The running kernel's config is `/proc/config.gz` or `/boot/config-$(uname -r)` when those files exist, and it is a different artifact from this source pin.

10. Architecture lens — x86-64, AArch64, and RISC-V

The generic directories (`kernel/entry`, `mm`, `fs`) are the ones the three architectures share. The entry instruction is not shared.

On x86-64 the system-call entry label is `entry_SYSCALL_64` in `arch/x86/entry/entry_64.S`. Exceptions use an IDT gate whose stub the `idtentry` macro emits in the same file. The page-fault status word is an error code pushed by the CPU, with the bit names in `arch/x86/include/asm/trap_pf.h`. Day 2 and Day 3 read those files. The architecture fact is: `SYSCALL` loads `rip` from an MSR and leaves `rsp` alone, so the kernel switches stacks itself.

On AArch64 the vector table is `vectors` in `arch/arm64/kernel/entry.S` (lines 519–530), sixteen slots for exception level, stack, and synchronous versus IRQ, FIQ, and SError. A synchronous exception from 64-bit EL0 lands in `el0t_64_sync_handler`, which reads `esr_el1` and switches on the exception class. The class `ESR_ELx_EC_SVC64` is the system call. A data abort is a different class. The register width, the exception level, and the syndrome register are AArch64 facts. The call into `kernel/entry` for enter-from-user and exit-to-user is the shared policy.

On RISC-V one label, `handle_exception` in `arch/riscv/kernel/entry.S` (line 95), receives traps. The cause CSR's sign bit distinguishes an interrupt from an exception (`bge` at line 183). The low bits index `excp_vect_table` (lines 430–447). Cause 8 is the user environment call. Causes 12, 13, and 15 are instruction, load, and store page faults. There is no single page-fault cause and no IDT. The shared C after the table lookup is still `do_trap_ecall_u` into the generic syscall enter path, and `do_page_fault` into the generic fault resolver.

Each architecture has a hardware status word and a kernel label. The 6.12 tree puts the status word's decode in `arch/` and the policy that runs after the decode in `kernel/` and `mm/`. `IS_ENABLED` and `obj-$(CONFIG_...)` are how a build keeps the other two architectures' objects out of an x86 image.

11. Mastery: Explain · Draw · Observe · Build

Explain

The 0.12 kernel is a handful of directories, a handwritten object list, and `sys_call_table(,%eax,4)`. Linux 6.12 at `adc218676eef` keeps the directory split, replaces the object list with Kconfig plus `obj-$(CONFIG_...)`, and puts machine entry under `arch/$(SRCARCH)`. `IS_ENABLED` is how C sees a `y` or `m` choice. `in_task`, `in_hardirq`, `in_serving_softirq`, and `in_nmi` are how a function knows which context it is in. A syscall is task context on the task's kernel stack. A hardirq is a borrowed stack that must return.

Draw

1. The directory map: `arch/x86`, `kernel/entry`, `mm`, `fs`, and an arrow from a syscall label in `arch/` into `do_syscall_64` and then into `mm` or `fs`.
2. The config chain: `select GENERIC_ENTRY` → `CONFIG_GENERIC_ENTRY` → `obj-$(CONFIG_GENERIC_ENTRY) += common.o` → `kernel/entry/common.c`.
3. Four contexts stacked: user, task-kernel, hardirq, NMI, with the predicate that is true in each.

Observe

On the wrapper's output, point at the commit and at one line number you can open. Change nothing. If a quoted line is not in your tree, stop and re-check `git rev-parse HEAD` before you trust any later day.

Build

Add a one-page index to your notes, in your own words, with these columns: path, one symbol you will look up there, and the context that code runs in. Include `entry_64.S`, `kernel/entry/common.c`, `arch/x86/mm/fault.c`, `arch/arm64/kernel/entry-common.c`, and `arch/riscv/kernel/entry.S`. You will use the index on Days 2 and 3.

Why? notebook

1. Why does `SRCARCH` fold `x86_64` and `i386` into `arch/x86`?
2. Why does `#ifdef CONFIG_FOO` stay false when the symbol is `m`?
3. Why is `IS_REACHABLE` a different test from `IS_ENABLED`?
4. Why is `include/linux/sys.h` almost empty in this revision?
5. Why does a second architecture force an `arch/` directory?
6. Why is "we are in kernel mode" too weak a statement to justify sleeping?
7. Why does ToyOS install vector 128 with attribute `0xee` and the exception vectors with `0x8e`?
8. Why does the source pin stay on `v6.12` when a newer mainline exists?

Next: Day 2 follows `entry_SYSCALL_64` from the `SYSCALL` instruction through `do_syscall_64` and back with `SYSRET` or `IRET`.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Source Map

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 14-kernel-entry-contexts-and-interrupts/challenges/day-001-source-map.sh
```

- View lab source on GitHub

Day 2 — System-Call Entry and Return

Target time: approximately 3 hours

- The assembly frame and the table: 35 minutes
- Linux 0.12 `int 0x80`, then the fast entry and the exit work: 45 minutes
- `entry_SYSCALL_64` through `do_syscall_64` and back: 55 minutes
- One trace of a `write`, then the three ABIs: 30 minutes
- Mastery, then the wrapper: 25 minutes

> A user program executes `SYSCALL` with `1` in `%rax`. Where is the kernel stack, who may change the call number before the function runs, and which instruction returns to user mode?

Day 1 put `entry_SYSCALL_64` on the map and left it unread. This day reads it. The path is the same idea as Linux 0.12: a deliberate entry from user mode, a number, a table, a C function, a return that restores user state. The 6.12 path adds a software stack switch, a `pt_regs` image, seccomp and audit before the call, and a return that uses `SYSRET` only when the saved user state still matches what `SYSRET` can restore.

The excerpts are from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. Line numbers belong to that tree.

1. Syntax needed for this source

The entry file is assembly with macros. `SYM_CODE_START(name)` emits a global code symbol. `pushq $__USER_DS` pushes an immediate. `pushq %r11` pushes a register. `movq PER_CPU_VAR(...), %rsp` loads a per-CPU address into the stack pointer. `movslq %eax, %rsi` sign-extends the 32-bit call number into the second argument register. `call do_syscall_64` is an ordinary call once `%rsp` is the kernel stack and `%rdi` points at the saved registers.

The saved image is `struct pt_regs`. On this path the pushes build it from the top of the frame downward: `ss`, `sp`, `flags`, `cs`, `ip`, `orig_ax`, then the general registers. Later C reads `regs->orig_ax` for the number the user placed in `%rax`, and `regs->ax` for the value that will be returned. Those two fields are different on purpose. `orig_ax` remembers the request. `ax` becomes the result.

A macro hides the C function that the table calls. `SYSCALL_DEFINE3(read, ...)` in `include/linux/syscalls.h` (lines 223–234 and 243–261) expands into metadata plus a real function `__do_sys_read`, and an alias `sys_read`. The x86 wrapper in `arch/x86/include/asm/syscall_wrapper.h` (lines 223–235) adds `__x64_sys_read(const struct pt_regs *)`, which pulls arguments out of the frame. The argument order for the 64-bit ABI is fixed at lines 56–59:

```
#define SC_X86_64_REGS_TO_ARGS(x, ...)              \
    __MAP(x,__SC_ARGS                   \
        ,,regs->di,,regs->si,,regs->dx          \
        ,,regs->r10,,regs->r8,,regs->r9)
```

The fourth argument is `regs->r10`, not `regs->cx`. The `SYSCALL` instruction itself writes the return address into `%rcx`, so the ABI passes the fourth argument in `%r10`. The stub moves it into the place the C function expects.

The table is no longer an array indexed in assembly. `x64_sys_call` is a switch whose cases are generated from the same list that used to be the array. A function pointer is still the idea. The control flow is a `switch` so the compiler can see every target.

2. Linux 0.12 first: `int 0x80` and `sys_call_table`

In 0.12 the user executes `int $0x80` with the number in `%eax` and the arguments in `%ebx`, `%ecx`, and `%edx`. The CPU pushes the user flags, the user `cs`, and the user `eip`, looks up IDT vector 128, and enters `system_call` in `kernel/system_call.s` at kernel privilege. The handler checks the number, reloads segments, and calls the table:

```
system_call:
    cmpl $nr_system_calls-1,%eax
    ja bad_sys_call
    push %ds
    push %es
    push %fs
    pushl %edx
    pushl %ecx      # push %ebx,%ecx,%edx as parameters
    pushl %ebx      # to the system call
    movl $0x10,%edx     # set up ds,es to kernel space
    mov %dx,%ds
    mov %dx,%es
    movl $0x17,%edx     # fs points to local data space
    mov %dx,%fs
    call sys_call_table(,%eax,4)
    pushl %eax
    movl current,%eax
    cmpl $0,state(%eax) # state
    jne reschedule
    cmpl $0,counter(%eax)   # counter
    je reschedule
```

`nr_system_calls` is 72. `%eax` above 71 takes `bad_sys_call`, which sets `%eax` to `-ENOSYS` and executes `iret`. A legal number uses `sys_call_table(,%eax,4)`: the slot is a 32-bit function pointer, and the three pushed registers are the C arguments. `include/linux/sys.h` writes the array out, beginning `sys_setup`, `sys_exit`, `sys_fork`, `sys_read`, `sys_write`, `sys_open`, `sys_close`.

The return path pops the saved registers and executes `iret`, which restores `eip`, `cs`, and flags from the CPU's interrupt frame. Before that, the code may jump to `schedule` when the task is no longer runnable or its counter is zero, and it may call `do_signal` when a signal is pending and the interrupted frame was user mode. The durable idea is already here: enter, range-check the number, call through a table, then consider reschedule and signals before returning to user mode.

What stays behind in 0.12 is the mechanism that implements the idea. The gate is `int 0x80`. The CPU switches stacks using the TSS. The kernel reloads `%ds`, `%es`, and `%fs` by hand. The table has 72 entries and lives in a header. There is one ABI. There is no seccomp, no audit, and no separate `SYSRET` path, because the 80386 has `iret` for this return.

3. Evolution: fast entry, several tables, work before and after the call

`int 0x80` still works as a compatibility entry, and it is a poor fit for the common case. The instruction behaves like a general software interrupt: it can arrive from any vector the IDT allows, it pushes a frame, and it returns with `iret`. A dedicated instruction can do less. On x86-64, `SYSCALL` saves the user `rip` in `%rcx`, saves `rflags` in `%r11`, masks a set of flag bits, and loads `cs` and the kernel `rip` from MSRs. It does not write the stack and it does not change `%rsp`. The kernel switches stacks in software and builds the `pt_regs` image the rest of the kernel already expects. `SYSRET` reverses the dedicated path. It is faster, and it restores a smaller, stricter set of state than `iret`. That strictness is why 6.12 sometimes refuses `SYSRET` and falls back to the full return.

The single array also stopped being the whole ABI. A 64-bit process, a 32-bit process on a 64-bit kernel, and the x32 ABI do not share numbers or register assignments. The numbers live in `arch/x86/entry/syscalls/syscall_64.tbl`. The generated header is included twice: once to declare `__x64_*` functions, once to emit the `switch` inside `x64_sys_call`. `include/linux/sys.h` in this revision says it is no longer used. The 0.12 file and the 6.12 file share a name and not a role.

Work accumulated on both sides of the call. A tracer must be able to change the number. Seccomp must see the number after the tracer. Audit must record the arguments that will actually be used. A tracepoint must run where probes can still change the number. That order is policy, and it is shared, so it moved to `kernel/entry/common.c` behind `CONFIG_GENERIC_ENTRY`. The architecture file keeps the instructions. The generic file keeps the checks.

The return accumulated the same way. Signals, `ptrace` stops, and `TIF_NEED_RESCHED` have to run before user mode resumes, with interrupts briefly enabled and then disabled again so the check is not stale. `exit_to_user_mode_loop` is that loop. 0.12's `reschedule` and `do_signal` tests are the ancestor. The modern loop also covers uprobes, live-patch state, and resume work.

Hardening sits on the fast return. `SYSRET` with a non-canonical `%rcx` has faulted in kernel mode on real CPUs, and a user program chooses the address that will land in `%rcx`. The C helper therefore returns false when the saved user state is not a clean 64-bit return, and the assembly takes the `iret` path instead. A bounds-checked call number uses `array_index_nospec` so a speculated index cannot pull a table target from outside the table.

4. ToyOS reconstruction

ToyOS keeps the 0.12 instruction on purpose. Week 8's ABI is a DPL-3 interrupt gate, not `SYSCALL`/`SYSRET`. `arch_init` in `projects/toyos/src/arch.c` (lines 212–213) installs vector 128 only when `CONFIG_MILESTONE >= 8`:

```
set_idt_gate(128, syscall_int_stub, 0, 0xee);
```

Attribute `0xee` is an interrupt gate that ring 3 may use. The other exception gates use `0x8e`, which ring 3 may not use. `syscall_int_stub` in `src/isr.S` (lines 59–63) pushes a zero error code and the vector 128, then falls into `isr_common`, which pushes the general registers and calls `interrupt_handler` with `%rdi` pointing at that frame. `interrupt_handler` sees vector 128 and calls `syscall_dispatch`.

`syscall_dispatch` in `src/process.c` (lines 184–262) switches on `frame->rax`. The teaching numbers are an `enum` in the same file: `SYS_WRITE` is 1, `SYS_GETPID` is 2, `SYS_READ` is 6. Linux x86-64 also uses 1 for `write` and uses 0 for `read` and 39 for `getpid`. The match on `write` is a coincidence of two small integers. The contract is the enum, not the Linux table. A missing number returns `-38`, the same `-ENOSYS` value 0.12 used. `SYS_WRITE` copies from the user pointer in `frame->rsi` through `copy_from_user` before it looks at the bytes. The length is capped by `MAX_IO`. The result is written back to `frame->rax`. `isr_common` pops the registers and executes `iretq`.

That is the whole mechanism in one screen: hardware entry, saved registers, a number, a C switch, a checked copy, a result in `%rax`, `iretq`. Linux 6.12 adds the fast instruction, the generic enter/exit work, and the conditions on `SYSRET`. It does not replace the idea.

5. Linux 6.12 x86-64: from `SYSCALL` to `do_syscall_64`

`arch/x86/entry/entry_64.S` documents the hardware contract in the comment at lines 62–77, then implements it. The instruction has already put the user `rip` in `%rcx` and the user flags in `%r11`. `%rsp` is still the user stack. `%rax` is the call number. Arguments are in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, and `%r9`.

```
SYM_CODE_START(entry_SYSCALL_64)
    UNWIND_HINT_ENTRY
    ENDBR

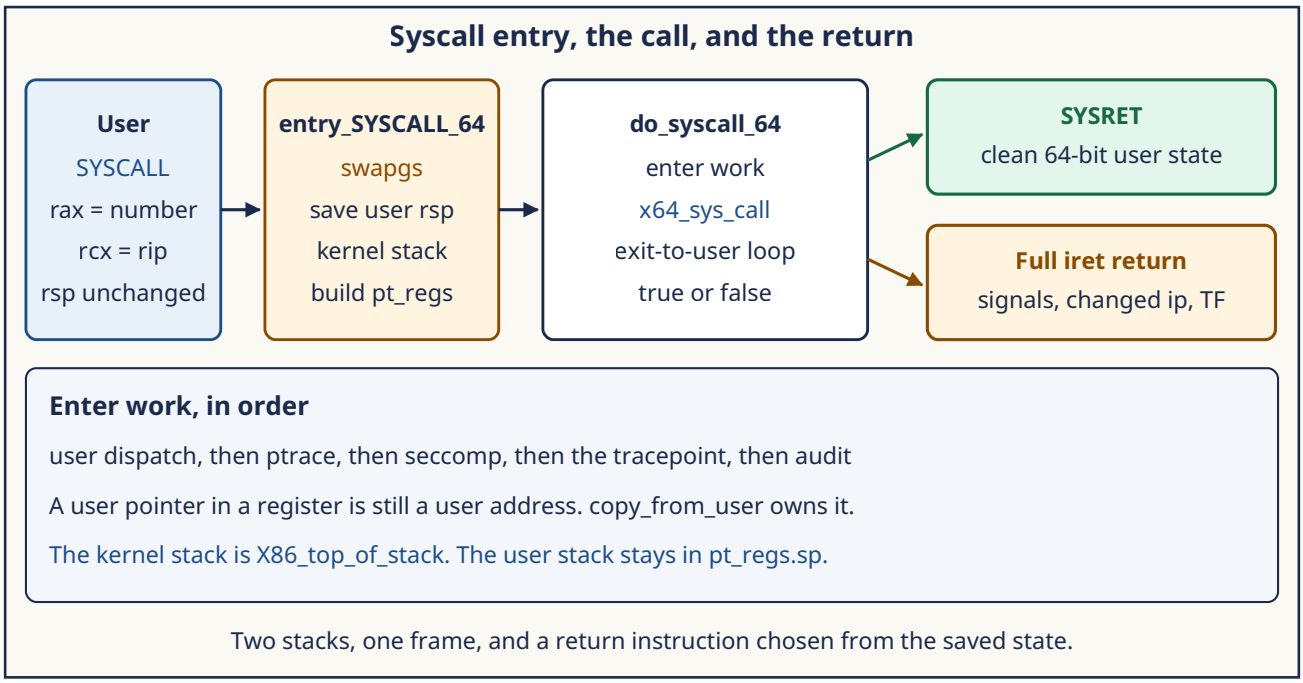
    swapgs
    /* tss.sp2 is scratch space. */
    movq    %rsp, PER_CPU_VAR(cpu_tss_rw + TSS_sp2)
    SWITCH_TO_KERNEL_CR3 scratch_reg=%rsp
    movq    PER_CPU_VAR(pcpu_hot + X86_top_of_stack), %rsp
```

`swapgs` exchanges the user `GS` base for the kernel per-CPU base, which is what makes `PER_CPU_VAR` address this CPU's data. The user `%rsp` is saved in `TSS.sp2` because the next instruction may overwrite `%rsp` while switching CR3. `X86_top_of_stack` is the top of this task's kernel stack. Only after that load is it safe to push.

The pushes at lines 100–109 build `pt_regs`: user `ss`, the saved user `rsp`, `r11` as flags, user `cs`, `%rcx` as `ip`, then `%rax` as `orig_ax`. `PUSH_AND_CLEAR_REGS` saves the rest and sets the value that will sit in `%rax` to `-ENOSYS` until a real call replaces it. `%rdi` is set to `%rsp`, so the C function receives the frame. `%rsi` receives the sign-extended number. Then:

```
    call    do_syscall_64       /* returns with IRQs disabled */
```

`do_syscall_64` in `arch/x86/entry/common.c` (lines 75–129) returns a boolean: true means `SYSRET` is allowed. The assembly tests that boolean unless this is a Xen PV guest, which always takes the `iret` path (lines 130–131). The `SYSRET` sequence restores registers, switches back to the user CR3, executes `swapgs`, and executes `sysretq` (lines 137–166). Anything the C helper rejects jumps to `swapgs_restore_regs_and_return_to_usermode`, the full return.



`SYSCALL` leaves the user stack pointer in place. The kernel switches stacks, and `SYSRET` is used only when the saved user state is still a clean 64-bit return.

The C helper itself is the policy boundary:

```
__visible noinstr bool do_syscall_64(struct pt_regs *regs, int nr)
{
    add_random_kstack_offset();
    nr = syscall_enter_from_user_mode(regs, nr);
    instrumentation_begin();
    if (!do_syscall_x64(regs, nr) && !do_syscall_x32(regs, nr) && nr != -1) {
        regs->ax = __x64_sys_ni_syscall(regs);
    }
    instrumentation_end();
    syscall_exit_to_user_mode(regs);
    /* SYSRET checks follow, then return true or false */
}
```

`add_random_kstack_offset` slides the stack a little so the kernel stack address is not a stable target. `do_syscall_x64` (lines 42–56) accepts the call when the unsigned number is below `NR_syscalls`, clamps the index with `array_index_nospec`, and calls `x64_sys_call`. A number that misses both the 64-bit table and the x32 table, and is not the explicit skip value `-1`, becomes `ni_syscall`, which returns `-ENOSYS`.

The `SYSRET` checks are the interesting part of the boolean. The helper returns false when `rcx` no longer equals the saved `ip`, when `r11` no longer equals the saved flags, when `cs` or `ss` differ from the user selectors `SYSRET` would restore, when `ip` is at or above `TASK_SIZE_MAX`, or when the flags contain `RF` or `TF`. The comment at lines 110–115 states why the address check exists: on Intel CPUs, `SYSRET` with a non-canonical `RCX` raises `#GP` in kernel mode, and

the user chose that address. `TASK_SIZE_MAX` is the bound the kernel uses for a user instruction pointer. A false return selects `iret`, which can restore the general case, including a signal frame whose registers the kernel rewrote.

6. The table and the enter/exit work

`arch/x86/entry/syscalls/syscall_64.tbl` line 12 is the first data row:

```
0   common  read            sys_read
1   common  write           sys_write
2   common  open            sys_open
3   common  close           sys_close
```

`arch/x86/entry/syscall_64.c` includes the generated list twice. Lines 19–27 keep an array for tracing. Lines 29–36 are the call:

```
#define __SYSCALL(nr, sym) case nr: return __x64_##sym(regs);
long x64_sys_call(const struct pt_regs *regs, unsigned int nr)
{
    switch (nr) {
    #include <asm/syscalls_64.h>
    default: return __x64_sys_ni_syscall(regs);
    }
};
```

`__x64_sys_write` is the stub from section 1. It calls `__do_sys_write` with `regs->di`, `regs->si`, and `regs->dx`. That function is the body `SYSCALL_DEFINE3(write, ...)` supplies. The body is allowed to sleep. It runs `in_task()`, on the kernel stack the assembly just installed.

Before the switch, `syscall_enter_from_user_mode` in `include/linux/entry-common.h` (lines 190–201) calls `enter_from_user_mode`, enables interrupts, and runs `syscall_enter_from_user_mode_work`. That work function (lines 163–170) calls `syscall_trace_enter` when `SYSCALL_WORK_ENTER` is set. `syscall_trace_enter` in `kernel/entry/common.c` (lines 28–71) is the policy:

```
SYSCALL_WORK_SYSCALL_USER_DISPATCH   may skip the call
SYSCALL_WORK_SYSCALL_TRACE / EMU     ptrace may change the number
SYSCALL_WORK_SECCOMP                 seccomp sees the tracer's result
read the number again
SYSCALL_WORK_SYSCALL_TRACEPOINT      tracepoint, then read the number
audit_syscall_entry                  record the arguments
```

Seccomp runs after `ptrace` because the comment says so: the filter must observe a number the tracer changed. The tracepoint runs after seccomp, and a probe there can change the number again, so the function re-reads it. Audit records the number that remains. A return of `-1` from this helper makes `do_syscall_64` skip the real call. The user still gets a system-call return; the function that runs is `ni_syscall` or the value the tracer left behind.

After the call, `syscall_exit_to_user_mode` reaches `exit_to_user_mode_loop` (lines 90–133). With interrupts enabled one work item at a time, the loop handles `_TIF_NEED_RESCHED` by calling `schedule`, delivers signals through `arch_do_signal_or_restart`, runs uprobes and live-patch updates, and runs `resume_user_mode_work`. It then disables interrupts and reads the

flags again. The loop ends when no exit-to-user work remains. The assembly return that follows observes the registers the signal path may have changed. A restarted `write` shows up as a return to the `SYSCALL` instruction, not as a second entry invented in the assembly file.

7. The kernel stack and the user-copy boundary

Two addresses must not be confused. The user stack is the `%rsp` saved in `TSS.sp2` and pushed as `pt_regs->sp`. The kernel stack is the pointer loaded from `pcpu_hot + X86_top_of_stack`. Pushes after that load belong to the kernel. A bug that pushed before the load would write the user stack while running in kernel mode.

The other boundary is the pointer the user passed as an argument. `regs->si` on a `write` is a user virtual address. It is not a pointer the kernel may dereference. `copy_from_user` and `get_user` exist so the copy runs with an exception handler registered for the faulting instruction. On x86-64, `get_user` in `arch/x86/include/asm/uaccess.h` (line 108) is the single-value form. The copy path in `arch/x86/include/asm/uaccess_64.h` (lines 121–135) brackets the copying instruction with `stac()` and `clac()`, which set and clear the AC flag so SMAP allows the kernel to touch a user address for the duration of that instruction. The fault label is `_ASM_EXTABLE_UA(1b, 2b)` (line 131), an exception-table entry of type `EX_TYPE_UACCESS` (`arch/x86/include/asm/asm.h` lines 229–230). Day 3 follows what happens when that instruction faults. Today the rule is the boundary: the syscall stub receives the address in a register, and only the user-access helpers may use it.

ToyOS draws the same line in a smaller function. `copy_from_user` in `projects/toyos/src/memory.c` (lines 275–280) rejects a null destination and a length above 4096, then translates the user address in that process's page tables. A failed translation returns false, and `syscall_dispatch` stores `-14` (`-EFAULT`) in `frame->rax`. Linux adds the exception table so a fault inside the copy becomes `-EFAULT` without crashing the kernel. The rule both kernels are enforcing is identical: a user address is interpreted by the user page tables, and only on purpose.

8. WHY the return has two instructions

`SYSRET` is the return that matches a `SYSCALL` which did not change privilege-sensitive fields beyond what the MSRs already describe. It requires `%rcx == rip` and `%r11 == rflags` because that is where the instruction takes them from. A signal handler's frame breaks that equality: the kernel wants user mode to resume at the handler, with flags and a stack the handler expects. `iret` can load that frame. `SYSRET` cannot. The boolean from `do_syscall_64` is how the C path tells the assembly which instruction is honest.

The enter-work order has a similar reason. If seccomp ran before `ptrace`, a tracer could not see the call the filter will judge, and a filter could not see the number the tracer substituted. The source puts `ptrace` first and says why. Audit comes last among those steps so the recorded arguments are the ones the call will use.

The stack switch has a hardware reason. `SYSCALL` does not change `%rsp`. Leaving the user stack in place would let a user pointer decide where the kernel pushes `pt_regs`. Saving `%rsp` to `TSS.sp2` and loading `X86_top_of_stack` is the kernel choosing its own stack before the first push.

9. Execution trace: `write(1, buf, 3)` on x86-64

Assume a 64-bit process, `syscall` instruction, number 1, file descriptor 1, a user pointer in `%rsi`, and length 3 in `%rdx`. Ignore seccomp and `ptrace`; the bits in the thread flag are clear.

```
User
  %rax = 1, %rdi = 1, %rsi = buf, %rdx = 3
  SYSCALL
    hardware: %rcx = user rip, %r11 = rflags, %rsp unchanged

entry_SYSCALL_64
  swapgs
  save user %rsp in TSS.sp2
  SWITCH_TO_KERNEL_CR3
  %rsp = X86_top_of_stack
  push ss, user rsp, r11, cs, rcx, rax
  PUSH_AND_CLEAR_REGS, with the working %rax set to -ENOSYS
  %rdi = pt_regs, %rsi = sign-extended 1
  call do_syscall_64

do_syscall_64
  syscall_enter_from_user_mode: enter_from_user_mode, sti,
      no syscall-work bits, so nr stays 1
  do_syscall_x64: 1 < NR_syscalls, array_index_nospec,
      x64_sys_call case 1 → __x64_sys_write(regs)
  __do_sys_write(fd=regs->di, buf=regs->si, count=regs->dx)
      copy_from_user of 3 bytes, then the file operation
      result stored through regs->ax
  syscall_exit_to_user_mode: no TIF work
  rcx still equals ip, r11 equals flags, cs and ss are the user
      selectors, ip < TASK_SIZE_MAX, TF and RF clear
  return true

entry_SYSCALL_64
  SYSRET path: restore regs, user CR3, swapgs, sysretq
User
  %rax = 3, %rip = the instruction after SYSCALL
```

A second pass with a pending signal takes the same entry and the same `__x64_sys_write`. `exit_to_user_mode_loop` sees `_TIF_SIGPENDING`, runs `arch_do_signal_or_restart`, and may change `regs->ip`. The `SYSRET` check `regs->cx != regs->ip` then fails, the helper returns false, and the assembly uses the full user return. The bytes still moved. The return instruction changed because the frame stopped being a plain `SYSCALL` return.

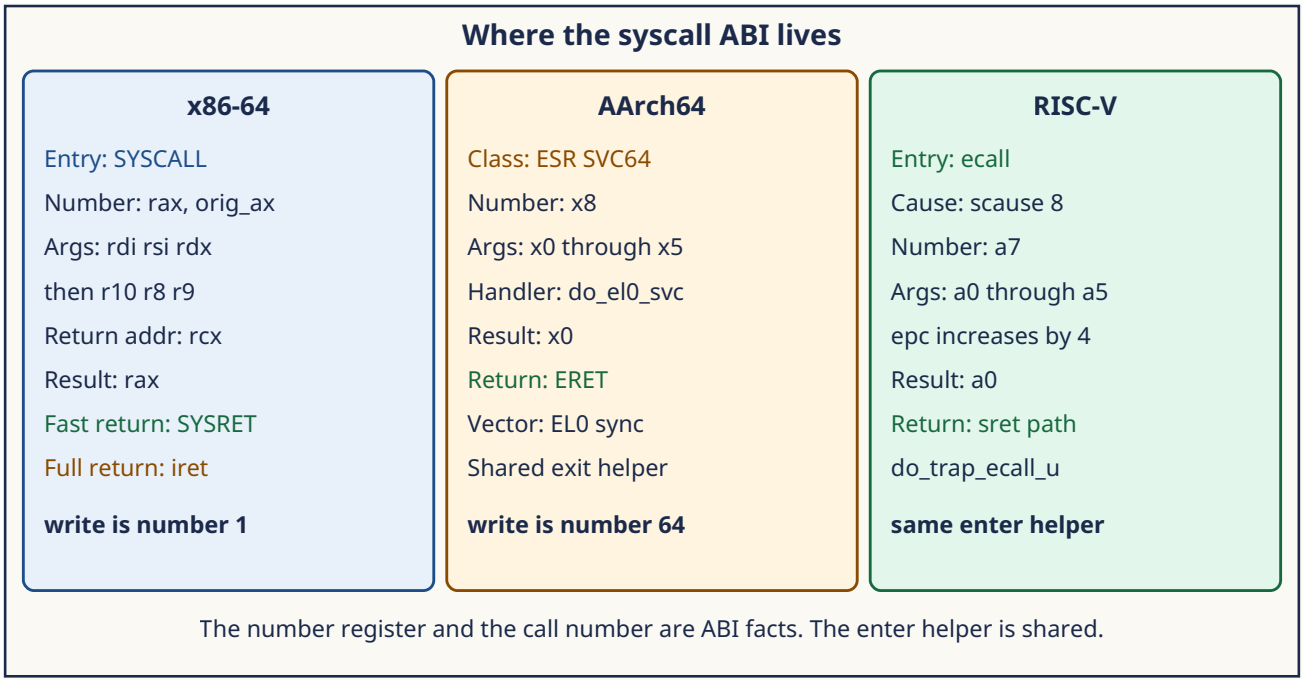
10. Safe evidence

Run `../challenges/day-002-syscall-entry.sh`. The wrapper checks the pinned tree: the `entry_SYSCALL_64` label, the `do_syscall_64` call, the `write` row in `syscall_64.tbl`, and the ToyOS vector-128 gate. It does not trace a production workload, and it does not read another process's memory. It then runs a userspace lab: `syscall(SYS_getpid)` matches `getpid`, and one pipe byte completes. That result is a completed syscall, not a trace of `SYSRET`.

A user-space `strace` of a small `write` is optional confirmation that the number which leaves libc is the number in the table. `strace` sees the libc side of the ABI. The stack switch and the `SYSRET` boolean are visible in the source, not in that trace. Record the source lines, and record one `strace` line only if you ran it: the number, the descriptor, and the returned count.

11. Architecture lens — x86-64, AArch64, and RISC-V

The three machines enter the same generic helpers and pass different registers.



The C helpers match. The register names and the instruction that enters the kernel belong to the architecture.

x86-64, this file. Number in `%rax` (`orig_ax`). Arguments in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, `%r9`. Return address in `%rcx`, flags in `%r11`, result in `%rax`. Entry instruction `SYSCALL`, fast return `SYSRET`, full return `iret`. The fourth argument register is `%r10` because `%rcx` is occupied by the return address. Table numbers for the common calls start at 0 for `read` (`syscall_64.tbl` lines 12–15).

AArch64. `arch/arm64/kernel/entry-common.c` `el0_svc` (lines 706–713) calls `enter_from_user_mode`, drops the DAIF mask, calls `do_el0_svc`, and calls `exit_to_user_mode`. `do_el0_svc` in `arch/arm64/kernel/syscall.c` (lines 149–151) passes `regs->regs[8]` as the number. That register is `x8`. Arguments are `x0` through `x5`, and the result is written back through `syscall_set_return_value`. The vector that got here is the synchronous 64-bit EL0 slot, and the syndrome class is `ESR_ELx_EC_SVC64` (value `0x15` in `arch/arm64/include/asm/esr.h` line 32), decoded by `el0t_64_sync_handler` (lines 724–731). There is no `SYSRET` boolean. The exception return instruction is `ERET`, and it restores the saved program state. The generic table in `include/uapi/asm-generic/unistd.h` numbers `openat` 56, `read` 63, and `write` 64. Those numbers are not the x86-64 numbers. A program compiled for AArch64 loads 64 to write.

RISC-V. `handle_exception` in `arch/riscv/kernel/entry.S` saves the user trap frame and indexes `excp_vect_table` by the exception code. Slot 8 is `do_trap_ecall_u` (line 439). In `arch/riscv/kernel/traps.c` (lines 314–345) the number is `regs->a7`. The function adds 4 to `regs->epc` so the return skips the `ecall`, saves `a0` in `orig_a0`, stores `-ENOSYS` in `a0`, calls `syscall_enter_from_user_mode`, calls `syscall_handler` when the number is in range, and calls `syscall_exit_to_user_mode`. `syscall_get_nr` in `arch/riscv/include/asm/syscall.h` (lines 27–

30) reads `a7`. Arguments are `a0` through `a5`. The result comes back in `a0`. The cause value 8 is an architecture constant (`EXC_SYSCALL` in `arch/riscv/include/asm/csr.h` line 99), not an x86 vector and not an AArch64 exception class.

Across all three, `syscall_enter_from_user_mode` and `syscall_exit_to_user_mode` are the shared policy. The stack that holds `pt_regs` is kernel memory. The pointers inside the argument registers are user addresses until a user-access helper translates them. Day 3 is what happens when that translation faults.

12. Mastery: Explain · Draw · Observe · Build

Explain

0.12 enters through `int 0x80`, range-checks `%eax`, calls `sys_call_table(,%eax,4)`, and returns with `iret`, after considering schedule and signals. Linux 6.12 at `adc218676eef` enters x86-64 through `SYSCALL`, switches to the kernel stack, builds `pt_regs`, and calls `do_syscall_64`. Enter work runs user-dispatch, `ptrace`, seccomp, the tracepoint, and audit, in that order. `x64_sys_call` switches to `__x64_sys_*`, which unpacks registers with `r10` as the fourth argument. Exit work loops on thread flags. `SYSRET` runs only when the helper returns true. A user pointer is copied with a helper that has an exception-table entry.

Draw

1. The x86-64 timeline from section 9, with the user stack and the kernel stack labeled at the moment of the first `pushq`.
2. The enter-work order as six boxes, and the exit loop as a separate box after the call.
3. Three columns for the number register and the result register on x86-64, AArch64, and RISC-V.

Observe

From the wrapper, record the `syscall_64.tbl` line for `write` and the ToyOS `set_idt_gate` line for vector 128. State which instruction each kernel uses to enter, and which value is `-ENOSYS` in each.

Build

Write a ten-line comment, in your notes, for a toy `ecall`/`SYSCALL` dispatcher that rejects an out-of-range number with `-38` and otherwise calls `table[n](frame)`. Then annotate the comment with the 6.12 function that implements each step: range check, enter work, call, exit work, fast return versus full return.

Why? notebook

1. Why does the x86-64 ABI pass argument 4 in `%r10`?
2. Why does `entry_SYSCALL_64` save `%rsp` before it switches `CR3`?
3. Why is `regs->orig_ax` distinct from `regs->ax`?
4. Why does seccomp run after `ptrace` in `syscall_trace_enter`?
5. Why does `do_syscall_64` return false when `regs->ip` is at or above `TASK_SIZE_MAX`?
6. Why is ToyOS's vector 128 a DPL-3 interrupt gate?
7. Why is AArch64 `write` number 64 when x86-64 `write` is number 1?

8. Why does RISC-V add 4 to `epc` before the syscall runs?
9. Why must `copy_from_user` exist when the pointer is already in a kernel register?

Next: Day 3 follows a page fault from the vector or the syndrome register through the fixup table or `handle_mm_fault`, and back.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Syscall Entry

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 14-kernel-entry-contexts-and-interrupts/challenges/day-002-syscall-entry.sh
```

- View lab source on GitHub

Day 3 — Exceptions, Page-Fault Entry, and Return

Target time: approximately 3 hours

- The trap frame and the error-code bits: 30 minutes
- Linux 0.12 `page.s`, then fixups and the generic resolver: 45 minutes
- The 6.12 x86 fault path, user versus kernel: 55 minutes
- AArch64 `ESR` and RISC-V `scause`, carefully: 35 minutes
- Mastery, then the wrapper: 25 minutes

> A store touches a not-present user page. Which status word says "write," which function may sleep to satisfy it, and how does the CPU get back to the store?

Day 2 entered the kernel on purpose. This day enters because an instruction could not complete. The page fault is the exception that the rest of the kernel depends on: demand paging, copy-on-write, and the user-copy helpers all come back through it. The classification is wider than a page fault. A breakpoint, a general-protection fault, and an interrupt are different entries. Today keeps them apart, then follows the page fault until it either fixes the mapping, applies an exception table entry, or stops the task.

The excerpts are from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`.

1. Syntax needed for this source

An x86-64 exception frame is the architectural interrupt frame plus the registers the kernel pushes. The CPU pushes `SS`, `RSP`, `RFLAGS`, `CS`, and `RIP` when privilege changes. For some vectors it also pushes an error code. For the others the stub pushes a zero so the frame has one layout. `struct pt_regs` is that image. `regs->ip` is the faulting instruction. `regs->orig_ax` holds the error code until the stub replaces it with `-1`, because this entry is not a syscall and must not be restarted as one.

The error code is a bitfield. `arch/x86/include/asm/trap_pf.h` (lines 7–30) names the page-fault bits:

```
enum x86_pf_error_code {
    X86_PF_PROT  = BIT(0),   /* 0 not present, 1 protection */
    X86_PF_WRITE = BIT(1),   /* 0 read, 1 write */
    X86_PF_USER  = BIT(2),   /* 0 supervisor access, 1 user access */
    X86_PF_RSVD  = BIT(3),
    X86_PF_INSTR = BIT(4),
    X86_PF_PK    = BIT(5),
    X86_PF_SHSTK = BIT(6),
    X86_PF_SGX   = BIT(15),
    X86_PF_RMP   = BIT(31),
};
```

`error_code & X86_PF_WRITE` is the C form of "bit 1 is set." Bit 2 is the CPU's statement about the access, which is a different fact from `user_mode(regs)`, the privilege of the interrupted program. Day 3 turns on that difference: a kernel `get_user` runs at supervisor privilege while touching a user address, so bit 2 can be clear and `user_mode(regs)` can still be true.

Assembly stubs are generated by the `idtentry` macro in `arch/x86/entry/entry_64.S` (lines 329–365). The parameter `has_error_code` selects whether the stub pushes a dummy `-1` into `orig_ax`. `idtentry_body` (lines 288–316) calls `error_entry`, passes `pt_regs` in `%rdi` and, when there is an error code, that code in `%rsi`, then `jmp error_return`.

A fault that the kernel expected is an exception-table entry, not a branch the C source writes by hand. `_ASM_EXTABLE_UA(from, to)` in `arch/x86/include/asm/asm.h` (lines 229–230) records the faulting instruction and a recovery instruction with type `EX_TYPE_UACCESS`. `fixup_exception` searches that table by the faulting `ip`.

2. Linux 0.12 first: `mm/page.s` and two C functions

The 80386 raises vector 14, pushes an error code, and loads `CR2` with the faulting linear address. `mm/page.s` is the stub. It swaps the error code into `%eax`, saves a few registers, loads the kernel data segments, pushes `CR2` and the error code, and branches on bit 0:

```
page_fault:
    xchgl %eax,(%esp)
    pushl %ecx
    pushl %edx
    push %ds
    push %es
    push %fs
    movl $0x10,%edx
    mov %dx,%ds
    mov %dx,%es
    mov %dx,%fs
    movl %cr2,%edx
    pushl %edx
    pushl %eax
    testl $1,%eax
    jne 1f
    call do_no_page
    jmp 2f
1:  call do_wp_page
2:  addl $8,%esp
    pop %fs
    pop %es
    pop %ds
    popl %edx
    popl %ecx
    popl %eax
    iret
```

`testl $1,%eax` inspects the present/protection bit. A clear bit means the page-table walk found no present page, and the stub calls `do_no_page`. A set bit means the page is present and the access violated its permission, and the stub calls `do_wp_page`. Both functions live in `mm/memory.c`. The write-protect case is how 0.12 breaks a copy-on-write page after `fork`. The not-present case is how it brings a page in. On the way out the stub pops the frame and executes `iret`, which resumes the faulting instruction now that the mapping exists.

What carries forward is the split. Hardware reports an address and a reason. A tiny stub builds a kernel context and chooses a C function. The C function changes a page table. `iret` runs the faulting instruction again. The historical limits are just as important. The stub branches only on bit 0. It does not search an exception table, so a kernel fault on a bad pointer and a user fault on an unmapped page share the same two calls. It does not separate an instruction fetch from a data access. It is a uniprocessor 32-bit kernel with one page size and no SMAP. The 386 error code already has the write bit and the user bit; this stub does not consult them.

`kernel/system_call.s` from Day 2 and `mm/page.s` are two entries into one kernel. The syscall is deliberate. The page fault is forced. Both return with `iret`.

3. Evolution: one resolver, many status words, a prepared fault

The 0.12 branch does not survive contact with more than one architecture, or with a kernel that must touch user addresses on purpose.

The resolver moved. Demand paging, copy-on-write, and stack growth are properties of a virtual memory area, not of the 386 error code. `handle_mm_fault` in `mm/memory.c` is that shared function. Each architecture translates its own status word into the flags the shared function already understands: `FAULT_FLAG_WRITE`, `FAULT_FLAG_INSTRUCTION`, and `FAULT_FLAG_USER`. x86 packs those facts into one error code. AArch64 reports a data abort or an instruction abort as different exception classes, and reports write-not-read as a bit in the syndrome. RISC-V reports three different exception causes for instruction, load, and store page faults. The shared function never reads `CR2`, `ESR`, or `scause`.

The stub generator replaced the hand-written `page.s`. Vector 14 is one of many. The `idtentry` macro emits a stub per vector so a missing error-code push cannot mis-align `pt_regs`. Interrupts reuse the same body with a vector number in the error-code slot. The classification —"which entry is this?"—is a macro argument and an IDT vector, not a copied prologue.

Expected kernel faults became data. `copy_from_user` contains an instruction that is allowed to fault. The exception table pairs that instruction with a recovery address. `fixup_exception` runs before the kernel declares the fault fatal. 0.12 had no such table: a bad kernel pointer in a syscall was not a recoverable `-EFAULT` in the modern sense. The table is also why a user-access helper and an ordinary kernel dereference are different bugs. The helper has a fixup. The ordinary dereference does not, and a fault there is an oops.

Return hardened along the way. `error_return` in `entry_64.S` (lines 1088–1093) tests the saved `CS`:

```
testb   $3, CS(%rsp)
jz  restore_regs_and_return_to_kernel
jmp swapgs_restore_regs_and_return_to_usermode
```

A fault from kernel mode restores kernel registers and returns with the kernel `GS` and kernel page tables still in place. A fault from user mode takes the user return, which runs the same exit work Day 2 showed: signals and reschedule before user instructions resume. A fixup that changes `regs->ip` returns to the recovery instruction, not to the faulting load. That is the modern form of "resume the faulting instruction," with a second choice for faults the kernel prepared for.

SMAP and the RISC-V `SUM` bit are the hardware half of the same boundary. Supervisor mode does not accidentally accept a user address. The user-access helper turns that permission on for one copy and turns it off again. A fault with the permission off is a kernel bug, and the handler says so instead of resolving it as a user page.

4. ToyOS reconstruction

ToyOS classifies exceptions by vector and prints the 386 error-code bits. It does not resolve the fault. That limit is the lesson: entry and resolution are separate, and this kernel has only the entry.

`src/isr.S` uses two macros. `ISR_NOERR` pushes a zero error word. `ISR_ERR` does not, because the CPU already pushed one. Vector 14 is `ISR_ERR 14` (line 33). `isr_common` pushes the general registers and calls `interrupt_handler`. The frame layout in `include/toyos.h` (lines 11–16) matches that push order: the general registers, then `vector`, `error`, `rip`, `cs`, `rflags`, `rsp`, and `ss`.

`interrupt_handler` in `src/arch.c` handles three vectors before the exception report. Vector 32 is the timer. Vector 128 is the Day 2 syscall. Vector 3 is the breakpoint self-test. Anything else prints the name. Vector 14 then reads `CR2` and decodes the same bits as `trap_pf.h`:

```
if (frame->vector == 14U) {
    uint64_t address;
    __asm__ volatile ("mov %%cr2, %0" : "=r"(address));
    serial_write("PAGE-FAULT address=");
    serial_hex(address);
    serial_write(" flags=");
    serial_write((frame->error & 1U) != 0U ? "protection " : "not-present ");
    serial_write((frame->error & 2U) != 0U ? "write " : "read ");
    serial_write((frame->error & 4U) != 0U ? "user " : "supervisor ");
    /* reserved-bit and instruction-fetch follow */
}
```

Bit 0 selects the words "protection" and "not-present." Bit 1 selects "write" and "read." Bit 2 selects "user" and "supervisor." That is the 0.12 status word, printed rather than dispatched to `do_no_page` and `do_wp_page`. If `EXPECT_FAULT` is 2, vector 14 is the expected self-test and the kernel exits. Otherwise it panics. There is no exception table and no `handle_mm_fault`. A faulting user store does not resume. Linux 6.12 is what you add when the store must resume: a search of the fixup table, or a change to the page tables, then `error_return`.

5. Linux 6.12 x86-64: classify, then resolve or fix up

The IDT entry for the page fault is declared in `arch/x86/include/asm/idtentry.h` (line 623):

```
DECLARE_IDTENTRY_RAW_ERRORCODE(X86_TRAP_PF, exc_page_fault);
```

`DEFINE_IDTENTRY_RAW_ERRORCODE` (lines 180–181) expands to a C function that receives `struct pt_regs *` and `unsigned long error_code`, and does not itself call `irqentry_enter`. The page-fault body wants to decide that for itself, because a fault in kernel mode on a user address may sleep, and the usual interrupt-entry RCU sequence would be the wrong context.

`exc_page_fault` in `arch/x86/mm/fault.c` (lines 1493–1542) reads the address, enters, and dispatches:

```
DEFINE_IDTENTRY_RAW_ERRORCODE(exc_page_fault)
{
    irqentry_state_t state;
    unsigned long address;

    address = cpu_feature_enabled(X86_FEATURE_FRED)
        ? fred_event_data(regs) : read_cr2();
    /* kvm async page fault may consume the event and return */
    state = irqentry_enter(regs);
    instrumentation_begin();
    handle_page_fault(regs, error_code, address);
    instrumentation_end();
    irqentry_exit(regs, state);
}
```

On a machine without FRED, the address is `CR2`, the same register 0.12 read. `handle_page_fault` (lines 1468–1491) traces the fault and splits on the address:

```
if (unlikely(fault_in_kernel_space(address)))
    do_kern_addr_fault(regs, error_code, address);
else
    do_user_addr_fault(regs, error_code, address);
```

`fault_in_kernel_space` (lines 1119–1130) is true when the address is at or above `TASK_SIZE_MAX`, with a special case for the vsyscall page. A kernel address takes `do_kern_addr_fault`, which may apply a spurious fault check or a kprobe and otherwise calls `bad_area_nosemaphore`. That path reaches `kernelmode_fixup_or_oops` (lines 720–728): if `fixup_exception` finds a table entry it returns, and the recovery `ip` is what `error_return` will resume. If it does not find one, the fault is an oops.

A user address takes `do_user_addr_fault` (lines 1210–1306 and following). Several rejects happen before any lock. An instruction fetch from user memory while the CPU was in kernel mode, with the erratum check set aside, is fatal: the kernel does not execute from a user page. A reserved bit in a user page-table entry is a bad page table. SMAP, when the access is a supervisor access (`X86_PF_USER` clear), AC is clear, and the feature is on, is fatal (lines 1257–1265): the kernel touched a user address without `stac()`. A fault while page faults are disabled, or with no `mm`, goes to `bad_area_nosemaphore` and from there to the fixup table (lines 1268–1274). That is the context check from Day 1, applied: an atomic context must not sleep in `handle_mm_fault`.

The flags passed to the resolver are built from the error code and from the frame, and the source comments on the user bit (lines 1300–1306):

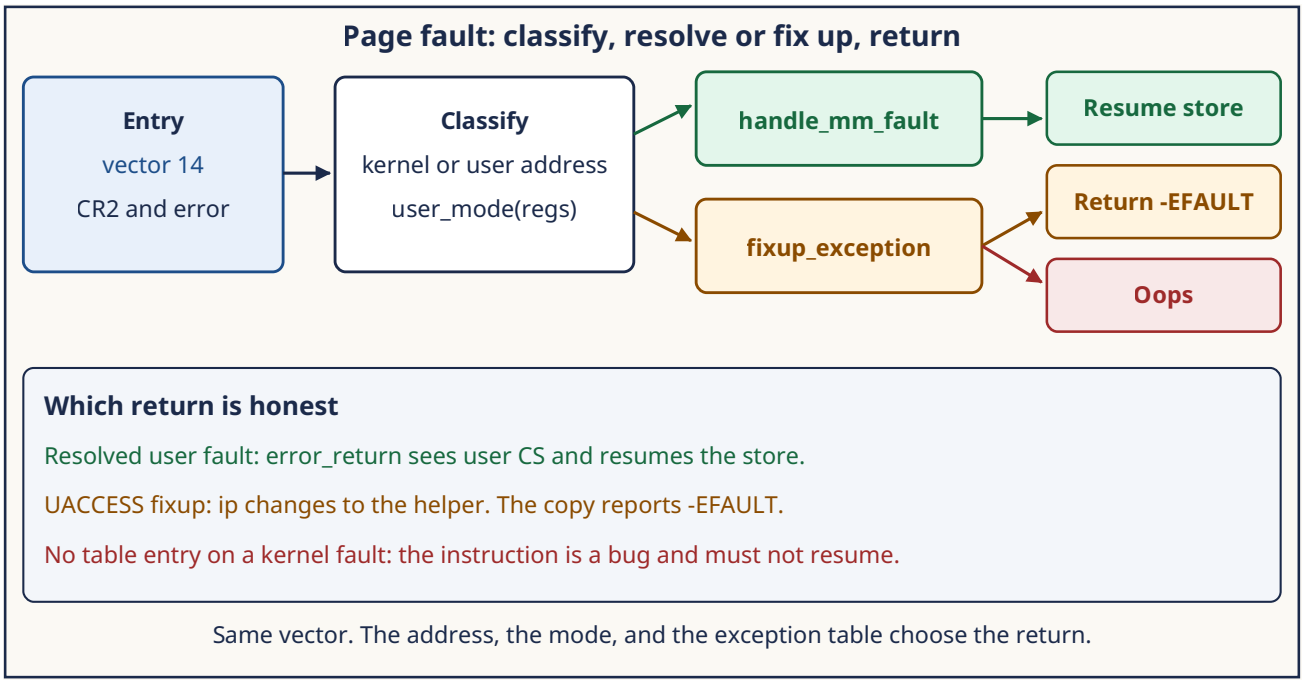
```
if (error_code & X86_PF_WRITE)
    flags |= FAULT_FLAG_WRITE;
if (error_code & X86_PF_INSTR)
    flags |= FAULT_FLAG_INSTRUCTION;
if (user_mode(regs))
    flags |= FAULT_FLAG_USER;
```

`FAULT_FLAG_USER` follows `user_mode(regs)`, the privilege in the saved `CS`, rather than `X86_PF_USER`. A `copy_from_user` inside a syscall is a kernel-mode access to a user address. The access bit in the error code describes the access. The mode bit in `CS` describes the interrupted program. The fault is still that program's fault for accounting and for signal delivery, which is why the flag follows `CS`.

From there the handler looks up the VMA and calls `handle_mm_fault` in `mm/memory.c` (lines 6042–6043):

```
vm_fault_t handle_mm_fault(struct vm_area_struct *vma, unsigned long address,
               unsigned int flags, struct pt_regs *regs)
```

Copy-on-write, a not-present anonymous page, and a file fault are inside that function and the routines it calls. They are the descendants of `do_wp_page` and `do_no_page`. A failure to resolve becomes a signal for a user fault, or the fixup-or-oops path for a kernel fault. `local_irq_disable` at the end of `handle_page_fault` puts interrupts back off before `irqentry_exit`, matching what `error_return` asserts.



A prepared kernel fault changes `ip` and returns to the helper. A resolved user fault returns to the store. An unprepared kernel fault is an oops.

`fixup_exception` in `arch/x86/mm/extable.c` (lines 300–336) searches by `regs->ip`. The entry's type selects the handler. `EX_TYPE_UACCESS` is the user-copy case. The handler sets the return value and the instruction pointer so the helper returns `-EFAULT` to the syscall. The store or load that faulted does not re-execute. The syscall returns. That is the user-copy boundary from Day 2, closed.

6. WHY a fault returns to different instructions

A not-present user store returns to the store. The page tables now contain a present page, and the store will complete. Returning anywhere else would skip the write the program asked for.

A `copy_from_user` fault returns to the fixup, not to the copying instruction. Re-executing the copy would fault again on the same bad address. The helper's contract is to stop and report `-EFAULT`. The exception table is how the fault path learns that contract, because the fault arrives far away from the helper's source line, with only `ip` to identify the instruction.

A kernel NULL dereference has no table entry and must not resume. The instruction is a bug. `oops` stops the task and prints `CR2`. Resuming would run the bug again.

The two status bits that people fold together stay apart for the same reason. `X86_PF_USER` answers "did this access use a user permission?" `user_mode(regs)` answers "was the interrupted program in user mode?" `get_user` needs both answers. SMAP uses the first. Signal delivery and `FAULT_FLAG_USER` use the second.

7. Execution trace: two faults, one vector

Both faults enter through vector 14, `exc_page_fault`, and `read_cr2()`. They split inside `handle_page_fault`.

```
A. User store through a not-present user page

  User instruction: mov %eax, (%rdi) with %rdi in the user half
  CPU: vector 14, error code with bit 1 set (write) and bit 2 set
       (user access), bit 0 clear (not present), CR2 = %rdi
  idtentry stub: error_entry, call exc_page_fault
  fault_in_kernel_space(CR2) is false
  do_user_addr_fault:
      user_mode(regs) is true, so FAULT_FLAG_USER
      X86_PF_WRITE sets FAULT_FLAG_WRITE
      VMA lookup succeeds
      handle_mm_fault allocates or copies a page
  irqentry_exit
  error_return sees user CS and takes the user return
  The mov executes again and completes

B. Kernel copy_from_user of a bad user pointer, inside write()

  The faulting instruction is the copy, marked _ASM_EXTABLE_UA
  CPU: vector 14, CR2 = the bad pointer
      X86_PF_USER is clear if this was a supervisor access
      user_mode(regs) is true, because the syscall came from user mode
  do_user_addr_fault builds flags, then cannot resolve the address
  kernelmode_fixup_or_oops → fixup_exception finds EX_TYPE_UACCESS
  regs->ip becomes the recovery label
  The copy helper returns -EFAULT to __do_sys_write
  Day 2's exit path returns that error to user mode
```

Trace A changes page tables and resumes the store. Trace B changes `ip` and resumes the helper. The vector number was 14 in both. The status word, the mode, and the exception table decide which return is legal.

8. Safe evidence

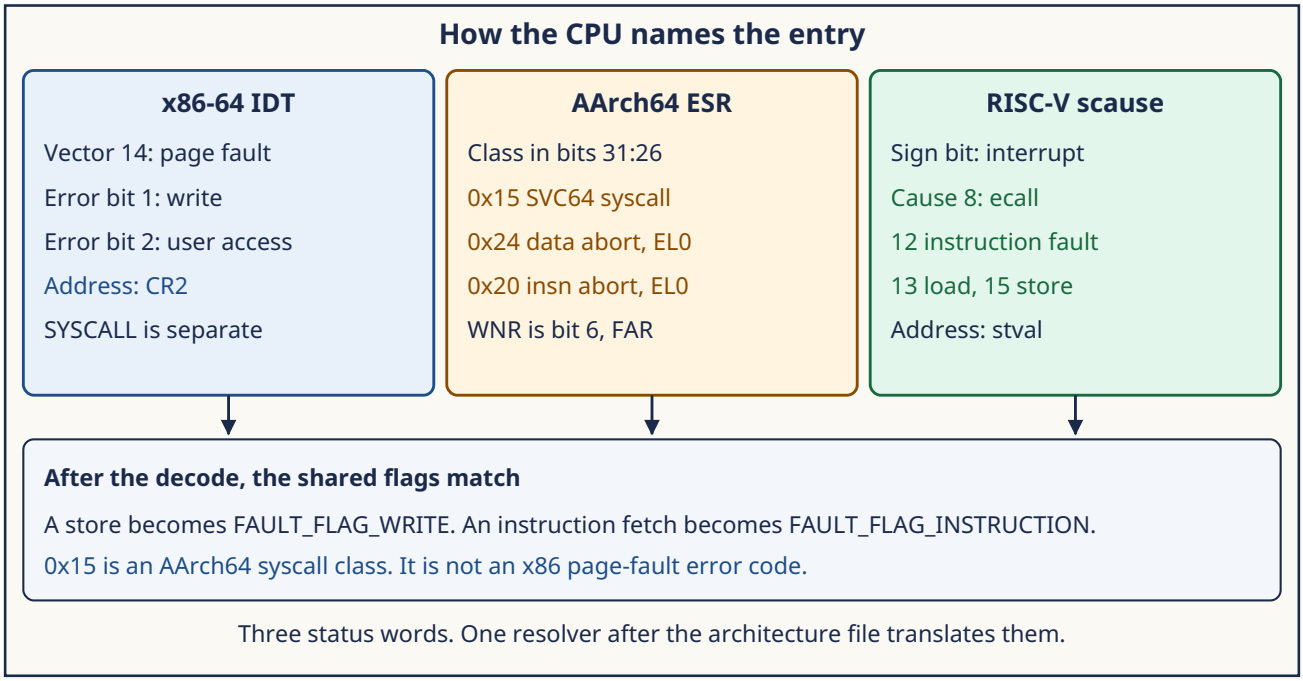
Run `../challenges/day-003-exception-entry.sh`. The wrapper reads the pinned tree and the ToyOS fault print. It checks that vector 14 is an `ISR_ERR` stub, that `trap_pf.h` names `X86_PF_WRITE` as bit 1, and that `exc_page_fault` reads `CR2` unless FRED is in use. It does not

inject a fault into the running kernel. It then runs a userspace child: a `PROT_NONE` store is observed as `SIGSEGV`, and a second child catches `SIGUSR1`. Those are host symptoms, not a trace of `exc_page_fault`.

ToyOS's expected page-fault self-test is the safe dynamic evidence for the entry half: a known vector, a printed `CR2`, and a printed bit decode, then a controlled exit. Resolving the fault and resuming the store is the 6.12 behavior you read in `handle_mm_fault`. Do not provoke an oops on a machine whose disk you care about in order to see trace B. The exception-table type in `extable.c` is the evidence for that path.

9. Architecture lens — x86-64, AArch64, and RISC-V

The three architectures do not share a status-word layout. Comparing them means naming the register and the field, then showing which shared flag it becomes.



x86 uses an IDT vector plus an error code. AArch64 uses an exception class in ESR. RISC-V uses scause, with the sign bit marking an interrupt.

x86-64, summarized from section 5. One vector, 14, for page faults. The error code's bit 0 is present versus protection, bit 1 is write, bit 2 is a user access, bit 4 is an instruction fetch. `CR2` holds the address. `user_mode(regs)` comes from `CS`, not from bit 2. `error_return` uses the low two bits of the saved `CS` to choose the kernel return or the user return.

AArch64 reports the class in `ESR_ELx`, bits starting at shift 26. `arch/arm64/include/asm/esr.h` defines the values this tree uses:

```
ESR_ELx_EC_SVC64     0x15    syscall from 64-bit EL0
ESR_ELx_EC_IABT_LOW  0x20    instruction abort, lower EL
ESR_ELx_EC_IABT_CUR  0x21    instruction abort, current EL
ESR_ELx_EC_DABT_LOW  0x24    data abort, lower EL
ESR_ELx_EC_DABT_CUR  0x25    data abort, current EL
ESR_ELx_WNR          bit 6   write, inside a data abort
```

`ESR_ELx_EC(esr)` at line 74 shifts the class out. `el0t_64_sync_handler` in `arch/arm64/kernel/entry-common.c` (lines 724–737) switches on that class. `ESR_ELx_EC_SVC64` calls `el0_svc`, which is Day 2's path. `ESR_ELx_EC_DABT_LOW` calls `el0_da`. `ESR_ELx_EC_IABT_LOW` calls `el0_ia`. Those are different cases, not bits in one page-fault error code. The faulting address is `FAR_EL1`, not a general-purpose register and not `CR2`. `do_mem_abort` in `arch/arm64/mm/fault.c` (lines 874–891) looks up a `fault_info` row from the low bits of the syndrome and, for a user-mode fault that the row does not handle, delivers a signal. A kernel-mode fault that the row does not handle calls `die_kernel_fault`. The class numbers are not x86 vectors: `0x14` is not page-fault vector 14, and `0x15` is a syscall class rather than an x86 error-code bit. Write-not-read lives at bit 6 of the syndrome (`ESR_ELx_WNR`, lines 85–86), not at bit 1.

RISC-V reports a cause in `scause`, saved as `PT_CAUSE` by `handle_exception` (`arch/riscv/kernel/entry.S` lines 152–158). The sign bit is the interrupt bit: `bge s4, zero` at line 183 means the value is an exception, and the handler indexes `excp_vect_table`. An interrupt goes to `do_irq` instead. The exception codes in `arch/riscv/include/asm/csr.h` (lines 90–104) are small integers with gaps:

```
EXC_SYSCALL            8     user environment call
EXC_INST_PAGE_FAULT   12
EXC_LOAD_PAGE_FAULT   13
EXC_STORE_PAGE_FAULT  15
```

There is no cause value that means "page fault" in general, and there is no error-code bitfield riding along with the cause. The store versus load distinction is which cause arrived. `excp_vect_table` (lines 443–447) points causes 12, 13, and 15 at `do_page_fault`, and cause 8 at `do_trap_ecall_u`. `handle_page_fault` in `arch/riscv/mm/fault.c` (lines 223–286) reads `regs->cause` and `regs->badaddr`. A store cause sets `FAULT_FLAG_WRITE`. An instruction-fault cause sets `FAULT_FLAG_INSTRUCTION`. User mode sets `FAULT_FLAG_USER`. The address then goes to `handle_mm_fault`, the same shared function x86 uses.

The RISC-V user-access check is the `SUM` bit in `sstatus`, and its polarity is the opposite of a casual reading of SMAP. In `handle_page_fault` (lines 274–278), a kernel-mode fault on an address below `TASK_SIZE` while `SUM` is clear is illegal:

```
if (!user_mode(regs) && addr < TASK_SIZE && unlikely(!(regs->status & SR_SUM))) {
    if (fixup_exception(regs))
        return;
    die_kernel_fault("access to user memory without uaccess routines",
              addr, regs);
}
```

`SUM` set means the supervisor is allowed to touch user memory. The entry path clears `SUM` on the way in (`SR_SUM` in the `csrrc` at `entry.S` line 146) and the copy routine sets it only around the copy. A fault with `SUM` clear still tries `fixup_exception` first, then dies. That is the same policy as the x86 SMAP check, expressed with a different bit and the opposite "permission present" polarity. SMAP's AC flag, when set, permits the access. `SUM`, when set, permits the access. The page-fault error-code bit `X86_PF_USER` is not either of those flags.

Folding these into one sentence — "the fault register's write bit" — mis-teaches all three. x86 bit 1 of the error code, AArch64 bit 6 of `ESR` inside a data-abort class, and RISC-V cause 15 are three encodings of a store. The shared encoding is `FAULT_FLAG_WRITE`, and it appears only after the architecture file has translated.

Return follows the same split. x86 `error_return` tests saved `CS`. AArch64 exception return is `ERET` after `exit_to_user_mode` or the kernel return path in the vector code. RISC-V `ret_from_exception` restores `sepc` and `sstatus` from the trap frame. A fixup that rewrote the saved PC returns to the helper on every architecture. A resolved fault returns to the faulting instruction on every architecture. The instruction that performs the return is the architecture's.

10. Mastery: Explain · Draw · Observe · Build

Explain

0.12 `page.s` reads `CR2`, branches on error-code bit 0, calls `do_no_page` or `do_wp_page`, and returns with `iret`. Linux 6.12 at `adc218676eef` enters x86 vector 14 through `exc_page_fault`, reads `CR2`, and splits on `fault_in_kernel_space`. A user address may reach `handle_mm_fault` with flags taken from the error code and from `user_mode(regs)`. A kernel fault looks for an exception-table entry first. `EX_TYPE_UACCESS` returns `-EFAULT` to the copy helper. `error_return` chooses the kernel or user restore from the saved `CS`. AArch64 classifies with `ESR_ELx_EC_*` and reports writes in `ESR_ELx_WNR`. RISC-V classifies with `scause` values 8, 12, 13, and 15, and uses `SUM` as the supervisor user-access permission.

Draw

1. Trace A and trace B from section 7 on one picture, splitting after `handle_page_fault`.
2. The x86 error-code bits 0, 1, 2, and 4, each with the flag or check that consumes it.
3. Three status words side by side: x86 error code, AArch64 `ESR` class plus `WNR`, RISC-V `scause` exception code. Arrow each store encoding at `FAULT_FLAG_WRITE`.

Observe

From the wrapper, record the ToyOS bit masks (`1U`, `2U`, `4U`) and the `trap_pf.h` enumerator values, and state that they describe the same 386 error code. Record one AArch64 class value and one RISC-V cause value from the wrapper's grep, and state which shared flag each becomes.

Build

In your notes, write the body of a function `classify(error_code, user_mode)` that returns the three `FAULT_FLAG_*` bits x86 sets in `do_user_addr_fault`. Then write the RISC-V version that switches on `cause` instead of masking bits. The two functions may call the same `handle_mm_fault`. They may not share a bit test.

Why? notebook

1. Why does `ISR_ERR` skip the zero push that `ISR_NOERR` performs?
2. Why does 0.12 `page.s` call `do_wp_page` when bit 0 is set?
3. Why is `FAULT_FLAG_USER` taken from `user_mode(regs)` when the error code already has `X86_PF_USER`?
4. Why does SMAP kill a supervisor access with AC clear before the VMA lookup?

5. Why does a `copy_from_user` fault return to the fixup instead of the copying instruction?
6. Why is `ESR_ELx_EC_SVC64` (0x15) not "page fault, error code 0x15"?
7. Why do RISC-V loads and stores have different exception causes?
8. Why does a kernel fault with RISC-V `SUM` clear try `fixup_exception` before it dies?
9. Why does `error_return` test the saved `CS` with `testb $3`?

Next: Day 4 follows the device line. Softirqs and sleepability come after that. Choosing the next task is Week 15, not this fault path. The entry instruction and the page-fault resolver now have a path you can read in this tree. Passing the wrapper does not complete mastery.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Exception Entry

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 14-kernel-entry-contexts-and-interrupts/challenges/day-003-exception-entry.sh
```

- View lab source on GitHub

Day 4 — Hardware Interrupts, IRQ Descriptors, and Handlers

Target time: approximately 3 hours

- An IRQ is a different entry from a fault: 25 minutes
- Linux 0.12's 8259 and fixed IDT slots: 40 minutes
- `irq_desc`, sharing, affinity, and chip order: 80 minutes
- Mastery, then the wrapper: 25 minutes

> A device asks for service. A controller turns that request into a CPU entry. Linux names the entry with an `irq_desc` and walks a chain of handlers. Mask, acknowledge, and end-of-interrupt stay with the chip.

Day 3 entered because an instruction could not finish. This day enters because a device asked. The classification stays: a page fault, a system call, and a device IRQ share a saved context and reach different functions. Today follows the device path from the controller to `action->handler`.

The 6.12 excerpts are from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. The 0.12 excerpts are from the historic `linux-0.12` tree. Record `uname -r` before you treat a 6.12 line number as true of the booted kernel.

1. Syntax needed for this source

The handler is a function pointer. A shared line is a linked list. The controller operation is another function pointer on a struct.

```
typedef irqreturn_t (*irq_handler_t)(int irq, void *dev_id);

struct irqaction {
    irq_handler_t handler;
    void *dev_id;
    struct irqaction *next;
};

struct irq_chip {
    void (*irq_ack)(struct irq_data *data);
    void (*irq_mask)(struct irq_data *data);
    void (*irq_eoi)(struct irq_data *data);
};
```

`dev_id` is the cookie the driver passed to `request_irq`. `next` is how a second driver joins the same Linux irq number. Any chip pointer may be NULL. A flow handler tests the pointer before it calls through it.

An x86 gate is a different object. `include/asm/system.h` in Linux 0.12 builds an IDT entry from a type and a DPL. Type 14 is an interrupt gate: the CPU clears IF. Type 15 is a trap gate: IF is left as it was. DPL 0 accepts the gate from the kernel. DPL 3 is what makes a software `int` reachable from user mode. Day 2 used DPL 3 for the syscall. A device IRQ uses the interrupt-gate form.

2. An IRQ and an exception take different entries

An exception is tied to the instruction stream. Divide error, invalid opcode, page fault, and a system call the program executed happen because of something the CPU was already doing. Day 3's vector 14 is the page-fault case: `CR2` and the error code describe that access.

An interrupt is a request from outside that instruction. A timer expires. A disk controller finishes. A network device has a packet. The CPU finishes or aborts the current instruction according to the architecture and vectors to a stub the kernel installed. The interrupted instruction is the victim. It is not the cause.

Linux keeps both in one vector table on x86, then splits them in software. Exception stubs land in trap handlers such as `exc_page_fault`. Device IRQs on the common path land in `common_interrupt`. NMI uses a stricter entry. The generic IRQ layer below is the device path. A page fault does not allocate an `irq_desc`.

3. Linux 0.12 first: one 8259 pair and fixed IDT slots

Linux 0.12 is a uniprocessor 80386 kernel. It has no `irq_desc`, no `irq_chip`, and no `request_irq`. The controller is a pair of Intel 8259 chips. `boot/setup.S` reprograms them in real mode, lines 152–185. The comment says the BIOS had placed hardware interrupts at `0x08`, which collides with CPU exception vectors, so the kernel moves them to `0x20`–`0x2F`.

The sequence sends `0x11` to command ports `0x20` and `0xA0`, then writes vector base `0x20` to the master data port `0x21` and base `0x28` to the slave data port `0xA1`. Cascade identity `0x04` tells the master that the slave hangs off IRQ2. Identity `0x02` tells the slave its cascade id. Mode byte `0x01` selects 8086 mode. A final `0xFF` on both data ports masks every input until a driver clears its bit.

IRQ0, the 8254 timer, therefore enters at IDT slot `0x20`. The slave's first input enters at slot `0x28`. Those addends are the whole "irq domain" this kernel has.

`include/asm/system.h`, lines 33–40, names the gate helpers:

```
#define set_intr_gate(n,addr) \
    _set_gate(&idt[n],14,0,addr)

#define set_trap_gate(n,addr) \
    _set_gate(&idt[n],15,0,addr)

#define set_system_gate(n,addr) \
    _set_gate(&idt[n],15,3,addr)
```

`kernel/traps.c`, `trap_init`, lines 185–212, installs exception gates, fills the rest of the first 48 slots with `reserved`, then overrides slot 45 with `irq13` and slot 39 with `parallel_interrupt`. It clears the master's IRQ2 mask (`inb_p(0x21)&0xfb`) so the cascade is visible, and the slave's IRQ13 mask (`inb_p(0xA1)&0xdf`) for the coprocessor.

`kernel/sched.c`, `sched_init`, lines 438–443, programs the PIT and opens the timer:

```
    outb_p(0x36,0x43);      /* binary, mode 3, LSB/MSB, ch 0 */
    outb_p(LATCH & 0xff , 0x40);    /* LSB */
    outb(LATCH >> 8 , 0x40);    /* MSB */
    set_intr_gate(0x20,&timer_interrupt);
    outb(inb_p(0x21)&~0x01,0x21);
    set_system_gate(0x80,&system_call);
```

`&~0x01` clears the IRQ0 mask. The timer gate is type 14, DPL 0. The syscall gate is type 15, DPL 3. Same IDT, different type and privilege.

The stub in `kernel/sys_call.s`, `_timer_interrupt`, lines 189–211, saves registers, loads kernel data segments, increments `jiffies`, and writes `0x20` to port `0x20`. That byte is a nonspecific end-of-interrupt to the master 8259. Then it masks the saved CS with 3 to recover the CPL and calls `do_timer`. The EOI is early. The 8259 may post another timer IRQ as soon as it accepts that byte.

`do_timer`, lines 346–367, charges `utime` or `stime` from the CPL. When the user-mode quantum hits zero it calls `schedule`. A kernel-mode tick sets `counter` to zero and returns without scheduling. Day 6 returns to that CPL test. Today the point is that the handler and the EOI are this one stub.

The hard-disk stub, lines 235–258 of the same file, EOIs the slave at `0xA0` and the master at `0x20`, exchanges `_do_hd` with NULL, and calls through that pointer. One planted function, not a list. A second driver cannot join that pointer.

What carries forward is a controller that maps a wire to a vector, a mask bit, an EOI, and a C function. The port numbers, the two-chip cascade, and the fixed bases stay in 1992.

4. Evolution: APIC, MSI, domains, generic IRQ

A second CPU has no single 8259 both processors share in the 0.12 way. An I/O APIC watches device wires and sends a message to a local APIC on one CPU. Software chooses the target CPU. That choice is affinity.

Message signaled interrupts remove the wire. The device writes a small message to an address the controller published. `include/linux/msi.h`, `struct msi_msg`, lines 61–74, holds `address_lo`, `address_hi`, and `data`. `irq_chip.irq_write_msi_msg` programs them. A message that already names its source is a poor match for the 8259 habit of hanging two devices on one wire. MSI lines are usually exclusive. A shared legacy wire still exists on machines that have one.

The Linux number and the hardware number diverged. `struct irq_domain` in `include/linux/irqdomain.h`, lines 169–176, is the map: a name, ops, a root, and a reverse map from `hwirq` to a Linux irq. The Linux irq is the index of an `irq_desc`. A hierarchy of domains is how an MSI domain sits on a vector domain on an APIC. The 0.12 identity "IRQ0 is IDT slot `0x20`" is the special case where the map is a constant addend.

The generic layer lives in `kernel/irq/`. Flow handlers know about level, edge, and end-of-interrupt. Chip drivers fill `irq_mask`, `irq_ack`, and `irq_eoi`. Device drivers call `request_irq`. They do not write IDT gates.

5. ToyOS reconstructs the PIC path

ToyOS keeps the 0.12 shape on purpose, in long mode. `projects/toyos/src/arch.c`, `pic_and_pit_init`, lines 183–201, sends `0x11` to `0x20` and `0xA0`, bases `0x20` and `0x28`, cascade `0x04` and `0x02`, then masks the master with `0xfe` so only IRQ0 is open. The PIT divisor `11932` is about 100 Hz.

`arch_init` installs exception gates with attribute `0x8e` and puts `irq0_stub` on vector 32. `projects/toyos/src/isr.S`, lines 52–56, pushes a dummy error and the vector 32, then `isr_common` saves the general registers and calls `interrupt_handler` with the frame in `rdi`.

`interrupt_handler`, lines 252–258, increments `ticks`, writes `0x20` to port `0x20`, and calls `scheduler_tick` when that milestone is built. The write is the same master EOI as `_timer_interrupt`. ToyOS has one timer handler and no `irq_desc`. The descriptor chain is what the 6.12 reading adds.

6. Linux 6.12: descriptor, chip, action, and the walk

`include/linux/irqdesc.h`, lines 67–73:

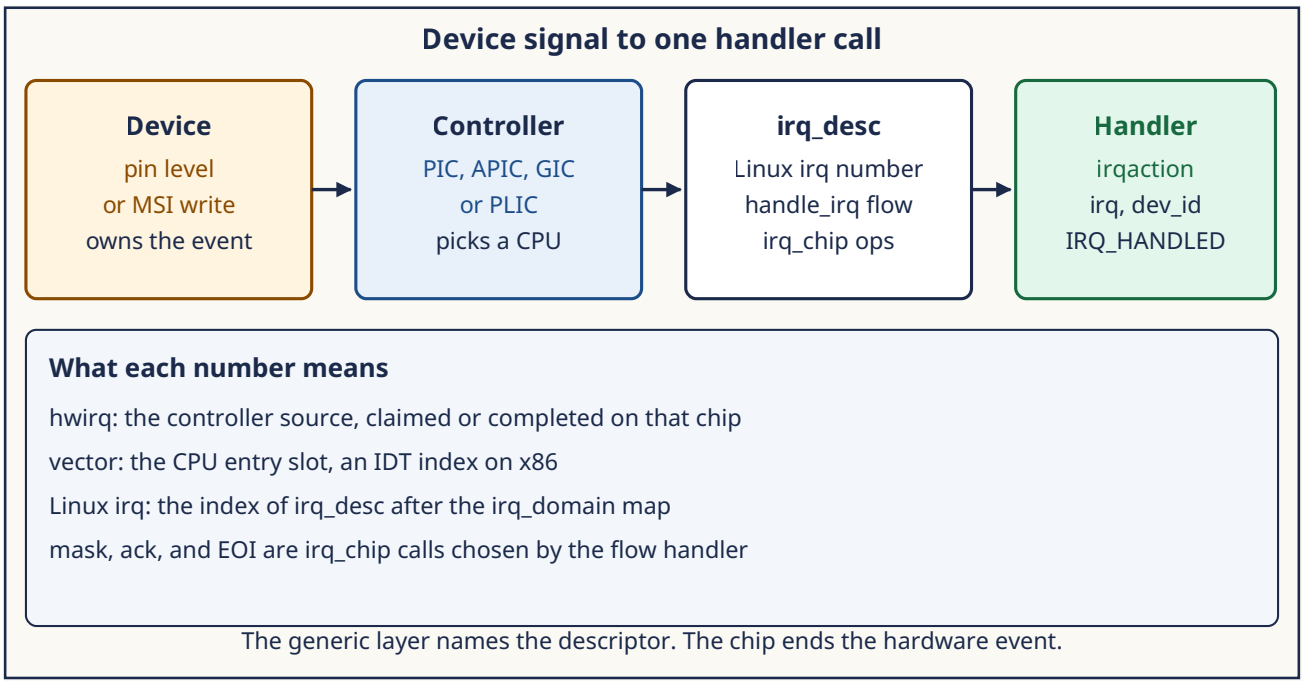
```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/irqdesc.h
 * lines: 67–73
 * function: struct irq_desc
 */
struct irq_desc {
    struct irq_common_data  irq_common_data;
    struct irq_data     irq_data;
    struct irqstat __percpu *kstat_irqs;
    irq_flow_handler_t  handle_irq;
    struct irqaction    *action;    /* IRQ action list */
```

`handle_irq` is the flow handler. `action` is the driver chain. `kstat_irqs` is the per-CPU counter `/proc/interrupts` prints. `irq_data` carries the Linux irq, the hwirq, the chip, and the domain. `lock` is the `raw_spinlock_t` the flow handler holds around the chip calls.

`include/linux/irq.h`, lines 501–512, is the chip: `irq_ack`, `irq_mask`, `irq_mask_ack`, `irq_unmask`, `irq_eoi`, and `irq_set_affinity` just below them. `kernel/irq/chip.c`, `mask_ack_irq`, lines 409–418, calls `irq_mask_ack` when it is set and otherwise calls `irq_mask` plus `irq_ack`.

`include/linux/interrupt.h`, lines 122–136, is the registration: `handler`, `dev_id`, `next`, `thread_fn`, `thread`, `irq`, `flags`, and `name`. `handler` is the hardirq function. `thread_fn` is Day 5. `IRQF_SHARED` is `0x00000080` at line 74.

`request_irq`, lines 168–172, calls `request_threaded_irq` with a NULL thread function and adds `IRQF_COND_ONESHOT`. Return codes are in `include/linux/irqreturn.h`, lines 11–15: `IRQ_NONE` is 0, `IRQ_HANDLED` is 1, `IRQ_WAKE_THREAD` is 2.



The controller picks a CPU and a hardware number. The domain names an irq_desc. The flow handler calls the action.

On x86-64 the entry stubs push the vector. `arch/x86/entry/entry_64.S`, lines 367–378, says the vector occupies the error-code slot and the common path is cache-line aligned. `arch/x86/kernel/irq.c`, `call_irq_handler`, lines 252–259, reads `vector_irq[vector]` on this CPU. A real descriptor goes to `handle_irq`, which on x86-64 calls `generic_handle_irq_desc`, which is `desc->handle_irq(desc)`.

`kernel/irq/handle.c`, `__handle_irq_event_percpu`, lines 147–163, walks the chain:

```
/* tree adc218676eef (Linux 6.12)
 * path: kernel/irq/handle.c
 * lines: 147–163
 * function: __handle_irq_event_percpu
 */
   for_each_action_of_desc(desc, action) {
       irqreturn_t res;

       trace_irq_handler_entry(irq, action);
       res = action->handler(irq, action->dev_id);
       trace_irq_handler_exit(irq, action, res);

       if (WARN_ONCE(!irqs_disabled(),"irq %u handler %pS enabled interrupts\n",
                 irq, action->handler))
           local_irq_disable();
```

Trace one exclusive line. The local APIC presents a vector. The IDT stub enters `common_interrupt`. `vector_irq[vector]` yields the descriptor. The flow handler talks to the chip and calls `handle_irq_event`. That function drops `desc->lock` for the walk (lines 202–214) and takes it again after. The action runs with the Linux irq number and the driver's cookie.

`common_interrupt`, lines 278–288, calls `apic_eoi` only when `call_irq_handler` returns nonzero. That return is the unused-vector path. The successful path ends the interrupt in the chip. `arch/x86/kernel/apic/vector.c`, `apic_ack_irq`, lines 907–910, calls `irq_move_irq` and then `apic_eoi`.

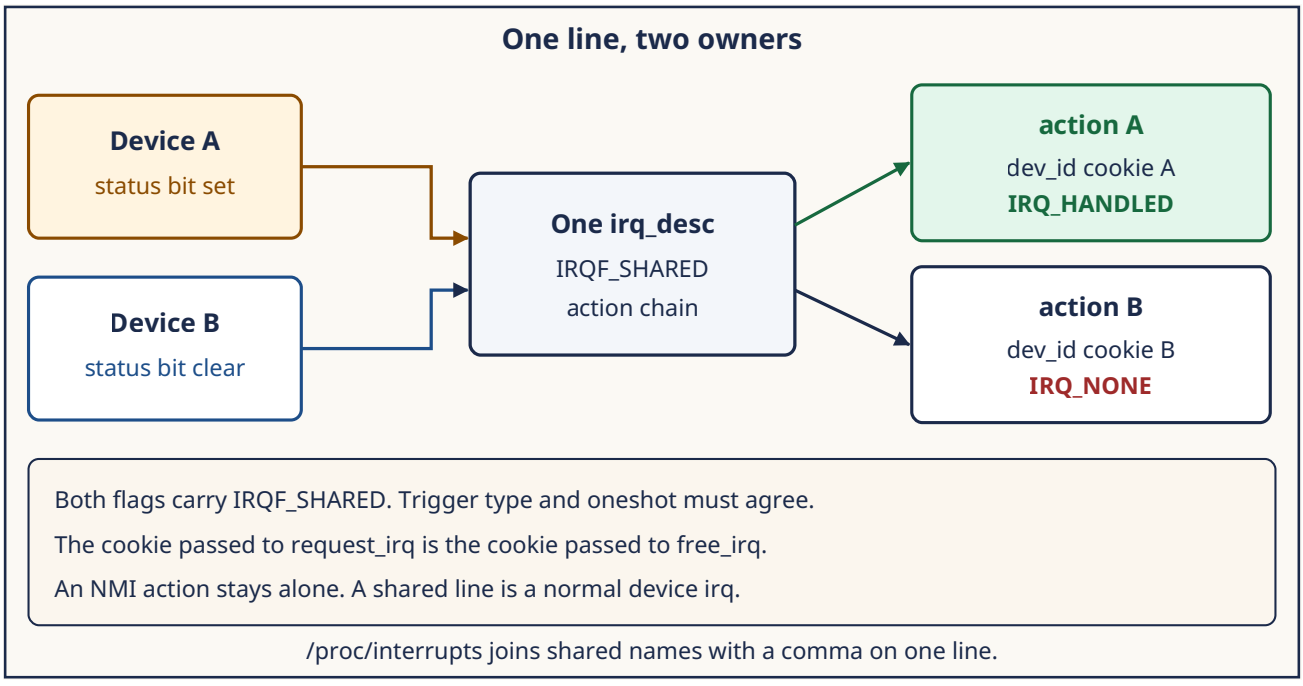
7. Shared IRQ ownership

`kernel/irq/manage.c`, `__setup_irq`, lines 1617–1648, runs when a second action arrives. The comment says both flag words must have `IRQF_SHARED`, the trigger type must match, and oneshot must agree. NMI lines cannot be shared. The test is:

```
        if (!((old->flags & new->flags) & IRQF_SHARED) ||
            (oldtype != (new->flags & IRQF_TRIGGER_MASK)))
            goto mismatch;
```

The `mismatch` label at line 1840 prints `Flags mismatch irq %d`. `request_threaded_irq`, lines 2172–2173, rejects a shared request whose `dev_id` is NULL, because `free_irq` finds the action by that cookie. The kernel-doc at lines 2139–2140 says the same thing.

Each handler reads its own device and returns `IRQ_NONE` when the status bit is clear. The walk ORs the return values. `note_interrupt` watches a line nobody claims. A handler that returns `IRQ_HANDLED` for a foreign event hides the real owner.



Device A handles the event. Device B returns IRQ_NONE. Each cookie is the argument free_irq will need.

`kernel/irq/proc.c`, `show_interrupts`, lines 522–526, prints the chain with commas:

```
    action = desc->action;
    if (action) {
        seq_printf(p, "  %s", action->name);
        while ((action = action->next) != NULL)
            seq_printf(p, ", %s", action->name);
    }
```

A comma is evidence of sharing. It is evidence that both devices fired only after you read their status bits, which `/proc/interrupts` does not show.

8. Top-half limits and affinity

The warning in section 6 fires when a hardirq handler turns interrupts back on. The sleep rules are Day 6. The consequences that matter today:

- The handler does not take a mutex, allocate with `GFP_KERNEL`, or copy from a user address that might fault.
- It leaves the device quiet enough that the same event will not be redelivered the instant the flow unmasks or sends EOI.
- It returns `IRQ_HANDLED`, `IRQ_NONE`, or `IRQ_WAKE_THREAD`. The last without a `thread_fn` hits `warn_no_thread` in the same loop.
- It does not assume it holds `desc->lock`. The walk drops that lock.

`irq_set_affinity` in `kernel/irq/manage.c`, lines 472–482, takes a cpumask and fails when the mask has no online CPU. The call reaches `chip->irq_set_affinity` when the chip implements it. The 8259 has nowhere to store a CPU mask. An I/O APIC or a GIC redistributor can steer the next interrupt. An interrupt a CPU has already accepted is not steered by the next write. `apic_ack_irq` runs `irq_move_irq` at EOI so a pending move completes on a boundary the hardware understands.

`/proc/irq/N/smp_affinity` is the userspace view on kernels that export it. This day's wrapper reads `/proc/interrupts`. It does not write affinity. `IRQF_NOBALANCING` and `IRQF_PERCPU` opt a line out of being moved. A local APIC timer is per CPU. It is not a device wire you aim at another package.

9. Architecture lens: mask, ack, and EOI

The order is the flow handler plus the chip. Three handlers in `kernel/irq/chip.c` are the patterns.

Level. `handle_level_irq`, lines 628–650, calls `mask_ack_irq`, runs the handler, then `cond_unmask_irq`. A level line stays asserted while the device is requesting service. Masking stops a reentry until the handler has cleared the device. The kernel-doc at lines 619–626 says that directly.

Edge. `handle_edge_irq`, lines 787–834, acknowledges once, then loops while `IRQS_PENDING` is set. An edge that arrives during the handler is latched. The comment at lines 772–785 says the edge must be acked to be reenabled, and that a second edge may require a mask depending on the controller.

Fast EOI. `handle_fasteoi_irq`, lines 678–736, runs the handler and then `chip->irq_eoi` through `cond_unmask_eoi_irq`. The comment says the chip receives a single end callback. Oneshot threaded interrupts mask until the thread finishes. `IRQCHIP_EOI_THREADED` can defer that EOI until unmask. Day 5 is the thread.

x86. The successful device path uses `apic_ack_irq`, which writes the local APIC EOI register. `common_interrupt` writes that register itself only for a vector with no descriptor.

ARM GICv3. `drivers/irqchip/irq-gic-v3.c`, `gic_mask_irq`, lines 462–464, pokes `GICD_ICENABLER`. `gic_eoi_irq`, lines 695–698, writes the hwirq to `ICC_EOIR1_EL1` and issues `isb`. The end register is a system register, not port `0x20`. The erratum path writes `GICD_ICACTIVER` after a `dsb`. Priority masking through `ICC_PMR_EL1` is a further switch in this file. The order "write EOIR, then deactivate" is a GICv3 rule.

RISC-V PLIC. `drivers/irqchip/irq-sifive-plic.c`, lines 369–387, claims by reading `CONTEXT_CLAIM` and calls `generic_handle_domain_irq`. `plic_irq_eoi`, lines 154–164, writes the hwirq back to that register. The comment says claim, then complete, and that the complete enables the source again. The claim read is the acknowledge. The write is the completion. There is no 8259 EOI byte and no LAPIC EOI register.

Same Linux objects, three chip sequences. Read `desc->handle_irq` and the chip pointers before you decide which write happens first.

10. Execution trace: one timer, then one shared line

Trace the 0.12 timer with the objects that exist in that tree.

1. The PIT counts down and asserts IRQ0 on the master 8259.
2. The master presents vector `0x20` because setup wrote that base.
3. The IDT interrupt gate at slot `0x20` clears IF and enters `_timer_interrupt`.
4. The stub increments `jiffies` and writes `0x20` to port `0x20`.
5. `do_timer` charges the current task and may call `schedule` when the interrupted CPL is 3 and the quantum is spent.
6. `ret_from_sys_call` restores the frame. IF returns with the interrupted `EFLAGS`.

There is no domain lookup and no action list. The vector number is the Linux identity of the line.

Trace a shared level line on the 6.12 x86 path. The numbers below are roles, not a claim about your machine's `/proc/interrupts`.

1. Two drivers have called `request_irq` with `IRQF_SHARED`, the same trigger flags, and distinct `dev_id` cookies. `__setup_irq` linked the second `irqaction` onto `desc->action`.
2. The device that owns the event asserts the wire. The I/O APIC delivers a vector to one CPU's local APIC, chosen from the affinity mask.
3. `common_interrupt` reads `vector_irq[vector]` and calls `desc->handle_irq`. For a level flow that is `handle_level_irq`.
4. `mask_ack_irq` masks the line. The wire may stay asserted. The controller will not present it again until the unmask.
5. `__handle_irq_event_percpu` calls driver A. Driver A sees its status bit and returns `IRQ_HANDLED`. Driver B sees a clear bit and returns `IRQ_NONE`.
6. `cond_unmask_irq` unmasks because the line is enabled and `threads_oneshot` is clear. `apic_ack_irq` writes the local APIC EOI when this chip's flow reaches `irq_eoi`. The exact call sits in the chip, which for the APIC vector domain is `apic_ack_irq`.
7. `show_interrupts` will later print both names joined by a comma, and `kstat_irqs` will have incremented on the CPU that ran step 3.

A GICv3 trace replaces steps 2 and 6. The redistributor presents the interrupt, `gic_mask_irq` clears an enable bit, and `gic_eoi_irq` writes `ICC_EOIR1_EL1`. A PLIC trace claims by reading `CONTEXT_CLAIM` and completes by writing the hwirq back. The action walk in step 5 is the part that stays.

11. Spurious lines, depth, and a disconnected irq

`note_interrupt`, called from `handle_irq_event_percpu` when the irq is not marked `no_debug`, watches the OR of the action return values. A line that keeps returning `IRQ_NONE` increments `irqs_unhandled` on the descriptor. The fields are in `struct irq_desc`: `irq_count`, `last_unhandled`, and `irqs_unhandled` (`include/linux/irqdesc.h`, lines 78–80). The core can then try to quiet a stuck line. That accounting is why a shared handler must return `IRQ_NONE` when the device is idle. Returning `IRQ_HANDLED` for every invocation makes a stuck peer look healthy.

`depth` on the same struct counts nested `disable_irq` calls. The comment in the header calls it disable-depth. The line stays masked until the matching enables bring the depth back to zero. This is a software nest count. It is not the hardware mask bit by itself. The flow handler consults `irqd_irq_disabled` and leaves a disabled line masked, setting `IRQS_PENDING` so a later enable can replay it. `handle_level_irq` does that at lines 642–644 when `action` is missing or the irq is disabled.

`IRQ_NOTCONNECTED` is `(1U << 31)` in `include/linux/interrupt.h`, lines 140–148. `request_threaded_irq` returns `-ENOTCONN` immediately when the irq number equals that sentinel (lines 2156–2157). The comment says a PCI device whose interrupt is not wired gets this value so the driver can tell "not connected" from a generic failure. It is not a vector, and it is not a line you will find in `/proc/interrupts`.

None of these three mechanisms exist in Linux 0.12. A spurious disk interrupt there calls `_unexpected_hd_interrupt` because `do_hd` was NULL. The 6.12 version is a counter, a depth, and a sentinel number, each with a different job.

12. Safe evidence

Run `../challenges/day-004-interrupts.sh`. The wrapper greps the pinned tree for the gate helper, `IRQF_SHARED`, `handle_level_irq`, `gic_eoi_irq`, and the PLIC claim comment. When `/proc/interrupts` is readable it prints a short summary and any line that contains a comma. It does not write `smp_affinity`, load a module, or call `request_irq`.

Before you run it, write three predictions. Which field on a numbered line is the chip name. Whether any line contains a comma, and what section 7 says that comma means. Which lines are words rather than numbers. On x86, `arch_show_interrupts` adds names such as `NMI` and `LOC` after the numbered irqs. Day 7 reads that function.

13. Mastery: Explain · Draw · Observe · Build

Explain

A page fault and a device IRQ can share a saved frame and still reach different functions. Linux 0.12 stores the timer at IDT slot `0x20` and writes EOI to port `0x20`. A GICv3 machine keeps the idea of a mask and an end operation and uses `GICD_ICENABLER` and `ICC_EOIR1_EL1`.

`irq_desc->handle_irq` is the flow. `irqaction->handler` is the driver. A shared registration passes a non-NULL `dev_id`. `IRQ_NONE` means this device did not own the event. `handle_level_irq` masks before the driver. `handle_fasteoi_irq` ends through `irq_eoi`.

Draw

1. The path figure with three numbers: hwirq, vector, Linux irq. Use a line from `/proc/interrupts` if you have one, or the 0.12 timer (wire 0, vector `0x20`, no domain).
2. Two actions on one descriptor, and the `free_irq` cookie that removes only one of them.
3. Level versus fast-EOI: where mask sits relative to the handler, and where EOI sits.

Observe

From the wrapper, record `uname -r` and one numbered line's chip field. If a comma appears, copy both names. If the tree grep finds `gic_eoi_irq` and `plic_irq_eoi`, record that both exist in this commit and that a running machine uses one of them.

Build

Write a one-page trace of one interrupt from the device to the return. Use the 0.12 timer if you want the whole stub in view, and name the 6.12 objects that replace the `outb` to port `0x20`. End with the sentence you will stand on: mask, ack, and EOI order come from the flow handler and the `irq_chip`.

Why? notebook

1. Why does 0.12 `setup.S` move the 8259 bases from `0x08` to `0x20`?
2. Why is the timer gate type 14 while the syscall gate is type 15 with DPL 3?
3. Why does `_timer_interrupt` write port `0x20` before `do_timer`?
4. Why is `vector_irq[vector]` not the same integer as the Linux irq on a modern x86 machine?
5. Why does `__setup_irq` require both actions to pass `IRQF_SHARED`?
6. Why does `free_irq` need the same `dev_id` the driver passed in?
7. Why does `handle_level_irq` mask before the driver runs?
8. Why does `common_interrupt` call `apic_eoi` on the failure return?
9. Why is a PLIC complete a write of the hwirq, not a write of `0x20`?

Next: Day 5 moves the long part of the handler out of hardirq into a softirq, a threaded handler, or a workqueue. The descriptor and the chip order from this day stay in place. What changes is which context runs after the top half returns.

Sleep rules for that later context are Day 6.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Interrupts

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 14-kernel-entry-contexts-and-interrupts/challenges/day-004-interrupts.sh
```

- View lab source on GitHub

Day 5 — Softirqs, Threaded IRQs, and Workqueues

Target time: approximately 3 hours

- Why the hardirq path has to end: 25 minutes
- What Linux 0.12 deferred, and the table it does not have: 35 minutes
- Softirq, tasklet, threaded IRQ, and workqueue in 6.12: 80 minutes
- Mastery, then the wrapper: 25 minutes

> The hardirq handler acknowledges the device and returns. The rest of the work runs in a context that matches its length and its need to sleep. Linux 6.12 has softirqs, threaded IRQ handlers, and workqueues. Tasklets still run, and the header marks that API deprecated.

Day 4 ended in `action->handler`. This day is what that function is allowed to start, and who finishes it.

The 6.12 excerpts are from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. The 0.12 excerpts are from the historic `linux-0.12` tree. Record `uname -r` beside any line you copy from the booted kernel.

1. Syntax needed for this source

A pending set is a bitmask. A handler table is an array of function pointers indexed by that bit. A work item is a struct on a list.

```
unsigned long pending;
void (*action)(void);

while ((bit = ffs(pending))) {
    action();
    pending >>= bit;
}

struct work_struct {
    atomic_long_t data;
    struct list_head entry;
    work_func_t func;
};
```

`ffs` returns the index of the least significant set bit, one-based. Shifting `pending` by that count drops the bit just handled. The softirq function pointer has no `dev_id`. The action looks at per-CPU data.

The work item's `data` word packs flags and a pool pointer. `entry` is the list node. `func` runs later. The caller of `queue_work_on` does not call `func`.

2. Why the rest of the work leaves the hardirq

Day 4's top half runs with the interrupted context still on the stack and, on the normal path, with the CPU's interrupt mask closed. A packet copy or a sleep either holds that mask or reopens it and accepts nesting. Linux 0.12 already shows the pressure. `do_timer` runs inside `_timer_interrupt` after the EOI. The EOI lets the 8259 raise another timer interrupt. Anything slow in that function delays every other interrupt and the task that was running.

The split is: the hardirq path records the event and quiets the device; a later context does the long part. That later context has to answer three questions. Which CPU runs it? May it sleep? What stops two runs of the same work from overlapping?

3. Linux 0.12 first: queues and pointers, no bottom-half table

A search of the Linux 0.12 tree finds no `bh_base`, no `bh_active`, and no `bh_mask`. The bottom-half bitmap is a later design. This kernel defers work with three mechanisms you can read end to end. Do not paste a 32-slot `bh_base` into this tree.

The timer list is walked inside the interrupt. `kernel/sched.c`, `do_timer`, lines 351–361:

```
    if (next_timer) {
        next_timer->jiffies--;
        while (next_timer && next_timer->jiffies <= 0) {
            void (*fn)(void);

            fn = next_timer->fn;
            next_timer->fn = NULL;
            next_timer = next_timer->next;
            (fn)();
        }
    }
```

`fn` runs before `_timer_interrupt` returns. The list is the ancestor of timer softirq in the sense that a tick drains expired callbacks. The callback's context is the timer interrupt. `do_timer` also calls `schedule` when the user-mode quantum expires and the CPL is not zero (lines 364–367). A kernel-mode tick does not reschedule. There is no separate bottom-half pass after the stub.

The hard disk uses a planted function pointer. `kernel/sys_call.s`, lines 251–258:

```
    xorl %edx,%edx
    xchgl _do_hd,%edx
    testl %edx,%edx
    jne 1f
    movl $_unexpected_hd_interrupt,%edx
1:  outb %al,$0x20
    call *%edx      # "interesting" way of handling intr.
```

`xchgl` takes `do_hd` and clears it. The driver stored the completion function before starting the command. One pointer, one CPU, no chain. The floppy stub does the same with `do_floppy`.

The tty path is the one that wakes a sleeper later. `kernel/chr_drv/tty_io.c`, `sleep_if_empty`, lines 97–103, closes interrupts with `cli`, sleeps on `queue->proc_list` while the queue is empty, and opens interrupts with `sti`. The keyboard interrupt fills the queue and wakes the sleeper. The bytes move in the interrupt. The reader runs afterward because it was asleep. `cli` is the lock. There is one CPU, so closing IF is enough.

What carries forward is the split between "the device is quiet" and "a process consumes the data," plus a queue and a sleep. What stays in 1992 is timer callbacks inside the interrupt and one global function pointer per block device.

4. Evolution: bitmap, softirq, tasklet, thread, workqueue

Later uniprocessor kernels added a bottom-half bitmap: pending bits, a mask, and a table of functions run on the way out of an interrupt. That design is not in the 0.12 tree opened here, so this file does not invent line numbers for it. The properties people used it for show up in the 6.12 successors: a small fixed set of deferred slots, run soon without scheduling a process, and serialized on one CPU because there was one CPU.

SMP breaks "one CPU implies one run." Softirqs keep the pending bitmask and make it per CPU. Tasklets sit on two of those slots and add "this callback does not run on two CPUs at once." Threaded IRQs move the long part into a kernel thread that may sleep. Workqueues move other work onto worker threads. One workqueue flag runs the function back in softirq context.

`include/linux/interrupt.h` still contains both generations of advice. Lines 549–553 tell you not to add a new softirq number:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/interrupt.h
 * lines: 549–553
 */
/* PLEASE, avoid to allocate new softirqs, if you need not _really_ high
   frequency threaded job scheduling. For almost all the purposes
   tasklets are more than enough. F.e. all serial device BHs et
   al. should be converted to tasklets, not to softirqs.
 */
```

Lines 626–632, immediately above `struct tasklet_struct`, say:

```
/* Tasklets --- multithreaded analogue of BHs.

   This API is deprecated. Please consider using threaded IRQs instead:
   https://lore.kernel.org/lkml/20200716081538.2sivhkj4hcyrusem@linutronix.de
```

Both comments are in this commit. The first is about the scarce softirq vector: do not add an eleventh `NR_SOFTIRQS` entry for a device. The second is about the tasklet API: consider a threaded IRQ for new code. The second does not delete the first. The first does not cancel the deprecation. Tasklets are compiled, exported, and called by existing drivers in 6.12. "Removed" would be false. "The recommended API for new code" would also be false, because the struct comment says deprecated.

Two helpers carry a narrower warning. `tasklet_disable_in_atomic`, lines 736–738, says "Do not use in new code." `tasklet_unlock_spin_wait` in `kernel/softirq.c`, lines 843–846, says the same about waiting for a tasklet from atomic context. Those sentences are about those waits. They are not a removal of `tasklet_schedule`.

The header still documents the runtime rules at lines 631–646. A tasklet runs on one CPU at a time. Different tasklets may run on different CPUs. A second schedule while one is queued collapses to one run. A schedule while it is running reschedules it. Anything shared with a different tasklet uses a spinlock.

5. ToyOS runs the tick inside the interrupt

ToyOS has the 0.12 timer shape. Vector 32 in `interrupt_handler` increments `ticks`, writes the PIC EOI, and calls `scheduler_tick` before it returns. There is no softirq pending word, no `ksoftirqd`, and no `irq/N-name` thread. The scheduler tick is part of the interrupt stack. Linux 6.12's `TIMER_SOFTIRQ` is the later split of that work.

6. The softirq vector and who runs it

The fixed enum is `include/linux/interrupt.h`, lines 555–569:

```
enum
{
    HI_SOFTIRQ=0,
    TIMER_SOFTIRQ,
    NET_TX_SOFTIRQ,
    NET_RX_SOFTIRQ,
    BLOCK_SOFTIRQ,
    IRQ_POLL_SOFTIRQ,
    TASKLET_SOFTIRQ,
    SCHED_SOFTIRQ,
    HRTIMER_SOFTIRQ,
    RCU_SOFTIRQ,    /* Preferable RCU should always be the last softirq */

    NR_SOFTIRQS
};
```

Ten slots. `HI_SOFTIRQ` runs tasklet-hi. `TASKLET_SOFTIRQ` runs ordinary tasklets. `open_softirq` installs the function. Adding a name here is the allocation the comment in section 4 asks you to avoid.

`raise_softirq` saves the interrupt mask and calls `raise_softirq_irqoff`. `kernel/softirq.c`, lines 667–684:

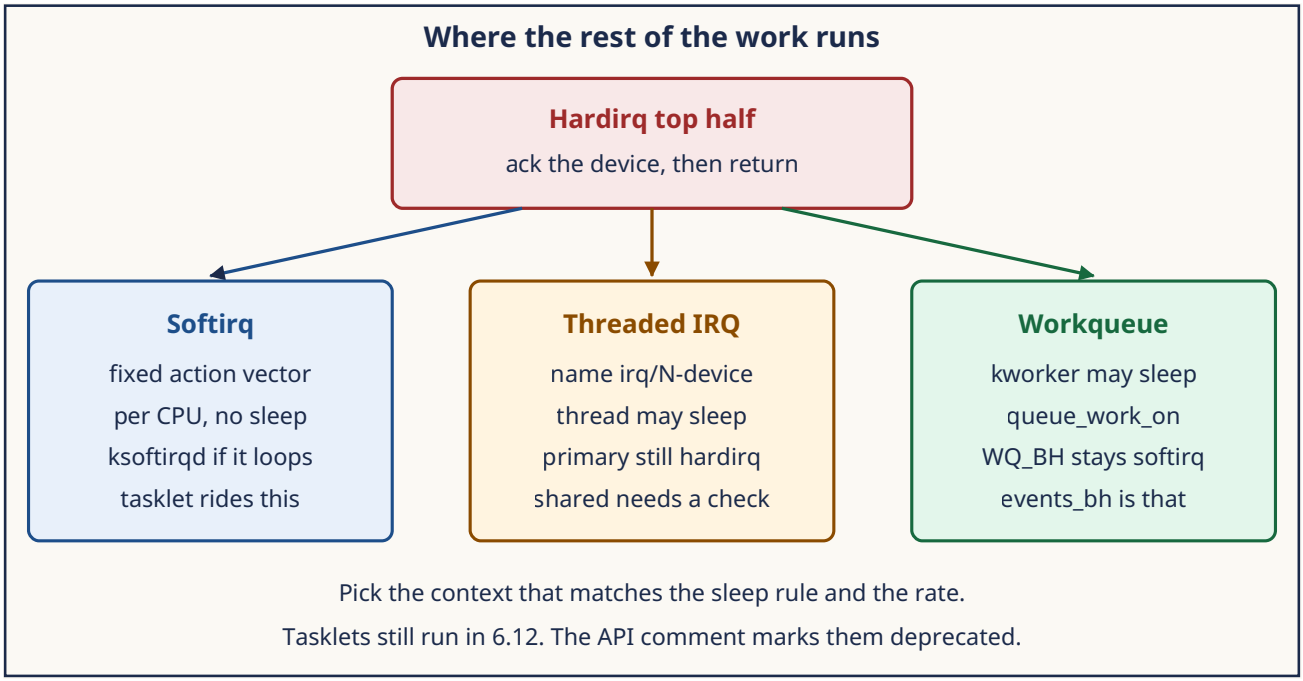
```
/* tree adc218676eef (Linux 6.12)
 * path: kernel/softirq.c
 * lines: 667–684
 * function: raise_softirq_irqoff
 */
inline void raise_softirq_irqoff(unsigned int nr)
{
    __raise_softirq_irqoff(nr);
    /*
     * If we're in an interrupt or softirq, we're done
     * (this also catches softirq-disabled code). We will
     * actually run the softirq once we return from
     * the irq or softirq.
     *
     * Otherwise we wake up ksoftirqd to make sure we
     * schedule the softirq soon.
     */
    if (!in_interrupt() && should_wake_ksoftirqd())
        wakeup_softirqd();
}
```

A raise from hardirq sets the per-CPU pending bit and returns. `__irq_exit_rcu`, lines 627–637, subtracts `HARDIRQ_OFFSET` and, when the pending mask is set and the CPU is no longer in interrupt context, calls `invoke_softirq`.

`handle_softirqs` bounds the loop with `MAX_SOFTIRQ_TIME` and `MAX_SOFTIRQ_RESTART`. Lines 542–561 call `h->action()` and complain if `preempt_count` changes across the call. If the mask is still set when the budget runs out, the function wakes `ksoftirqd`. The thread's `comm` is `ksoftirqd/%u` (line 971).

The action runs with the softirq count raised. `ksoftirqd` is a task. The actions it enters are still softirq context for `in_serving_softirq`. Day 6 draws that bit. Sleeping in the action is the bug `might_sleep` reports.

Each CPU has its own pending mask. The same softirq number can run on two CPUs at once. The action uses per-CPU lists or a lock. A tasklet is the exception inside that rule.



Softirq stays non-sleeping and per CPU. A threaded handler may sleep. A normal work item may sleep. WQ_BH stays in softirq context.

7. Tasklets: still here, marked deprecated, serialized with themselves

`tasklet_schedule` sets `TASKLET_STATE_SCHED` and, if it was clear, queues the tasklet on this CPU's `tasklet_vec` and raises `TASKLET_SOFTIRQ`. If the bit was already set, the second schedule does not queue a second copy.

`tasklet_action_common`, lines 763–792, pulls that per-CPU list and tries `tasklet_trylock`. On SMP the lock is `TASKLET_STATE_RUN`. The same tasklet running on another CPU fails the trylock and is put back. Different tasklets take different struct bits, so they overlap. On a uniprocessor build the trylock returns true, which matches the old one CPU story.

`tasklet_kill` prints `Attempt to kill tasklet from interrupt` when `in_interrupt` is true, then waits until the scheduled bit clears. That wait sleeps. It belongs in process context.

Status line for this commit: the implementation runs. The API comment says deprecated and names threaded IRQs. Existing drivers still call `tasklet_setup`. New code in this course follows the deprecation comment. The older "tasklets are more than enough" sentence remains the reason not to add a softirq slot for a device.

8. Threaded IRQs

`request_threaded_irq` takes a primary `handler` and a `thread_fn`. The kernel-doc in `kernel/irq/manage.c`, lines 2126–2146, is the contract. The primary still runs in hardirq context and checks whether this device raised the line. If it did, it quiets the device and returns `IRQ_WAKE_THREAD`. The comment says this split is required for shared interrupts. `IRQF_ONESHOT` runs `thread_fn` with the line masked.

`__handle_irq_event_percpu` calls `__irq_wake_thread` for `IRQ_WAKE_THREAD`. Waking every thread on a shared chain for every edge would be wrong. The primary is the filter.

`setup_irq_thread`, lines 1457–1488, creates `irq/%d-%s`, or `irq/%d-s-%s` for a secondary forced thread. It holds a reference on the `task_struct` so a late interrupt does not touch a freed task. It sets `IRQTF_AFFINITY` so the thread moves onto the CPUs the irq affinity names.

A primary that returns `IRQ_HANDLED` does not wake the thread. The oneshot unmask path from Day 4 then runs because `threads_oneshot` is clear. The thread function itself may sleep. A stuck thread holds a oneshot mask and the device stays quiet. `synchronize_irq` waits on `threads_active` for that reason.

`force_irqthreads` threads handlers that did not ask for it. `IRQF_NO_THREAD` opts out. New device code in this lesson follows the kernel-doc: a primary that can return `IRQ_NONE`, and a `thread_fn` when the rest may sleep.

9. Workqueues, including the ones that do not sleep

`include/linux/workqueue_types.h`, lines 16–20, is `struct work_struct`: `data`, `entry`, and `func`. `queue_work_on` is declared at `include/linux/workqueue.h` lines 586–587. A worker thread calls `func`. That worker is process context. Sleeping blocks the worker, not the interrupted task.

`WQ_BH` is the exception. Line 370:

```
    WQ_BH          = 1 << 0, /* execute in bottom half (softirq) context */
```

`kernel/workqueue.c` allocates `system_bh_wq` as `events_bh` with that flag (line 7802). Lines 5671–5675 say BH workqueues share a single execution context per CPU. A function queued there follows the softirq sleep rule. `queue_work` does not tell you the context until you see which workqueue.

The pending bit collapses a second queue of the same item while the first is still pending. Flush and cancel wait in process context. Calling them from the hardirq handler is the same class of mistake as `tasklet_kill` from interrupt context.

10. Context and serialization

Deferred contexts side by side

Context	Who runs it	Sleep	Serialization
Hardirq	IRQ entry on this CPU	**No**	desc lock, IF off
Softirq	irq exit or ksoftirqd/%u	**No**	per-CPU pending tasklet self-lock
Threaded	irq/N-name kthread	**Yes**	oneshot mask until the thread returns
Work item	kworker thread or events_bh	**Yes** **No if WQ_BH**	pool and CPU

A tasklet is a softirq client. It is serialized with itself.
Different tasklets may run on different CPUs at the same time.

Hardirq and softirq stay non-sleeping. The threaded handler and a normal worker may sleep. WQ_BH follows the softirq row.

Context	Runs as	Sleep	Same work overlaps?
Hardirq handler	IRQ entry on that CPU	no	`desc->lock` covers this irq
Softirq action	irq exit or `ksoftirqd/%u`	no	same slot may run on two CPUs
Tasklet	`TASKLET_SOFTIRQ` or `HI_SOFTIRQ`	no	one tasklet, one CPU
Threaded IRQ	`irq/N-name`	yes	oneshot holds the line
Worker	kworker	yes	one item is not queued twice while pending
`WQ_BH` item	`events_bh`	no	one BH context per CPU

High-rate per-CPU work that already has a softirq slot stays there. Device work that must sleep becomes a threaded IRQ. A retry or a free that is not the interrupt itself goes to a normal workqueue. A tasklet is the cell older drivers used for non-sleeping self-serialized work. The 6.12 header tells new code to consider a threaded IRQ instead.

11. One pending bit, from raise to action

Take `TIMER_SOFTIRQ`. The timer hardirq raises that bit with interrupts still off. `raise_softirq_irqoff` sees `in_interrupt` true and does not wake `ksoftirqd` yet. The hardirq returns. `__irq_exit_rcu` drops `HARDIRQ_OFFSET`. The pending mask is nonempty, so `invoke_softirq` runs.

`ffs` picks the lowest set bit, so `HI_SOFTIRQ` runs before `TIMER_SOFTIRQ` when both are pending. `h->action()` for the timer slot is the timer subsystem's softirq, not the device's hardirq handler. `kstat_incr_softirqs_this_cpu` is the counter `/proc/softirqs` shows. If `preempt_count` changes across the action, the core prints the "huh, entered softirq" line and repairs the count. That print is a bug report.

When the restart budget is spent, `wakeup_softirqd` runs and the interrupted task gets the CPU back. The softirq thread later enters `handle_softirqs` again. Lockdep accounts for the handoff. `lockdep_softirq_start`, lines 485–496, drops the hardirq lockdep state for the softirq pass, and `lockdep_softirq_end` restores it, so a lock taken in the softirq is not recorded as a hardirq lock.

`trace_softirq_entry` and `trace_softirq_raise` are the optional tracefs names Day 7 mentions. Reading `/proc/softirqs` twice is enough to see the timer row move.

12. Three drivers, three choices

The table in section 10 is easier to trust on three concrete jobs.

A network receive that already owns `NET_RX_SOFTIRQ` builds a per-CPU list in the hardirq, raises that softirq, and returns. The softirq action walks the list. It may run on every CPU that received a packet. It does not take a mutex. If the walk exceeds the softirq budget, `ksoftirqd/%u` continues it. That is the high-rate cell.

A sensor driver that must wait for a completion or take a mutex uses `request_threaded_irq`. The primary reads the status register. A foreign device on a shared line gets `IRQ_NONE`. The owning device is quieted, the primary returns `IRQ_WAKE_THREAD`, and `irq/N-name` runs `thread_fn` with `IRQF_ONESHOT` holding the line masked. The thread may sleep. It may not forget to re-enable the device, because the oneshot mask stays until the thread returns.

A driver's `remove` path has to free a buffer the interrupt might still touch. That free is not the interrupt handler. `queue_work_on` puts a `work_struct` on a normal workqueue. The kworker sleeps if it must. The hardirq only queues. If someone instead uses `system_bh_wq`, the function runs as `WQ_BH` and the sleep is illegal again. The workqueue pointer is part of the decision, not a decoration.

A fourth job, an old driver with a tasklet, stays understandable and stays legacy. `tasklet_schedule` from the hardirq raises `TASKLET_SOFTIRQ`. The callback does not sleep. It does not run beside itself. New code that wants that shape and also wants to sleep uses the threaded IRQ. New code that wants that shape and must not sleep can still use a per-CPU list inside an existing softirq, which is what the "avoid new softirqs" comment is protecting. It does not require a new tasklet.

`handle_softirqs` refuses to be the place those long jobs hide. After the action loop, lines 569–577:

```
/* tree adc218676eef (Linux 6.12)
 * path: kernel/softirq.c
 * lines: 569–577
 * function: handle_softirqs
 */
   local_irq_disable();

   pending = local_softirq_pending();
   if (pending) {
       if (time_before(jiffies, end) && !need_resched() &&
           --max_restart)
           goto restart;

       wakeup_softirqd();
   }
```

`need_resched` is a flag Day 6 names and Week 15 explains. Here it is only the reason the softirq pass gives the CPU back. The pending bits remain set. `ksoftirqd` will observe them. A design that needed the hardirq exit path to finish a userspace copy has already chosen the wrong context.

The three jobs also name what you will look up in `/proc`. The network job moves `NET_RX` in `/proc/softirqs`. The sensor job creates a `irq/N-name` task. The deferred free creates a `kworker` task and does not, by itself, add a row to `/proc/softirqs`. A tasklet job moves `TASKLET` or `HI` and does not create a task of its own. Seeing a kworker therefore does not tell you an interrupt fired. Seeing `NET_RX` move does not tell you which device. Day 7 keeps those hypotheses apart.

13. Safe evidence

Run `../challenges/day-005-deferred-work.sh`. The wrapper greps the pinned tree for the deprecation sentence, the softirq enum, `ksoftirqd/%u`, and `WQ_BH`. When `/proc/softirqs` is readable it prints the header and the tasklet rows. It scans command names for `ksoftirqd/` and `irq/`. It does not raise a softirq or change a thread's affinity.

Predict before you run. The columns are CPUs. The row names match the enum. `ksoftirqd/0` is a task whose softirq actions are still softirq context. An `irq/N-name` line is a threaded handler. There is no `/proc/tasklets`. Tasklet runs show up as the `TASKLET` or `HI` row. The same wrapper then polls an `eventfd` and a `timerfd`. Those are userspace analogs. `not_a_softirq`, `not_an_irq_handler`, and `not_a_workqueue` mark that limit.

14. Mastery: Explain · Draw · Observe · Build

Explain

`do_timer` in Linux 0.12 runs timer callbacks before the stub returns. `TIMER_SOFTIRQ` is the later home for that work. The 0.12 tree has no bottom-half table. The two `interrupt.h` comments you quote are the "avoid new softirqs" paragraph and the "This API is deprecated" paragraph. `raise_softirq` from hardirq sets a bit. `invoke_softirq` on the way out calls the action, or `ksoftirqd/%u` does it later. A shared threaded IRQ keeps a hardirq primary so it can return `IRQ_NONE`. `IRQ_WAKE_THREAD` wakes `thread_fn`. `WQ_BH` puts `work_struct.func` in softirq context.

Draw

1. The choice figure for a device that must take a mutex: primary, `irq/N-name`, oneshot mask.
2. The same device if it only enqueues a per-CPU list: the softirq slot, and the fact that the slot may run on two CPUs.
3. One tasklet struct with `TASKLET_STATE_SCHED` and `TASKLET_STATE_RUN`, and a second tasklet that is allowed to run beside it.

Observe

From the wrapper, record `uname -r` and one softirq row whose count moved, or the row you sampled once if you only read the file one time. Copy one `ksoftirqd` name and, if present, one `irq/` name. If the grep prints both tasklet comments, quote the deprecated sentence in your notes and the sentence that still says tasklets are enough for avoiding a new softirq number.

Build

Write half a page that places a tasklet in 6.12 without overclaiming. State that the code runs. Quote both comments. Say which context a new sleeping handler should use. End with the serialization sentence: a tasklet is exclusive with itself and concurrent with other tasklets.

Why? notebook

1. Why does 0.12 call timer callbacks from `do_timer` instead of from a second pass?
2. Why is the absence of `bh_base` in that tree a fact about 0.12, and a bottom-half bitmap a later design?
3. Why can `NET_RX_SOFTIRQ` run on two CPUs at once?
4. Why does `tasklet_trylock` exist if softirqs already have a per-CPU list?
5. Why does the deprecation comment not mean `tasklet_schedule` is compiled out of v6.12?
6. Why does a shared threaded IRQ still need a hardirq primary?
7. Why does `IRQF_ONESHOT` keep the line masked until the thread returns?
8. Why is `events_bh` a workqueue that still must not sleep?
9. Why does `handle_softirqs` wake `ksoftirqd` instead of looping until the pending mask is empty?

Next: Day 6 names the counter that makes "may I sleep here?" a check, and keeps the scheduler's run-queue policy for Week 15. The counter is the subject. The policy is not.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Deferred Work

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 14-kernel-entry-contexts-and-interrupts/challenges/day-005-deferred-work.sh
```

- View lab source on GitHub

Day 6 — Preemption, Atomic Context, and Sleepability

Target time: approximately 3 hours

- `cli` in Linux 0.12 versus a counter any helper can read: 35 minutes
- `preempt_count` fields, hardirq, softirq, and NMI: 70 minutes
- Sleep rules, `might_sleep`, and lockdep: 45 minutes
- Mastery, then the wrapper: 25 minutes

> A helper three calls down from an interrupt does not remember the stub that entered it. Linux records that fact in `preempt_count`. Sleeping is legal only in some of those fields. The scheduler that would run after a legal sleep is Week 15.

Day 4 placed the driver in hardirq. Day 5 moved some of the work to a softirq, a thread, or a workqueue. This day is the rule those choices are enforcing.

The 6.12 excerpts are from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. Record `uname -r` before you treat a line as true of the booted kernel. The wrapper's source check requires this commit.

1. Syntax needed for this source

`preempt_count` is one integer used as several small counters. A shift selects a field. A mask isolates it. Adding a constant enters a context. Subtracting the same constant leaves it.

```
#define PREEMPT_BITS    8
#define SOFTIRQ_BITS    8
#define HARDIRQ_BITS    4
#define NMI_BITS        4

#define PREEMPT_SHIFT   0
#define SOFTIRQ_SHIFT   (PREEMPT_SHIFT + PREEMPT_BITS)
#define HARDIRQ_SHIFT   (SOFTIRQ_SHIFT + SOFTIRQ_BITS)
#define NMI_SHIFT       (HARDIRQ_SHIFT + HARDIRQ_BITS)

#define HARDIRQ_OFFSET  (1UL << HARDIRQ_SHIFT)
#define in_hardirq()    (hardirq_count())
```

Bits 0–7 are the preemption depth. Bits 8–15 are the softirq field. Bits 16–19 are hardirq. Bits 20–23 are NMI. Those ranges are what `include/linux/preempt.h` states at lines 14–31. The high bit, `PREEMPT_NEED_RESCHED`, is a flag the scheduler consults. Week 15 is where that flag becomes a run-queue decision. This day only needs to know the bit is not one of the four context counters.

A nonzero hardirq, softirq-serving, or NMI field means this context must not sleep. A nonzero preemption field means "do not preempt me here." Those are related and they are not the same test. Holding a spinlock is a third reason not to sleep, and on a non-preemptible kernel the preemption field may not move when the lock is taken.

2. Linux 0.12 first: `cli`, `sti`, and a CPL test

Linux 0.12 has no `preempt_count`. `include/asm/system.h`, lines 16–17, is the whole interrupt mask API:

```
#define sti() __asm__ ("sti"::)
#define cli() __asm__ ("cli"::)
```

`cli` clears IF. `sti` sets it. There is one CPU, so closing IF is mutual exclusion against every interrupt stub. `sleep_if_empty` in `kernel/chr_drv/tty_io.c` wraps the empty-queue wait in `cli` and `sti`. The sleep itself is `interruptible_sleep_on`, which is process context that has chosen to wait. The timer stub does not call it.

`do_timer` in `kernel/sched.c`, lines 364–367, is the other half of the convention:

```
    if ((--current->counter)>0) return;
    current->counter=0;
    if (!cpl) return;
    schedule();
```

`cpl` came from the low two bits of the saved CS. A tick that interrupted the kernel returns without `schedule`. A tick that interrupted user mode may schedule. The kernel does not ask a counter whether this is atomic. It relies on the stub not calling `schedule` except through this test, and on drivers not calling `sleep_on` from the interrupt body. A helper three calls down has no way to check. The convention works while every author remembers the stub.

What carries forward is the split between "this code owns the CPU until it returns" and "this code may wait." What stays in 0.12 is encoding that split only as IF and as a CPL test in one function.

3. Evolution: one word any helper can read

Nesting breaks a single IF bit. A hardirq that raises a softirq on the way out is two contexts on one stack. An NMI can arrive while IF is already clear. A second CPU does not share IF. The kernel needed a value a helper could read without knowing which stub called it.

`preempt_count` is that value. `preempt_disable` adds `PREEMPT_OFFSET` when `CONFIG_PREEMPT_COUNT` is set, and adds nothing when it is not (`include/linux/preempt.h`, lines 148–151). Softirq entry adds `SOFTIRQ_OFFSET`. `__irq_enter` in `include/linux/hardirq.h`, lines 34–39, adds `HARDIRQ_OFFSET`, tells lockdep, and accounts the time. `__nmi_enter`, lines 107–113, adds `NMI_OFFSET + HARDIRQ_OFFSET` together, so an NMI is visible to both `in_nmi` and `in_hardirq`. The comment above it says NMI entry can nest up to 15 times because `NMI_BITS` is 4.

`might_sleep` is the check a sleeping function leaves at its door. Under `CONFIG_DEBUG_ATOMIC_SLEEP`, `include/linux/kernel.h` lines 93–106 define it as `__might_sleep` plus `might_resched`. Without that config, line 148 is only `might_resched`. The bug print and the voluntary preemption point are different jobs packed into one macro. Week 15 explains what a reschedule does with a run queue. This week stops at "this is a legal place to ask."

4. ToyOS has `sti` and no counter

`interrupts_enable` in `projects/toyos/src/arch.c` is the `sti` instruction. The timer path writes the PIC EOI and returns. It does not wait. There is no per-task preemption count and no `might_sleep`. The toy can still teach the rule in words: the stub acknowledges the controller and returns. The host lab in section 12 prints that rule as data. Neither one reads a Linux `preempt_count`.

5. The fields in this tree

`include/linux/preempt.h`, lines 14–53, is the map. The comment is worth reading as the specification:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/preempt.h
 * lines: 27–31 and 50–55
 * function: preempt_count field macros
 */
 *         PREEMPT_MASK:    0x000000ff
 *         SOFTIRQ_MASK:    0x0000ff00
 *         HARDIRQ_MASK:    0x000f0000
 *             NMI_MASK:    0x00f00000
 * PREEMPT_NEED_RESCHED:    0x80000000
 */
#define HARDIRQ_OFFSET  (1UL << HARDIRQ_SHIFT)
#define NMI_OFFSET  (1UL << NMI_SHIFT)
#define SOFTIRQ_DISABLE_OFFSET  (2 * SOFTIRQ_OFFSET)
```

`SOFTIRQ_OFFSET` is bit 8, the least significant softirq bit. `SOFTIRQ_DISABLE_OFFSET` is twice that, so `local_bh_disable` sets bit 9. `in_serving_softirq` tests bit 8 only:

```
#define in_serving_softirq()    (softirq_count() & SOFTIRQ_OFFSET)
```

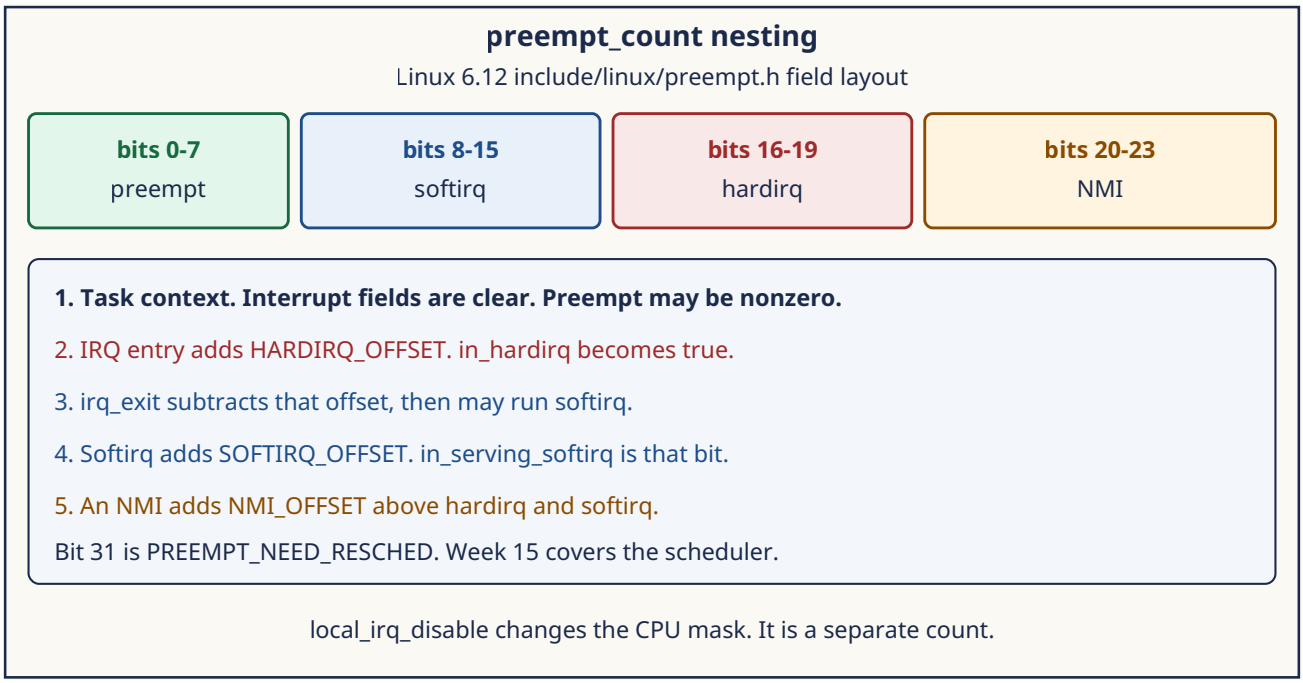
A CPU inside `__do_softirq` has bit 8 set. A CPU that called `local_bh_disable` and is still in process context has bit 9 set and bit 8 clear. `in_softirq` is the whole softirq mask, so it is true for both. `in_interrupt` on a non-RT kernel is `irq_count()`, which includes the softirq mask, so BH-disabled code counts as `in_interrupt` even though it is not serving a softirq. The names are easy to swap. The masks are the definition.

`CONFIG_PREEMPT_RT` changes `softirq_count` to `current->softirq_disable_cnt & SOFTIRQ_MASK` (lines 110–116). The field diagram in the comment is the non-RT layout. Quote the `ifdef` when you are on an RT kernel. This lesson's sleep table is the same either way: softirq context does not sleep.

`interrupt_context_level`, lines 81–99, folds the masks into 0, 1, 2, or 3:

```
 *  0 - normal context
 *  1 - softirq context
 *  2 - hardirq context
 *  3 - NMI context
 */
    level += !!(pc & (NMI_MASK));
    level += !!(pc & (NMI_MASK | HARDIRQ_MASK));
    level += !!(pc & (NMI_MASK | HARDIRQ_MASK | SOFTIRQ_OFFSET));
```

NMI entry added both NMI and hardirq offsets, so the first two terms fire, and the third term fires because the hardirq bit is set. The level is 3. A plain hardirq fires the second and third terms: level 2. A serving softirq fires only the third term, because that term looks at `SOFTIRQ_OFFSET` and not at the BH-disable bit. Level 1. BH-disabled process context stays level 0. That is why "bottom halves are off" is not the same sentence as "I am inside a softirq."



Bits 0–7 preempt, 8–15 softirq, 16–19 hardirq, 20–23 NMI. Hardirq exit may then run softirq and add SOFTIRQ_OFFSET.

6. Process, hardirq, softirq, NMI

Process context is `in_task`: the NMI, hardirq, and serving-softirq bits are clear (lines 129–132). A system call, a kernel thread, and a workqueue worker start here. Sleep is legal until something in this section disables it.

Hardirq context is `in_hardirq`, which is `hardirq_count()`. `__irq_enter` added `HARDIRQ_OFFSET`. Day 4's `action->handler` runs here. The warning in `__handle_irq_event_percpu` fires if the handler enables interrupts. Sleeping here is the bug.

Softirq context is `in_serving_softirq`. Day 5's `h->action`, including tasklets, runs here. `ksoftirqd` is a task, and the actions it enters still have `SOFTIRQ_OFFSET` set. Sleeping in the action is the same bug as sleeping in the hardirq handler.

NMI context is `in_nmi`. `__nmi_enter` also raised the hardirq field, so `in_hardirq` is true as well. The extra constraint is that the locks you take must be safe against an NMI that interrupted the owner of an ordinary spinlock. `handle_fasteoi_nmi` in Day 4 runs one action and does not share the line. This day does not add a second NMI handler. It only records that NMI is a stricter atomic context than hardirq.

Threaded IRQ handlers and ordinary kworkers are process context. They may sleep. `WQ_BH` work runs in softirq context, which is Day 5's `events_bh`. The workqueue pointer decides. The function pointer does not.

7. Spinlocks and the local interrupt mask

`local_irq_disable` is the modern `cli`, plus a trace hook when `CONFIG_TRACE_IRQFLAGS` is on. `include/linux/irqflags.h`, lines 200–213:

```
#define local_irq_disable()             \
    do {                        \
        bool was_disabled = raw_irqs_disabled();\
        raw_local_irq_disable();        \
        if (!was_disabled)          \
            trace_hardirqs_off();       \
    } while (0)

#define local_irq_save(flags)               \
    do {                        \
        raw_local_irq_save(flags);      \
        if (!raw_irqs_disabled_flags(flags))    \
            trace_hardirqs_off();       \
    } while (0)
```

The saved flags are the previous mask, so `local_irq_restore` does not turn interrupts on if they were already off. That is the nest `cli` cannot express with a single bit unless the caller remembers the old IF. `preempt_count` does not move. Disabling the local interrupt mask is not `in_hardirq`.

`spin_lock_irqsave` in `include/linux/spinlock.h`, lines 379–382, takes the lock and saves the interrupt mask together:

```
#define spin_lock_irqsave(lock, flags)              \
do {                                \
    raw_spin_lock_irqsave(spinlock_check(lock), flags); \
} while (0)
```

You use that form when process context and a hardirq handler share the data. The handler runs with IRQs already off on the normal path and takes the lock with the irqsave form or with the knowledge that IRQs are off. Process context must not take the plain `spin_lock` and then be interrupted by the handler that needs the same lock. `spin_lock_bh` is the matching form when the other side is a softirq: it disables bottom halves and, when preemption counts, disables preemption. `SOFTIRQ_LOCK_OFFSET` at line 177 is `SOFTIRQ_DISABLE_OFFSET` plus the preemption-lock offset.

On `CONFIG_PREEMPT_RT`, `PREEMPT_LOCK_OFFSET` is 0 (lines 157–161). RT locks do not disable preemption the way a non-RT spinlock does. The sleep rule you can carry without an RT course is: do not sleep while you hold a spinlock, and do not take a mutex from hardirq, softirq, or NMI.

A mutex sleeps. `down` and `wait_event` sleep. `kmalloc(GFP_KERNEL)` can sleep. `copy_from_user` can fault and sleep, which is why Day 3's fixup path exists and why a hardirq handler does not call it. `queue_work_on` from atomic context is legal when the queue operation itself does not sleep. The function it queues is the thing that might sleep later, in the worker.

8. Who may sleep

Context and sleepability

Context	Sleep	Mutex	Spinlock
Process task	**Yes**	**Yes**	**Yes**
Hardirq	**No**	**No**	irqs already off
Softirq / tasklet	**No**	**No**	spin_lock_bh
NMI	**No**	**No**	NMI-safe only
Threaded IRQ	**Yes**	**Yes**	**Yes**
Worker thread	**Yes**	**Yes**	**Yes**
WQ_BH work	**No**	**No**	same as softirq

might_sleep warns under DEBUG_ATOMIC_SLEEP. in_atomic is a count test.
Holding a spinlock still forbids sleep after the lock is taken.

Process context, a threaded IRQ, and a normal worker may sleep. Hardirq, softirq, NMI, and WQ_BH may not. A spinlock forbids sleep after it is taken.

Context	Sleep	Mutex	Spinlock
Process task	yes	yes	yes, then do not sleep
Hardirq	no	no	with IRQs already off
Softirq or tasklet	no	no	`spin_lock_bh` from process side
NMI	no	no	only an NMI-safe lock
Threaded IRQ	yes	yes	yes, then do not sleep
Worker thread	yes	yes	yes, then do not sleep
`WQ_BH` work	no	no	same as softirq

The lab in section 12 prints five of these rows: hardirq, softirq, `wq_bh`, `worker_thread`, and process. NMI and the threaded-IRQ row are in this table because the source distinguishes them. The lab does not claim to have entered those contexts.

9. `might_sleep` and lockdep

Under `CONFIG_DEBUG_ATOMIC_SLEEP`, `__might_sleep` in `kernel/sched/core.c`, lines 8571–8585, checks `current`'s task state and then calls `__might_resched` with a zero offset:

```
void __might_sleep(const char *file, int line)
{
    unsigned int state = get_current_state();
    WARN_ONCE(state != TASK_RUNNING && current->task_state_change,
            "do not call blocking ops when !TASK_RUNNING; "
            "state=%x set at [<%p>] %pS\n", state,
            (void *)current->task_state_change,
            (void *)current->task_state_change);

    __might_resched(file, line, 0);
}
```

`__might_resched`, lines 8610–8634, returns immediately when the preempt count matches the allowed offset, interrupts are enabled, and a few other quiet conditions hold. Otherwise it prints:

```
BUG: sleeping function called from invalid context at %s:%d
```

That line is the runtime form of this day's table. It is compiled in when the debug option is on. A production kernel can define `might_sleep` as only `might_resched` and never print it. The absence of the print is not permission to sleep.

`in_atomic` is the tempting shortcut and the wrong one for driver code. Lines 179–186:

```
 * Are we running in atomic context?  WARNING: this macro cannot
 * always detect atomic context; in particular, it cannot know about
 * held spinlocks in non-preemptible kernels.  Thus it should not be
 * used in the general case to determine whether sleeping is possible.
 * Do not use in_atomic() in driver code.
 */
#define in_atomic() (preempt_count() != 0)
```

The wrapper's source check looks for that sentence and for `#define in_atomic() (preempt_count() != 0)`. A non-preemptible kernel can hold a spinlock with `preempt_count` still zero. `in_atomic` is then false while sleep is still illegal. Know the context you were called from.

Lockdep is the other witness. `local_irq_save` records hardirqs-off when tracing is configured. `lockdep_assert_irqs_disabled` is what `__raise_softirq_irqoff` calls. Day 5's `lockdep_softirq_start` drops the hardirq lockdep state for the softirq pass so a lock taken there is not recorded as a hardirq lock. Lockdep does not choose the next task. It records the order and the context of locks so a later acquire can warn. Reading a lockdep splat means "this lock was taken in a context that disagrees with an earlier take." It does not mean the scheduler ran.

10. Architecture lens: the mask is not the counter

`preempt_count` is a Linux word. The architectural interrupt mask is a different register. `local_irq_disable` reaches the register through `raw_local_irq_disable`. The three trees in this commit are:

x86. `arch/x86/include/asm/irqflags.h`, `native_irq_disable`, lines 35–37, is `cli`. `arch_local_irq_disable` calls it when the kernel is not using the paravirt indirection. IF is one bit. It is the 0.12 instruction with a C wrapper. It does not store the hardirq nest count. `__irq_enter` does that in software after the CPU has already vectored.

AArch64. `arch/arm64/include/asm/irqflags.h`, `__daif_local_irq_disable`, lines 52–56, executes `msr daifset, #3`, which masks IRQ and FIQ in `DAIF`. `arch_local_irq_disable`, lines 71–77, uses that path or, when priority masking is on, writes `GIC_PRIO_IRQOFF` to `ICC_PMR_EL1`. The second form is how this kernel supports pseudo-NMIs: the CPU still takes a higher priority, and Linux's `preempt_count` is what says the ordinary IRQ handler must not sleep. The system register is not `preempt_count`.

RISC-V. `arch/riscv/include/asm/irqflags.h`, `arch_local_irq_disable`, lines 24–27, clears `SR_IE` in `sstatus` with `csr_clear`. That bit is the supervisor interrupt enable. It is one bit, like IF. The PLIC claim from Day 4 is a device-controller operation that happens after the CPU has entered. Clearing `SR_IE` does not claim the PLIC, and claiming the PLIC does not increment `preempt_count`. `__irq_enter` still does.

Three masks, one Linux counter. A bug report that says "interrupts were disabled" has to say which fact it measured: IF, DAIF, `sstatus.SIE`, or a `preempt_count` field.

11. A trace you can do on paper

Start in process context. `preempt_count` has the preemption field at whatever `preempt_disable` nesting this task already holds. The hardirq, softirq-serving, and NMI fields are clear. `in_task` is true. `local_irq_disable` clears IF, DAIF, or `sstatus.SIE` and does not change those fields.

A device interrupt arrives after `local_irq_enable`. The CPU vectors. `__irq_enter` adds `HARDIRQ_OFFSET`. `in_hardirq` is true. `in_task` is false. The handler calls `raise_softirq_irqoff`. The pending bit changes. `preempt_count` does not gain `SOFTIRQ_OFFSET` yet.

`__irq_exit_rcu` subtracts `HARDIRQ_OFFSET`. If the pending mask is set and the CPU is outside interrupt context, `invoke_softirq` adds `SOFTIRQ_OFFSET` for the action and removes it before returning. `in_serving_softirq` is true only across that action. A `might_sleep` in the action hits `__might_resched` with a nonzero count and, when the debug option is on, prints the invalid-context line.

An NMI during the hardirq calls `__nmi_enter`, which adds `NMI_OFFSET + HARDIRQ_OFFSET` on top of the hardirq count already present. `interrupt_context_level` becomes 3. `__nmi_exit` subtracts the same pair. The hardirq count returns to the value `__irq_enter` set. The paper trace is wrong if the NMI exit clears the original hardirq bit.

`spin_lock_irqsave` from process context saves the mask and takes the lock before the handler can run on this CPU. The handler's `spin_lock` then waits or, if it uses the same irqsave form, runs only after the process releases. Sleeping while that lock is held is illegal even when `in_atomic` is false on a non-preemptible build. That is the comment in section 9, applied to one lock.

The context lab never performs this trace. Its timerfd wait is process context with the architectural mask left to the kernel's syscall entry. The markers say so. Use the lab to see a legal wait. Use this paragraph to see the count move.

12. What this day deliberately does not explain

`schedule`, `pick_next_task`, the run queue, and what `PREEMPT_NEED_RESCHED` does with a sleeping task are Week 15. `do_timer` in 0.12 calling `schedule` is enough history to see that a tick can switch tasks. It is not the 6.12 policy. `might_resched` may be a preemption point. This lesson treats it as "a legal place to ask," and stops.

`need_resched` in Day 5's softirq budget is the same deferral. The softirq pass wakes `ksoftirqd` and returns so the scheduler can run. Which task runs is not this week's question.

The same boundary applies to lockdep. A splat names a lock and two stacks. It does not pick the next task, and it does not change `smp_affinity`. Copy the splat into the Day 7 report only if you actually triggered one. This lesson's wrapper does not.

`CONFIG_PREEMPT_NONE` is still a Kconfig symbol in `kernel/Kconfig.preempt`. The source check requires that line. On that model, `PREEMPT_DISABLE_OFFSET` is zero unless the kernel was built with a count anyway. The hardirq and softirq offsets remain. Sleep is still illegal there. The preemption model changes when the scheduler may enter. It does not make a hardirq handler a place to take a mutex. `config PREEMPT_VOLUNTARY` is the neighboring symbol. It still does not move the hardirq bits.

13. Safe evidence

The context lab at `projects/linux-kernel-contexts/src/context_lab.c` prints five rule lines, polls an empty `eventfd` with timeout 0, and polls a 20 ms `timerfd` with a 500 ms cap. The markers it prints on success are `zero_poll_immediate=yes`, `blocking_poll_waited=yes`, `userspace_poll_is_not_kernel_context=yes`, `kernel_preempt_claim=none`, and `scheduler_dive=deferred_to_week_15`. The zero-timeout poll returns without waiting. The timerfd poll waits. Both are process context. Neither reads `preempt_count`.

Run `../challenges/day-006-context-rules.sh`. The wrapper builds that lab and runs `check-source.sh` for day 6, which requires `PREEMPT_BITS` 8, `HARDIRQ_BITS` 4, `in_hardirq`, the `in_atomic` warning, `__irq_enter`'s `HARDIRQ_OFFSET` add, `might_sleep`, and `config PREEMPT_NONE`. It does not write the kernel tree.

Predict the five `rule_*_may_sleep` lines before you run it. Hardirq, softirq, and `wq_bh` are no. The worker and process are yes. Then predict that the zero timeout returns immediately and the timerfd wait takes time.

14. Mastery: Explain · Draw · Observe · Build

Explain

Linux 0.12 closes IF with `cli` and calls `schedule` from `do_timer` only when the interrupted CPL is not zero. v6.12 records hardirq with `HARDIRQ_OFFSET` inside `preempt_count`. `in_serving_softirq` is bit 8. `local_bh_disable` uses twice that offset, so BH-disabled process context is not serving a softirq. `__nmi_enter` adds `NMI_OFFSET` and `HARDIRQ_OFFSET`. `might_sleep` prints under `DEBUG_ATOMIC_SLEEP`. `in_atomic` is the wrong driver test because a non-preemptible spinlock may not move the count. `cli`, `msr daifset, #3`, and `csr_clear` of `SR_IE` are the architectural masks. They are not the counter.

Draw

1. The preempt_count word with the four fields and bit 31 named but not explained as a scheduler.
2. A stack: process, then hardirq (`HARDIRQ_OFFSET`), then softirq on the way out (`SOFTIRQ_OFFSET`), then the hardirq bit cleared.
3. The sleep table, with `WQ_BH` on the softirq row and a threaded IRQ on the process row.

Observe

From the wrapper, copy the five rule lines and the two poll markers. Record that `kernel_preempt_claim=none`. If the source check prints `commit=adc218676eef25575469234709c2d87185ca223a`, keep that beside `uname -r`.

Build

Pick one function you would call from a device driver: `mutex_lock`, `queue_work_on` onto `system_bh_wq`, or `kfree` of a small object. Write the context it may be called from, the `preempt_count` field that would be nonzero, and whether `might_sleep` would be a legal call. One paragraph. Do not describe `pick_next_task`.

Why? notebook

1. Why does 0.12 `do_timer` return immediately when `cpl` is 0?
2. Why is `cli` not a nest count?
3. Why does `local_bh_disable` add `2 * SOFTIRQ_OFFSET`?
4. Why is `in_serving_softirq` false in BH-disabled process context?
5. Why does `__nmi_enter` add `HARDIRQ_OFFSET` as well as `NMI_OFFSET`?
6. Why does the `in_atomic` comment forbid driver use?
7. Why can `might_sleep` be silent on a kernel built without `DEBUG_ATOMIC_SLEEP`?
8. Why is `msr daifset, #3` not a write to `preempt_count`?
9. Why does the context lab's timerfd wait not prove a `preempt_count` value?

Next: Day 7 collects what you can show from `/proc`, from the source hooks, and from an optional strace, and it keeps a guess out of the report.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Context Rules

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 14-kernel-entry-contexts-and-interrupts/challenges/day-006-context-rules.sh
```

- View lab source on GitHub

Day 7 — Entry, Interrupt Tracing, and a Debugging Report

Target time: approximately 3 hours

- What Linux 0.12 could print, and what `/proc/interrupts` counts: 35 minutes
- Source hooks versus a host observation: 50 minutes
- A report that keeps a guess out of the evidence: 55 minutes
- Mastery, then the wrapper: 25 minutes

> You want evidence of one IRQ handler. Which file on this machine can show you a count, which source line is the hook, which tool printed `SKIP`, and which sentence would be a guess if you wrote it anyway?

Days 2 through 6 named entries: syscall, page fault, device IRQ, softirq, and the `preempt_count` fields. This day is how you show your work. A count, a source line, and a trace are different kinds of evidence. The report says which kind you have.

The 6.12 excerpts are from `/home/jd/Documents/linux-6.12`, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`. Record `uname -r` beside that id. The wrapper checks the commit and does not modify the tree.

1. Syntax needed for this source

`/proc/interrupts` is text. A numbered line has a count per CPU, a chip name, and one or more action names. A comma joins shared actions. A line whose first field is a word, such as `NMI` or `LOC` on x86, comes from the architecture's extra summary, not from `irq_desc->action`.

A tracepoint in C is an ordinary call:

```
trace_irq_handler_entry(irq, action);
res = action->handler(irq, action->dev_id);
trace_irq_handler_exit(irq, action, res);
```

If the tracer is off, the call records nothing you can read back. If a tracer is on, the same line is the record. Seeing the call in the source proves the hook exists. It does not prove a tracer ran.

`strace` prints syscalls. `clock_nanosleep` is a process entry. The timer interrupt that wakes the sleeper is a different entry. A report that uses strace to name an IRQ handler has changed the subject.

2. Linux 0.12 first: printk, not `/proc/interrupts`

Linux 0.12 has no ftrace, no `trace_irq_handler_entry`, and no `/proc/interrupts` in the modern format. The observation tools are `printk` and the fact that `ret_from_sys_call` is one place to look when a syscall or a timer does not return. You edit the source, rebuild, and reboot. That is workable when the kernel is a few dozen files. It is a poor way to ask which IRQ ran on a machine you are not willing to rebuild.

`_unexpected_hd_interrupt` is the 0.12 version of a spurious-interrupt report: the stub found `do_hd` NULL and called a function whose job is to say so. There is no per-CPU counter and no comma-separated name list. The durable idea is to separate the path from the observation. The path is the stub. The observation is whatever you printed. A print you did not add is not evidence.

3. Evolution: counters, tracepoints, and a syscall observer

Rebuilding to add a `printk` does not scale, and a `printk` inside a hardirq handler can itself be the bug. The kernel grew a counter file and tracepoints on the handler path.

`/proc/interrupts` is the counter view from Day 4. `kernel/irq/proc.c`, `show_interrupts`, walks irq numbers and prints `kstat_irqs` per online CPU, the chip name, the hwirq when a domain exists, and the action names. That file answers "how many times did this Linux irq run on this CPU, and what is it called?" It does not answer "what did the handler do" or "which instruction was interrupted."

Tracepoints sit on the path. `trace_irq_handler_entry` is inside `__handle_irq_event_percpu`. `trace_softirq_entry` is inside `handle_softirqs`. `kernel/trace/trace_irqsoff.c` holds a tracer for how long interrupts stayed off. Turning that tracer on writes tracefs and changes timing. This week's wrapper does not write it.

`strace` is older than those tracers and answers a different question. It is a ptrace observer of the syscalls one process made. It shows `clock_nanosleep`. It does not show `SYSRET` versus `IRET`, and it does not show an IRQ the process did not invoke. `perf` can count events when the kernel permits it. When ptrace or perf is blocked, the honest result is `SKIP`.

4. ToyOS logs to the serial line

ToyOS writes `SELFTEST` and `EXCEPTION` from `interrupt_handler` in `projects/toyos/src/arch.c`. The page-fault path prints `CR2` and the error-code bits. That is the 0.12 method: a print on the path. The serial line is not ftrace. A ToyOS log that says vector 32 ran is evidence about ToyOS. It is not evidence about `trace_irq_handler_entry` in Linux 6.12. Keep the two logs in two paragraphs of the report.

5. What `show_interrupts` actually prints

`kernel/irq/proc.c`, lines 463–526, is the function behind `/proc/interrupts` for the numbered lines. The header is one column per online CPU. Each later line that has an action prints the irq number, the per-CPU counts, the chip, and the names:

```
/* tree adc218676eef (Linux 6.12)
 * path: kernel/irq/proc.c
 * lines: 497–526
 * function: show_interrupts
 */
    seq_printf(p, "%*d: ", prec, i);
    for_each_online_cpu(j)
        seq_printf(p, "%10u ", desc->kstat_irqs ? per_cpu(desc->kstat_irqs->cnt, j) : 0);
    /* chip name, then hwirq if a domain is attached */
    action = desc->action;
    if (action) {
        seq_printf(p, "  %s", action->name);
        while ((action = action->next) != NULL)
            seq_printf(p, ", %s", action->name);
    }
```

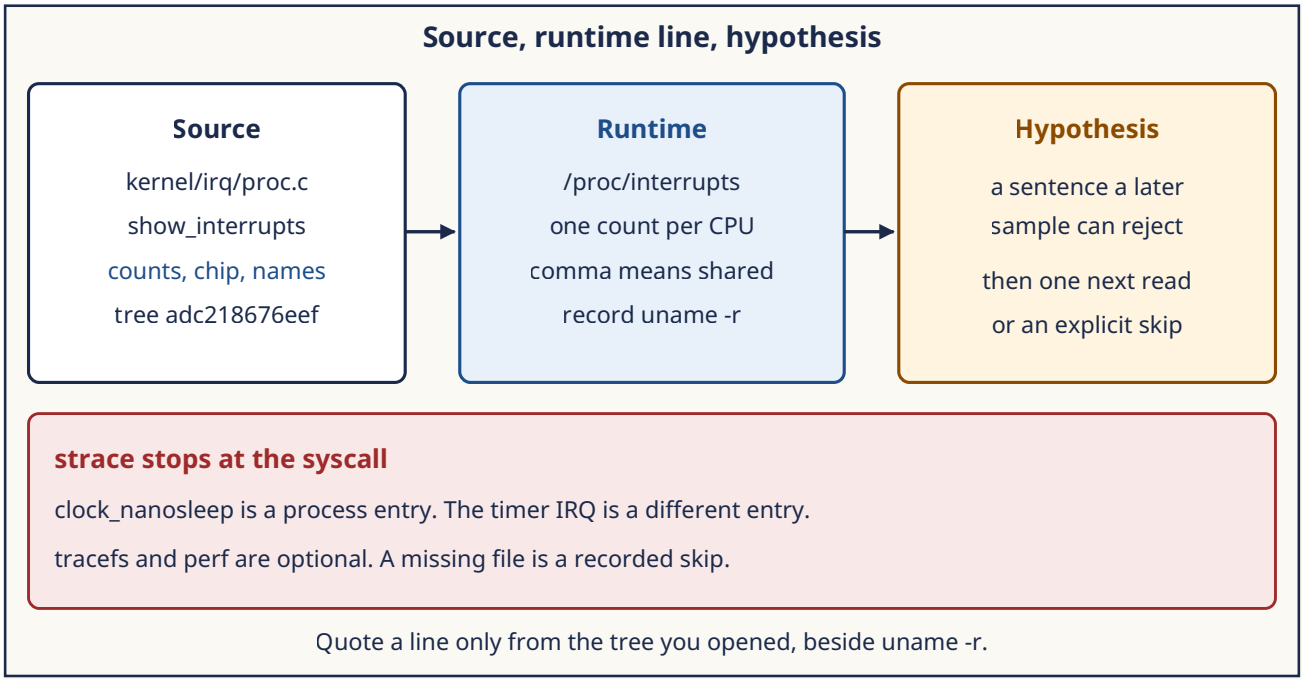
A hidden descriptor, a chained irq, or a descriptor with no action is skipped. A zero count is still a line if an action is registered. The count is `unsigned int` per CPU in this struct. It wraps. Two samples close together can both show a large number. The difference is the evidence, and only if you recorded which CPU column you subtracted.

The architecture appends its own lines when the index reaches `ACTUAL_NR_IRQS`. `arch_show_interrupts` is weak and returns 0 in the generic file. x86 overrides it. `arch/x86/kernel/irq.c`, lines 61–73:

```
int arch_show_interrupts(struct seq_file *p, int prec)
{
    int j;

    seq_printf(p, "%*s: ", prec, "NMI");
    for_each_online_cpu(j)
        seq_printf(p, "%10u ", irq_stats(j)->__nmi_count);
    seq_puts(p, "  Non-maskable interrupts\n");
#ifdef CONFIG_X86_LOCAL_APIC
    seq_printf(p, "%*s: ", prec, "LOC");
    for_each_online_cpu(j)
        seq_printf(p, "%10u ", irq_stats(j)->apic_timer_irqs);
    seq_puts(p, "  Local timer interrupts\n");
```

`NMI` here is `__nmi_count`. `LOC` is the local APIC timer. They are not `irq_desc` lines and they have no `dev_id`. A rising `LOC` means the local timer is firing. It does not mean a device wire is stuck. Other architectures print their own names from their own `arch_show_interrupts`. Read the label on the line you have. Do not import the x86 names onto a machine whose file does not contain them.



show_interrupts produces one runtime line. A hypothesis is a sentence a later sample can reject. strace stays on the syscall side of the figure.

6. Hooks you can name without enabling them

The day-7 source check looks for these strings and fails if they are absent. They are the names a report may cite as source, after the check has accepted the commit.

`kernel/irq/handle.c` places the handler between `trace_irq_handler_entry(irq, action)` and `trace_irq_handler_exit(irq, action, res)`. That is the per-action hook from Day 4. Citing it says "the tree contains the hook." Citing it as "I saw handler X run at time T" requires a trace you captured.

`kernel/trace/trace_irqsoff.c` contains `static struct trace_array *irqsoff_trace`. That is the irqsoff tracer state. `Documentation/trace/ftrace.rst` begins with the title `ftrace - Function Tracer`. The document describes how to use the tracer. The wrapper does not follow those steps.

`arch/x86/entry/entry_64.S` defines `SYM_CODE_START(entry_SYSCALL_64)`. That is Day 2's syscall entry. `arch/x86/mm/fault.c` defines `DEFINE_IDTENTRY_RAW_ERRORCODE(exc_page_fault)`. That is Day 3's fault entry. A report about an IRQ that also mentions those symbols has to say why they are in scope. They are in the source check because this week's entries share a report. They are not IRQ handlers.

The check prints `source_modified=no`. It does not mount tracefs and it does not write a tracer file.

7. strace stops at the syscall

Take `sleep`, or the lab's `timerfd` wait from Day 6. `strace` can show `clock_nanosleep` or `poll` and the time it blocked. That is the process asking to wait. The wake may be a timer interrupt, a softirq, and a scheduler decision. strace's line does not name `TIMER_SOFTIRQ`, `LOC`, or `apic_ack_irq`.

Use strace when the question is "which syscall did this process enter, and what did it return?" Use `/proc/interrupts` when the question is "which Linux irq count moved?" Use the source when the question is "which function would run if that irq fired?" Write the question down before you pick the tool. A tool that cannot answer the question still produces output. Output is not an answer.

If `strace` fails because ptrace is restricted, the wrapper's optional path prints `SKIP`. Keep the skip. Do not describe a syscall trace you do not have. The same rule applies to `perf`. `perf stat` counts events when it is allowed to. A permission error is a skip. It is not a zero count.

8. Optional tracefs, left off

`/sys/kernel/tracing` or the older debugfs path may be readable, and it may not be. Readable means you can see the files. It does not mean you should write `current_tracer`. Enabling `irqsoff` or a function tracer changes timing on the machine and is outside this week's wrapper. The report lab prints `tracefs_written=no` for that reason.

If the directory is missing, write `tracefs=skipped` and the reason "not mounted" or "not readable." If it is readable, write `tracefs=present` and `enabled=no`. Both are complete observations. A paragraph that describes events you would have seen is a hypothesis that escaped into the results.

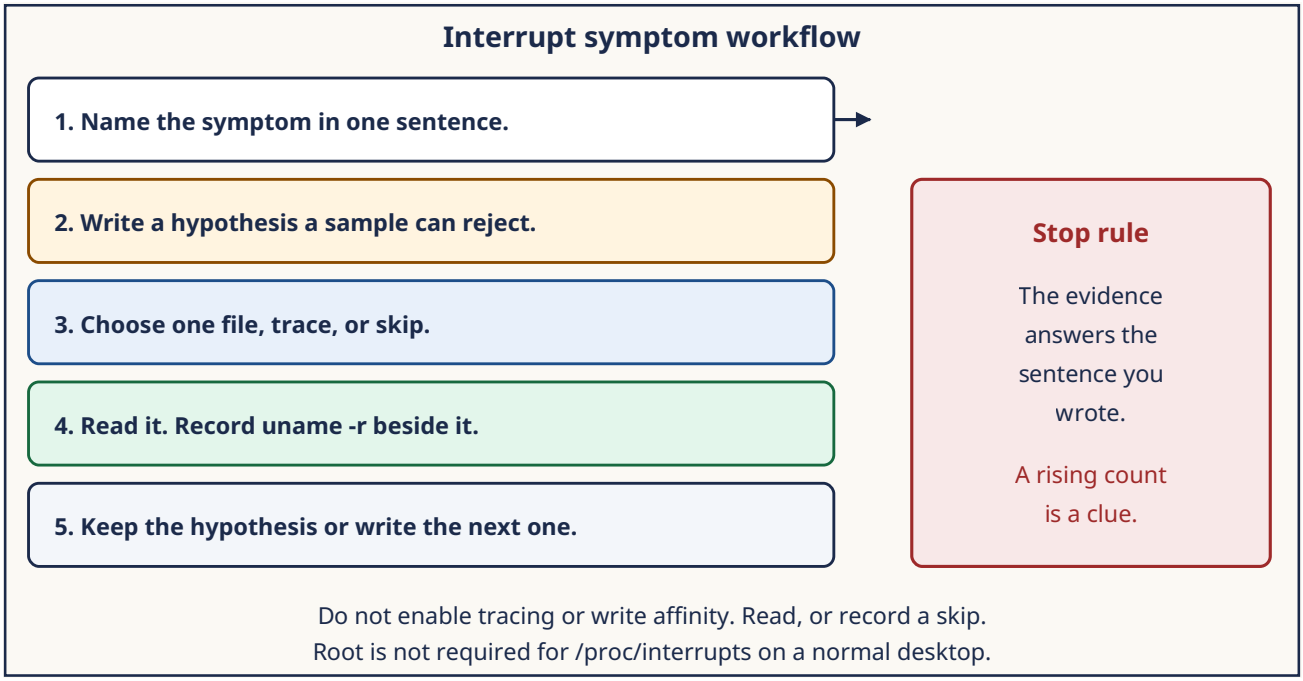
The events you are allowed to name from source, without claiming they fired, include `irq_handler_entry`, `irq_handler_exit`, `softirq_raise`, and `softirq_entry`. Day 5 raised `TIMER_SOFTIRQ` through `raise_softirq_irqoff`, which calls `trace_softirq_raise`. That call is in the tree. A captured trace would show it. You do not have that capture unless your log says you do.

9. Hypotheses that a sample can reject

A hypothesis is one sentence a measurement can make false. These are the shapes that fit this week. They are examples of form. They are not claims about your machine until you fill in a number you read.

- "The comma on irq N joins names A and B, so those two actions share one `irq_desc`." A second look at `/proc/interrupts` can show the comma gone, which rejects the sharing claim for that boot.
- "Column CPU2 of irq N increased between sample 1 and sample 2, so that CPU ran the flow handler at least once." A flat column rejects it. The column does not say which action returned `IRQ_HANDLED`.
- "`LOC` increased, so a device driver on a numbered line is stuck." This one is already false as a form: `LOC` is the local timer in the x86 summary. Reject it by reading the label, before you sample.
- "`ksoftirqd/0` exists, so a softirq ran during my sample window." The task existing only shows the thread was created. The `/proc/softirqs` delta is the measurement that can reject "it ran just now."
- "strace shows `clock_nanosleep`, so I observed the timer IRQ handler." strace did not print the handler. The sentence is not a hypothesis about the log you have. It is a change of subject.

Write the sentence, then the file you will read, then what result would make the sentence false. If you cannot name the rejecting result, you do not have a hypothesis yet.



Name the symptom, write a sentence a sample can reject, read one source, record uname, then keep or replace the sentence.

10. Architecture lens: the summary lines are not portable

The numbered `irq_desc` lines are the generic part. The trailer is `arch_show_interrupts`. On x86 the trailer includes `NMI`, `LOC`, and, when SMP is on, `RES` and `CAL` for reschedule and call interrupts. Those counts live in `irq_stats`, not in `kstat_irqs`. A GICv3 machine prints IPIs and error counts from the arm64 implementation of the same weak function. A RISC-V machine prints its own names. The lesson is the same one Day 4 used for EOI: identify the file and the label before you interpret the number.

`/proc/softirqs` is more portable across architectures because the rows are the Linux enum: `HI`, `TIMER`, `NET_RX`, `TASKLET`, `SCHED`, `RCU`, and the rest. The columns are still CPUs. A `TIMER` increase does not say whether the tick came from an 8259, a local APIC, or a GIC interrupt. It says the softirq action ran. Pair it with the interrupt file when you need the controller.

11. The report lab, and what it refuses to claim

`projects/linux-kernel-contexts/src/report_lab.c` prints `uname`, checks that `/proc/interrupts` opens and that the first line contains `CPU`, and looks for `voluntary_ctxt_switches:` in `/proc/self/status`. A missing file becomes `unreadable` or `absent`. The program still exits 0. It then prints:

```
tracefs_written=no
modules_loaded=no
network_used=no
kernel_source_modified=no
scheduler_dive=deferred_to_week_15
mastery_claim=none
```

`voluntary_ctxt_switches` is a field name. The number, if you read it yourself, says this task has given up the CPU on purpose some number of times. It is not a run-queue diagram. Week 15 is where that number gets a mechanism. The lab does not print the count. It prints whether the field is present. Do not promote presence into a scheduling result.

`mastery_claim=none` means the program did not grade you. The report you write is the grading material. A passing wrapper is evidence that the tree and the host commands ran. It is not a filled-in trace.

The optional `make tools` target in that project may run strace or perf. Those lines count only when the tool actually runs. A `SKIP` stays a skip. The wrapper itself tells you that, and it points at `make tools` as optional. You do not need root, and you do not need that target, to finish the report.

12. Safe evidence

Run `../challenges/day-007-entry-debug-report.sh`. It builds the report lab and runs the day-7 source check. Predict before you run: `sysname` from `uname`, and whether `/proc/interrupts` opened for you on Day 4. After it runs, copy `release=`, the `proc_interrupts=` line, the context-switch field line, and `tracefs_written=no`.

If you also run the optional tools and they skip, copy the skip line. Do not replace it with a description of `entry_SYSCALL_64` from memory and call that a trace. You may cite `entry_SYSCALL_64` as a source symbol the check found. Label it `source`, not `captured`.

13. Two samples, and a labeled example that is not your host

Take two reads of `/proc/interrupts` a few seconds apart. Write the clock time next to each. Pick one numbered line and one CPU column. Subtract. A positive difference means `kstat_irqs` on that CPU increased. It means `desc->handle_irq` ran, or at least the flow handler reached `kstat_incr_irqs_this_cpu`, which Day 4's level and fast-EOI paths do before the driver. It does not mean every action on a shared line returned `IRQ_HANDLED`. The comma tells you the names. The status registers inside the devices tell you which one fired. You do not have those registers from this file.

A zero difference rejects "this irq ran on this CPU during the window." It does not reject "the device is alive on another CPU." Read the other columns before you widen the claim. Affinity from Day 4 is why one column can move while its neighbor stays still.

A wrapped `unsigned int` can make a later sample look smaller. If the second number is less than the first, say "the counter went backwards or wrapped" and do not treat the subtraction as a rate. You do not need a second tool to notice that. You need both numbers written down.

The trailer lines follow the same arithmetic and a different struct. On x86, `LOC` is `apic_timer_irqs` in `irq_stats`, printed by `arch_show_interrupts`. Subtracting `LOC` answers "did the local timer run?" Subtracting a numbered line answers "did this `irq_desc` run?" Mixing them is how a timer looks like a stuck device.

`/proc/softirqs` uses the same column layout for a different counter. `kstat_incr_softirqs_this_cpu` in `handle_softirqs` is the increment. A `TIMER` delta says the timer softirq action ran. It does not name the irq that raised it. Pair the two files when the question needs both. Leave the second file out when the question is only the irq line.

This example is a form. The numbers are not from a machine you can cite. Replace every integer with a number you read before the line counts as an observation.

```
EXAMPLE, not a host log
uname -r: recorded beside the file, not invented here
sample1 irq 16 CPU0: 1000   name: ahci
sample2 irq 16 CPU0: 1004   name: ahci
delta: 4
hypothesis: CPU0 ran this irq_desc's flow during the window
reject if: the second count equals the first
kept: the delta is positive, so the flow ran at least once
not claimed: which command the adapter finished
strace: not run
tracefs: enabled=no
```

The example keeps four sentences apart. The delta is a count. The hypothesis is the sentence the delta supports. The adapter command is outside the file. strace and tracefs are explicit non-claims. Your report replaces `1000` and `1004` and `ahci` with the line you actually copied. If your second count equals the first, you write "rejected" and stop. You do not explain what the device would have done if the count had moved.

A second example, still not a host log, shows a label hypothesis that needs only one sample:

```
EXAMPLE, not a host log
line: "16:  1000  0  IR-PCI-MSI  ...  ahci, nvme"
hypothesis: one irq_desc has two actions
reject if: the line has no comma
kept by the comma, for this sample only
not claimed: both devices fired
```

Day 4's `show_interrupts` loop prints that comma from `action->next`. One sample is enough for the sharing claim. It is not enough for a rate. The report labels which claim it is making.

Shared-line symptoms follow the same split. "The line is shared" is the comma. "The wrong driver is returning `IRQ_HANDLED`" is not in the file. You would need the handler source, or a trace of `irq_handler_exit` that you captured, to say which return value won. Until then the report says the comma and stops.

Affinity symptoms follow the columns. "CPU3's column is the only one moving" supports "recent deliveries went to CPU3." It does not by itself say the affinity mask contains only CPU3. The mask is `/proc/irq/N/smp_affinity` if that file is readable. This week's wrappers do not write it. Reading it is optional and still not a change. If you do not read it, do not describe the mask.

Storm symptoms are a large delta, not a story about a missing EOI. A large delta means the flow handler ran many times. Day 4's level path masks before the driver and unmasks after. A driver that fails to clear a level source can make the next unmask return immediately. That mechanism is a candidate. It becomes a conclusion only when you can point at the handler and the device status. The count by itself is the candidate's motive, not the verdict.

Work the five steps on that candidate so the workflow figure is a procedure.

1. Symptom: "irq 16's count climbed by tens of thousands in one second." Write the two numbers. If you do not have them, this is not yet a symptom you measured.
2. Hypothesis: "the level source stayed asserted across unmask." Name the rejecting result: a handler that clears the device status, or a delta that stops when the device is idle. "The kernel is broken" is not a hypothesis. It has no rejecting sample.
3. Evidence: the `/proc/interrupts` delta, the chip name on that line, and `handle_level_irq` or `handle_fasteoi_irq` from the tree if you opened it. One of those, not all of the week's files.
4. Read and record `uname -r`. If the chip name is `IR-PCI-MSI`, say so. MSI often has no level wire. A level-unmask story then needs a different chip, or you reject the hypothesis because the line is not a level wire.
5. Keep or replace. A rejected level story becomes the next sentence: "this line is message-signaled, so the next hypothesis is about the device posting another MSI." Stop when the evidence answers the sentence you wrote. Do not enable tracefs to decorate a conclusion you already cannot support.

The stop rule is the right-hand box of the workflow figure. A climbing count is a clue. It is not permission to load a module, write affinity, or turn on `irqsoff`. Those actions change the machine. This week's evidence is a read.

If step 4's chip name is not MSI and not a level flow you can name from the tree, write "flow handler not identified" and stop. Guessing `handle_level_irq` because the count was large repeats the mistake the example is built to prevent. The figure's stop rule applies to a missing fact the same way it applies to a rejected sentence. One sample still cannot support a rate. Two samples that name different CPUs are two hypotheses. Subtract each column on its own. A column you did not copy is a column you cannot talk about. The header row names the CPUs. Subtract only a column whose name you copied.

14. Write the report

Use this outline. Leave a line blank when the log is silent. A blank line is better than a sentence you cannot point at.

1. Commit the landmark script accepted, and `uname -r` from the report lab. If they differ in the release string, say so. The tree id is the source. `uname` is the host.
2. One numbered `/proc/interrupts` line: irq, one CPU column, chip, name. If a comma is present, quote it and apply Day 4. If you did not sample twice, do not claim the count moved.
3. One architecture trailer line, or the sentence "this file's trailer does not use the x86 names" if `NMI` and `LOC` are absent.
4. One `/proc/softirqs` row from Day 5, labeled as a softirq count, not as a device.

5. The source hook you are willing to cite: `trace_irq_handler_entry` in `kernel/irq/handle.c`, labeled source.
6. strace or perf: `PASS` with the syscall you saw, or `SKIP`, or "not run." `clock_nanosleep` does not name an IRQ handler.
7. tracefs: `tracefs_written=no`, plus whether the directory was readable. Do not describe events.
8. One hypothesis in the form from section 9, and the observation that keeps it or rejects it. If you only have one sample, the honest hypothesis is about the label, not about a delta.
9. One sentence that Week 15, not this report, explains which task ran after a sleep. `scheduler_dive=deferred_to_week_15` is that sentence in the lab output.

A report that contains all nine, with blanks where the log is silent, is finished. A report that explains an irqsoff trace you did not capture is not.

15. Mastery: Explain · Draw · Observe · Build

Explain

Linux 0.12 observes with `printk` and a named unexpected-interrupt function. v6.12's `show_interrupts` prints per-CPU `kstat_irqs`, the chip, and action names joined by commas. x86 `arch_show_interrupts` adds `NMI` and `LOC` from `irq_stats`. `trace_irq_handler_entry` is a hook beside the handler call. strace prints syscalls. `tracefs_written=no` means this lab did not enable a tracer. `in_atomic` and `preempt_count` are Day 6. They do not appear in `/proc/interrupts`.

Draw

1. The evidence figure: source file, one `/proc` line, one hypothesis, and strace in its own box.
2. The five-step workflow with your actual symptom in step 1. If you have no symptom, use "I do not yet know whether irq N is shared."
3. Two columns, `source` and `captured`, and place `trace_irq_handler_entry`, `LOC`, and `clock_nanosleep` in the column they belong to after your run.

Observe

From the wrapper, record `release`, `proc_interrupts`, the context-switch field marker, and `mastery_claim=none`. Record one source-check line that names `trace_irq_handler_entry` or `irqsoff_trace`. Keep `SKIP` lines if you ran the optional tools.

Build

Write the nine-line report from section 13 in your notes. Every line that states a fact names the file or the marker it came from. The hypothesis line includes the result that would reject it.

Why? notebook

1. Why is a `printk` you did not add absent from the evidence?
2. Why does a comma in `/proc/interrupts` not prove both devices fired?
3. Why is a rising `LOC` a local-timer fact on x86, and a fact you must not import onto a file that has no `LOC` line?
4. Why does `trace_irq_handler_entry` in the source not mean you captured a trace?

5. Why does strace of `clock_nanosleep` not name `apic_ack_irq`?
6. Why is a readable tracefs directory still `enabled=no` in this lab?
7. Why is `voluntary_ctxt_switches` present not a scheduler explanation?
8. Why does the report lab print `mastery_claim=none`?
9. Why does a single sample support a label hypothesis and not a delta hypothesis?

Next: Week 15 takes `PREEMPT_NEED_RESCHED`, `schedule`, and the run queue. The entries, the irq descriptor, the deferred contexts, and the sleep rules now have a path you can cite from this tree and a report shape that keeps a guess labeled as a guess.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Entry Debug Report

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 14-kernel-entry-contexts-and-interrupts/challenges/day-007-entry-debug-report.sh
```

- View lab source on GitHub